

CTF Crypto: 从入门到进阶

Triode

目录

1	前言	1
2	基础知识	3
2.1	初等数论	3
2.1.1	整除与最大公约数	3
2.1.2	二元一次不定方程与扩展欧几里得算法	5
2.1.3	质数、同余与素性测试	5
2.1.4	同余式与孙子定理	8
2.1.5	连分数	11
2.2	线性代数	13
2.2.1	矩阵与行列式	13
2.2.2	线性变换	13
2.2.3	向量空间	13
2.3	抽象代数	14
2.3.1	集合	14
2.3.2	代数系统	14
2.3.3	群	14
2.3.4	环	14
2.3.5	域	14
3	对称密码	15
3.1	流密码	15
3.2	分组密码	15
4	非对称密码	17
4.1	RSA	17
4.1.1	直接分解模数 n	18
4.1.2	Wiener 攻击 [10]	26
4.1.3	共模攻击	28
4.1.4	相同明文的广播攻击	31
4.1.5	相关消息攻击 [11]	32
4.1.6	e 与 $\varphi(n)$ 不互质	37
4.2	椭圆曲线密码学	40
4.3	数字签名算法	41
4.4	格密码	42
4.4.1	格基规约算法	43

4.4.2	解决最近向量问题	44
4.4.3	背包问题 (Knapsack Problem)	46
4.4.4	隐藏数问题 (Hidden Number Problem, HNP)	51
4.4.5	近似公约数问题 (Approximate Common Divisor Problem, ACD Problem) [20] . .	60
4.4.6	容错学习问题 (Learning with Errors, LWE)	61
4.4.7	NTRU	67
4.4.8	隐子集和问题 (Hidden Subset Sum Problem) [23]	77
4.5	同源密码学	81
4.5.1	前置知识 [24, 25, 26]	81
4.5.2	同源	82
4.5.3	超奇异同源 Diffie-Hellman 密钥交换体系 (SIDH)	86
Appendices		97
A 附录		97

Chapter 1

前言

Chapter 2

基础知识

2.1 初等数论

由于现代密码学多数以数论为基础，所以在学习密码学之前，我们需要了解一些初等数论的知识，在这里对理论知识只作简要的介绍，若想深入学习初等数论相关知识，推荐阅读《数论概论》([美]Joseph H. Silverman 著) 或者《初等数论》(闵嗣鹤、严士健编)。

2.1.1 整除与最大公约数

整除是数论中的基本概念，所以在学习初等数论之前，我们需要了解整除的定义。

整除的定义

设 a, b 是任意两个整数 (其中 $b \neq 0$)，若存在一个整数 q 使得等式

$$a = bq$$

成立，则称 a 被 b 整除 (或称 b 整除 a)，记作 $b|a$ ，此时称 b 为 a 的因数， a 为 b 的倍数。反之，若整数 q 不存在则称 a 不能被 b 整除 (或称 b 不能整除 a)，记作 $b \nmid a$ 。

下面简要介绍一些整除相关的定理：

- 若 $b|a$ 且 $c|b$ ，则有 $c|a$
- 若 $a_1, a_2, \dots, a_n$ 都是 m 的倍数，而 $k_1, k_2, \dots, k_n$ 是任意 n 个整数，则 $k_1a_1 + k_2a_2 + \dots + k_na_n$ 也是 m 的倍数
- (带余除法) 若 a, b 是任意两个整数 (其中 $b > 0$)，则存在唯一的一对整数 q, r 使得等式 $a = bq + r$ 成立，其中 $0 \leq r < b$ 称为余数， q 称为不完全商 (有时也简称为商)

整除的概念以及相关定理在数论乃至在密码学中都是十分重要的。

在简要了解了整除的概念之后，我们将要了解一个重要的概念——最大公约数。

公约数与最大公约数

设 $a_1, a_2, \dots, a_n$ 是 $n(n \geq 2)$ 个整数, 若存在一个整数 d 使得 $d|a_1, d|a_2, \dots, d|a_n$, 则称 d 是 $a_1, a_2, \dots, a_n$ 的一个公约数.

在 $a_1, a_2, \dots, a_n$ 的所有公约数中, 最大的一个称为 $a_1, a_2, \dots, a_n$ 的最大公约数, 记作 $(a_1, a_2, \dots, a_n)$, 而若 $(a_1, a_2, \dots, a_n) = 1$, 则称 $a_1, a_2, \dots, a_n$ 互素, 若 $a_1, a_2, \dots, a_n$ 中每两个数互素, 则称 $a_1, a_2, \dots, a_n$ 两两互素.

在了解了最大公约数的概念之后, 我们需要首先了解最大公约数的一些性质:

- 若 $a_1, a_2, \dots, a_n$ 为任意 n 个不为 0 的整数, 则 $a_1, a_2, \dots, a_n$ 的公约数与 $|a_1|, |a_2|, \dots, |a_n|$ 的公约数相等, 且 $(a_1, a_2, \dots, a_n) = (|a_1|, |a_2|, \dots, |a_n|)$
- 若 a 是一个非零整数, 则 $(0, a) = |a|$
- 设 a, b, c 是任意三个不全为 0 的整数, 且有 $a = bq + c$ (其中 $q \neq 0$), 则 $(a, b) = (b, c)$

由上面提到的性质我们可以得出一系列等式:

$$\begin{aligned} a &= bq_1 + r_1, & 0 \leq r_1 < b \\ b &= r_1q_2 + r_2, & 0 \leq r_2 < r_1 \\ r_1 &= r_2q_3 + r_3, & 0 \leq r_3 < r_2 \\ &\dots \\ r_{n-2} &= r_{n-1}q_n + r_n, & 0 \leq r_n < r_{n-1} \\ r_{n-1} &= r_nq_{n+1} + r_{n+1}, & r_{n+1} = 0 \end{aligned}$$

根据这一系列等式我们可以知道对于任意两整数 a, b , (a, b) 就是这一系列等式中的最后一个非零余数 r_n , 这一系列等式就是著名的辗转相除法¹, 也称为欧几里得算法. 欧几里得算法的流程如下:

Algorithm: 欧几里得算法 $\gcd(a, b)$

Input: a, b

Output: (a, b)

```
while  $b \neq 0$  do
   $r \leftarrow a \bmod b$ 
   $a \leftarrow b$ 
   $b \leftarrow r$ 
end
return  $a$ 
```

幸运的是, Python 中的 `math`, `gmpy2` 等库都提供了计算最大公约数的函数 (一般为 `gcd`), 我们可以直接调用这些函数来计算最大公约数.

在整除和最大公约数的基础上, 我们可以进一步得到一个重要的定理:

裴蜀定理

设 a, b 是任意两个不全为 0 的整数, 则对于任意整数 x, y , 都有

$$(a, b) | ax + by$$

¹辗转相除法与更相减损术 (《九章算术·卷一》中提及的“约分术”: 可半者半之; 不可半者, 副置分母、子之数, 以少减多, 更相减损, 求其等也. 以等数约之) 略有不同, 辗转相除法的效率会略优于更相减损术

通过裴蜀定理定理，我们可以知道对于任意两个不全为 0 的整数 a, b ，都存在整数 x, y 使得 $ax + by = (a, b)$ ，这个结论在密码学中有着重要的应用。

2.1.2 二元一次不定方程与扩展欧几里得算法

所谓二元一次不定方程，就是指形如 $ax + by = c$ 的方程，其中 a, b, c 是已知的整数， x, y 是未知的整数，在密码学中通常要求解这类问题，由裴蜀定理我们很容易可以知道形如 $ax + by = c$ 的方程存在整数解的充要条件是 $(a, b) | c$ ，而如若知道方程 $ax + by = c$ 的一组整数解为 $(x, y) = (x_0, y_0)$ ，我们很容易可以知道这组方程的所有解为可以表示为 $(x, y) = (x_0 - bt, y_0 + at)$ (其中 $t \in \mathbb{Z}$)。所以我们的问题就转化为了求解 $ax + by = (a, b)$ 的其中一组整数解，为求解这一方程，我们可以使用扩展欧几里得算法。

扩展欧几里得算法的流程如下：

Algorithm: 扩展欧几里得算法 $\text{extgcd}(a, b, x, y)$

```

if  $b = 0$  then
     $x \leftarrow 1$ 
     $y \leftarrow 0$ 
    return  $a$ 
end
 $d \leftarrow \text{extgcd}(b, a \bmod b, x, y)$ 
 $x_0 \leftarrow x$ 
 $y_0 \leftarrow y$ 
 $x \leftarrow y_0$ 
 $y \leftarrow x_0 - \lfloor \frac{a}{b} \rfloor y_0$ 
return  $d$ 

```

通过使用扩展欧几里得算法，我们可以在求 a, b 的最大公约数的同时求出方程 $ax + by = (a, b)$ 的一组整数解 (x_0, y_0) ，从而可以求解二元一次不定方程 $ax + by = c$ 的所有整数解。

在实际应用中，我们可以使用 `gmpy2` 库中的 `gcdext` 函数来求解二元一次不定方程 $ax + by = (a, b)$ ，实际用例如下：

```

1  import gmpy2
2  a=123456789
3  b=987654321
4  x,y,d=gmpy2.gcdext(a,b)
5  print(x,y,d)

```

该函数接受两个参数 a, b ，并返回三个值 d, x, y ，其中 d 为 a 与 b 的最大公约数， x, y 则是方程 $ax + by = d$ 的其中一组整数解。

在后面也会对拓展欧几里得算法的应用进行更加详细的介绍。

2.1.3 质数、同余与素性测试

质数 (prime number, 又称素数) 是指除了 1 和它本身之外没有其他因数的大于 1 的自然数，比如 2, 3, 5, 7, 11, ... 都是质数，而不是质数的数称为合数。在密码学中，质数是一个很重要的概念。

关于质数，有一个很重要的定理——算术基本定理：

算术基本定理

每一个大于 1 的自然数都可以唯一表示为一系列质数的乘积, 即对于任意一个大于 1 的自然数 n , 都存在唯一的一组质数 $p_1, p_2, \dots, p_k$ 使得:

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

其中 $p_1 < p_2 < \cdots < p_k$ 是质数, $e_1, e_2, \dots, e_k$ 是正整数.

在初步了解了质数与算术基本定理之后, 我们即将需要了解同余, 同余是数论中的一个重要概念, 它描述了整数之间的一种关系, 这也是 RSA 公钥体系以及很多密码学理论的基础.

同余

给定一正整数 m , 若对整数 a, b , 用 a 除以 m 所得的余数等于 b 除以 m 所得的余数, 则称 a 与 b 对模 m 同余, 记作 $a \equiv b \pmod{m}$, 否则称 a 与 b 对模 m 不同余.

通过对定义的延申, 我们可以得到同余的一些性质:

- (同余关系的自反性) $a \equiv a \pmod{m}$
- (同余关系的对称性) 若 $a \equiv b \pmod{m}$, 则 $b \equiv a \pmod{m}$
- (同余关系的传递性) 若 $a \equiv b \pmod{m}$, $b \equiv c \pmod{m}$, 则 $a \equiv c \pmod{m}$
- 若 $a_1 \equiv b_1 \pmod{m}$ 且 $a_2 \equiv b_2 \pmod{m}$, 则 $a_1 + a_2 \equiv b_1 + b_2 \pmod{m}$, 而且 $a_1 a_2 \equiv b_1 b_2 \pmod{m}$
- 若 $a + b \equiv c \pmod{m}$, 则 $a \equiv c - b \pmod{m}$
- 若有 $a = a'p, b = b'p$ 而且 $(p, m) = 1$, 则如果 $a \equiv b \pmod{p}$, 那么有 $a' \equiv b' \pmod{m}$
- 若 $a \equiv b \pmod{m}$, 则 $a^k \equiv b^k \pmod{m}$ (其中 k 是整数)
- 若 $a \equiv b \pmod{m}$, 则对于任意整数 k , 有 $ka \equiv kb \pmod{mk}$
- 若 $a \equiv b \pmod{m}$, 则对于 a, b, m 的任意一个公约数 d , 有 $\frac{a}{d} \equiv \frac{b}{d} \pmod{\frac{m}{d}}$
- 若对于整数 $m_1, m_2, \dots, m_k$ 均有 $a \equiv b \pmod{m_i} (i = 1, 2, \dots, k)$, 则对于它们的最小公倍数 m , 有 $a \equiv b \pmod{m}$
- 若 $a \equiv b \pmod{m}$, 则对于 m 的任一正因数 d , 有 $a \equiv b \pmod{d}$
- 若 $a \equiv b \pmod{m}$, 则必然有 $(a, m) = (b, m)$

在同余的基础上, 我们有一个重要的定理——欧拉定理. 在介绍欧拉定理之前, 我们需要先了解欧拉函数:

欧拉函数

对于任意一个正整数 n , 欧拉函数 $\varphi(n)$ 定义为 $1, 2, \dots, n-1$ 中与 n 互素的数的个数. 设 $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$, 则我们可以得到欧拉函数的计算公式:

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right)$$

同时我们可以得到一个重要的结论: 若 a, b 为两个互素的正整数, 则 $\varphi(ab) = \varphi(a)\varphi(b)$.

此时我们就可以引入一个在数论中以及密码学中十分重要的定理——欧拉定理：

欧拉定理

对于任意一个大于 1 的正整数 n ，若 $(a, n) = 1$ ，则有 $a^{\varphi(n)} \equiv 1 \pmod{n}$ 。

当 n 为质数时，我们可以得到一个推论——费马小定理¹：

费马小定理

对于任意一个质数 p ，若 $(a, p) = 1$ ，则有 $a^{p-1} \equiv 1 \pmod{p}$ 。

欧拉定理是 RSA 公钥体系的基础，在 RSA 公钥体系中，我们往往需要通过欧拉定理来求出私钥，这将会在之后的章节讲到。

接下来是本节中最后一个重要的概念——素性测试，素性测试可以用于判定所给自然数是否为素数，这在密码学中有着重要的应用。

素性测试一般分为两类：确定性素性测试和概率性素性测试，其中确定性素性测试可以绝对确定一个数是否为素数，而概率性素性测试可以较快速地以很高的概率确定一个数是否为素数，但是存在一定的概率错误地将一个合数判断为素数。

在确定性素性测试中，最简单的算法是试除法，即对于一个数 n ，我们可以从 2 到 $n-1$ 遍历所有的整数，若存在一个整数可以整除 n ，则可以断定 n 为合数，否则 n 为素数，显然这个算法的时间复杂度为 $O(n)$ ，而我们知道，如果 n 为合数，则必然存在一个小于 $\sqrt{n}$ 的因数，所以我们可以将试除法的时间复杂度降低到 $O(\sqrt{n})$ ，那么我们可以将该算法改进为：

Algorithm: 试除法 isPrime(n)

```

for  $i = 2$  to  $\sqrt{n}$  do
    if  $i | n$  then
        return  $n$  不是素数
    end
end
return  $n$  是素数

```

然而若 n 是一个很大的整数， $O(\sqrt{n})$ 的时间复杂度仍然是不可接受的，所以我们需要更高效的确定性素性测试算法，例如 AKS 算法，不过这些确定性素性测试算法在效率上仍然不如概率性素性测试中的 Miller-Rabin 算法。

在概率性素性测试中，最著名的算法是 Miller-Rabin 算法，该算法基于费马小定理与二次探测定理，现在先介绍二次探测定理：

二次探测定理

对于质数 p ，若有 $x^2 \equiv 1 \pmod{p}$ ，则该方程小于 p 的正整数解只有两个： $x \equiv 1 \pmod{p}$ 或 $x \equiv p-1 \pmod{p}$ 。

基于费马小定理与二次探测定理，在一轮测试中，使用随机底数 a 对数 n 进行测试，则对于一个用费马小定理判断为可能是质数的数 n ，则必然满足 $a^{n-1} \equiv 1 \pmod{n}$ ，此时我们可以将 $n-1$ 写成 $2^s d$ 的形式（其中 d 为奇数），所以在一轮测试中，我们可以通过判断 $a^{2^i d} (i = 0, 1, 2, \dots)$ 在模 n 下是否等于 1

¹费马小定理的逆定理并不成立，所以费马小定理并不能用来准确地判断一个数是否素数，因为存在合数 $341 = 11 \times 31$ ，使得 $2^{341-1} \equiv 1 \pmod{341}$ ，这类合数一般被称为伪素数 (pseudoprime number)

或 $n-1$ 来判断 n 是否为素数, 若对中间某一个状态 $a^{2^i d} \equiv n-1 \pmod{n}$, 则可以初步判断 n 为素数, 为提高准确率, 我们需要多进行几轮测试, 若 n 经过 k 轮测试都通过, 则可以认为 n 为素数, 否则 n 为合数. 由此我们可以得到 Miller-Rabin 算法的流程如下页算法所示.

Algorithm: Miller-Rabin 素性测试 MillerRabin(n, k)

```

if  $n < 2$  or  $n \% 2 = 0$  then
  | return  $n$  不是素数
end
if  $n = 2$  then
  | return  $n$  是素数
end
将  $n-1$  写成  $2^s d$  的形式
for  $i = 1$  to  $k$  do
  | 随机选取  $a \in [2, n-2]$ 
  |  $x \leftarrow a^d \pmod{n}$ 
  | if  $x = 1$  or  $x = n-1$  then
  | | 进入下一轮测试
  | end
  | for  $j = 1$  to  $s$  do
  | |  $x \leftarrow x^2 \pmod{n}$ 
  | | if  $x = 1$  or  $x = n-1$  then
  | | | 进入下一轮测试
  | | end
  | end
  | return  $n$  不是素数
end
return  $n$  是素数

```

Miller-Rabin 算法的时间复杂度为 $O(k \log n)$, 是一个很优秀的算法.

在实际应用中, 我们并不需要自己实现素性测试算法, 在很多 Python 库中都有素性测试的函数, 例如在 pycryptodome 库中有 isPrime 函数, sympy 库中有 isprime 函数, gmpy2 库中有 is_prime 函数, 这些函数都可以帮助我们快速、准确地判断一个数是否为质数.

2.1.4 同余式与孙子定理

在了解同余的概念之后, 我们可以进一步了解同余式以及解法.

同余式

对多项式 $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0$ (其中 $a_0, a_1, \dots, a_n \in \mathbb{Z}$), 则对于正整数 m

$$f(x) \equiv 0 \pmod{m}$$

称为模 m 的同余式, 而若 a_n 模 m 不为零, 则称该式的次数为 n .

若存在整数 x_0 使得 $f(x_0) \equiv 0 \pmod{m}$, 则称 x_0 为同余式 $f(x) \equiv 0 \pmod{m}$ 的一个解.

在介绍高次 (次数大于 2) 同余式的解法之前, 我们先介绍一次同余式的解法.

对于一条一次同余式 $ax \equiv b \pmod{m}$ (在之后的讨论中, 我们默认 a 不为 m 的倍数), 我们可以通过

同余的定义将其转化为一个二元一次不定方程 $ax + my = b$ ，那么我们很容易可以知道这个方程有解的条件是 $(a, m) | b$ ，所以我们通过二元一次不定方程的知识可以知道这个方程有解的充要条件是 $(a, m) | b$ ，从而我们可以通过扩展欧几里得算法求出这个方程的解。

对于一类特殊的同余式 $ax \equiv 1 \pmod{m}$ ，若这类同余式有解（即 a 与 m 互质），则称 a 在模 m 下有乘法逆元，该同余式的解记作 $x \equiv a^{-1} \pmod{m}$ ，称为 a 模 m 下的乘法逆元（简称逆元）。对于在模 m 下确定存在逆元的数 a ，Python 提供了多种方法来求解 a 在模 m 下的逆元，例如可以通过 Python 自带的 `pow` 函数通过 `pow(a,-1,m)` 来求解 a 在模 m 下的逆元，也可以通过 `gmpy2` 库中的 `invert` 函数或者 `pycryptodome` 库中的 `inverse` 函数来求解 a 在模 m 下的逆元。

在了解了一次同余式的解法后，我们将要进一步了解一元一次同余式组的解法，一元一次同余式组的形式如下：

$$\begin{cases} x \equiv b_1 \pmod{m_1} \\ x \equiv b_2 \pmod{m_2} \\ \vdots \\ x \equiv b_n \pmod{m_n} \end{cases} \quad (2.1.4.0.1)$$

《孙子算经》最早给出了这类问题的解法，故该类问题的解法被称为孙子定理，孙子定理在国外亦被称为中国剩余定理（Chinese remainder theorem，简称 CRT），该定理详细地介绍了同余式组 (2.1.4.0.1) 如何求解：

孙子定理（中国剩余定理，CRT）

设 $m_1, m_2, \dots, m_k$ 是 k 个两两互质的正整数，令 $m = m_1 m_2 \cdots m_k$ ，记 $M_i = \frac{m}{m_i} (i = 1, 2, \dots, k)$ ，则同余式组 (2.1.4.0.1) 的解为：

$$x \equiv \sum_{i=1}^k M'_i M_i b_i \equiv M'_1 M_1 b_1 + M'_2 M_2 b_2 + \cdots + M'_k M_k b_k \pmod{m}$$

其中 M'_i 是 M_i 在模 $m_i (i = 1, 2, \dots, k)$ 下的逆元，即 $M'_i M_i \equiv 1 \pmod{m_i}$ 。

据此我们可以得到通过孙子定理求解同余式组的流程：

Algorithm: CRT($b_1, b_2, \dots, b_k; m_1, m_2, \dots, m_k$)

```

 $m \leftarrow m_1 m_2 \cdots m_k$  for  $i = 1$  to  $k$  do
     $M_i \leftarrow \frac{m}{m_i}$ 
     $M'_i \leftarrow \text{inv}(M_i, m_i)$  //求解  $M_i$  在模  $m_i$  下的逆元
end
 $x \leftarrow 0$ 
for  $i = 1$  to  $k$  do
     $x \leftarrow x + M'_i M_i b_i \pmod{m}$ 
end
return  $x$ 

```

在实际应用中，我们可以通过 Python 的 `sympy` 库中的 `crt` 函数或者 Sagemath 中的 `crt` 函数来求解同余式组 (2.1.4.0.1)。

在了解了孙子定理（之后均简称为 CRT）之后，我们就可以通过之前所学的知识来求解高次同余式了。通过同余式组的知识，我们知道如下事实：若 $m_1, m_2, \dots, m_k$ 是 k 个两两互质的正整数， $m = m_1 m_2 \cdots m_k$ ，

那么同余式 $f(x) \equiv 0 \pmod{n}$ 与下述同余式组等价:

$$\begin{cases} f(x) \equiv 0 \pmod{m_1} \\ f(x) \equiv 0 \pmod{m_2} \\ \vdots \\ f(x) \equiv 0 \pmod{m_k} \end{cases}$$

而且如果设同余式 $f(x) \equiv 0 \pmod{m_i} (i = 1, 2, \dots, k)$ 的解数为 n_i , 则同余式 $f(x) \equiv 0 \pmod{n}$ 的解数为 $n_1 n_2 \cdots n_k$.

所以假设当 $n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$ 时, 要求解同余式 $f(x) \equiv 0 \pmod{n}$, 我们只需要先求出 $f(x) \equiv 0 \pmod{p_i^{\alpha_i}} (i = 1, 2, \dots, k)$ 的解, 然后通过 CRT 就可以求出 $f(x) \equiv 0 \pmod{n}$ 的所有解了.

假设 p 是一个很大的质数, 如果想要在 Sagemath 中求解 $f(x) \equiv 0 \pmod{p^\alpha}$ (α 为大于 1 的整数) 的解是比较困难的, 但是求解 $f(x) \equiv 0 \pmod{p}$ 是比较容易的, 所以我们可以通过求解 $f(x) \equiv 0 \pmod{p}$ 的解, 然后通过一些方式来求出 $f(x) \equiv 0 \pmod{p^\alpha}$ 的解, 下面介绍通过 Hensel 引理来通过同余式 $f(x) \equiv 0 \pmod{p}$ 的解来求 $f(x) \equiv 0 \pmod{p^\alpha}$ 的解.

Hensel 引理

设 p 是一个质数, $x \equiv x_{\alpha-1} \pmod{p}$ 为同余式 $f(x) \equiv 0 \pmod{p^{\alpha-1}}$ 的一个解 ($\alpha \geq 2$), 且 $f'(x_{\alpha-1}) \not\equiv 0 \pmod{p}$, 则在模 p 意义下必然存在一个正整数 $t_{\alpha-1}$ 使得 $f(x_{\alpha-1} + t_{\alpha-1}p^{\alpha-1}) \equiv 0 \pmod{p^\alpha}$.

Hensel 引理向我们表明了一个事实: 存在一种方法可以通过同余式 $f(x) \equiv 0 \pmod{p}$ 的解来求解 $f(x) \equiv 0 \pmod{p^\alpha}$ 的解, 通过 Hensel 引理的证明, 我们可以得到在引理中提及的 $t_{\alpha-1}$ 实际上满足:

$$t_{\alpha-1} \cdot f'(x_{\alpha-1}) \equiv -\frac{f(x_{\alpha-1})}{p^{\alpha-1}} \pmod{p} \quad (2.1.4.0.2)$$

由于 $x_\alpha \equiv x_{\alpha-1} + t_{\alpha-1}p^{\alpha-1} \equiv \cdots \equiv x_1 \pmod{p}$, 所以必然有 $f'(x_\alpha) \equiv f'(x_1) \pmod{p}$, 所以式 (2.1.4.2) 可以得到一解:

$$t_{\alpha-1} \equiv -\frac{f(x_{\alpha-1})}{p^{\alpha-1}} f'(x_{\alpha-1})^{-1} \pmod{p}$$

令该解为 $t_{\alpha-1} \equiv t'_{\alpha-1} \pmod{p}$, 则有 $t_{\alpha-1} = t'_{\alpha-1} + pt_\alpha (t_\alpha \in \mathbb{Z})$, 所以我们可以得到 $f(x_\alpha) \equiv 0 \pmod{p^\alpha}$ 的一解为:

$$x \equiv x_{\alpha-1} + p^{\alpha-1}(t'_{\alpha-1} + pt_\alpha) \equiv x_{\alpha-1} - f(x_{\alpha-1})f'(x_{\alpha-1})^{-1} \pmod{p^\alpha}$$

通过 Hensel 引理, 假设同余式 $f(x) \equiv 0 \pmod{p}$ 的一个解为 $x \equiv x_1 \pmod{p}$ 我们可以得到同余式 $f(x) \equiv 0 \pmod{p^\alpha} (\alpha = 2, 3, \dots)$ 的解满足以下递推关系:

$$x_\alpha \equiv x_{\alpha-1} - f(x_{\alpha-1})f'(x_{\alpha-1})^{-1} \pmod{p^\alpha}$$

结合上面的知识, 当遇到同余式 $f(x) \equiv 0 \pmod{n} (n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k})$ 时, 我们就可以分别求解 $f(x) \equiv 0 \pmod{p_i} (i = 1, 2, \dots, k)$ 的解, 然后通过上面的递推关系和 CRT 来求出 $f(x) \equiv 0 \pmod{n}$ 的解了.

2.1.5 连分数

连分数

形如：

$$a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \frac{1}{\ddots + \frac{1}{a_n}}}}$$

的分数称为连分数.

为缩减篇幅方便书写, 一般常用符号

$$a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \cdots \frac{1}{a_n}}}$$

或者

$$[a_1, a_2, a_3, \dots, a_n]$$

来表示上面的连分数, 其中第二种表示方法比较常用. 而若 a_1 为整数, $a_2, a_3, \dots, a_n \dots$ 为正整数, 则称连分数 $[a_1, a_2, a_3, \dots, a_n, \dots]$ 为简单连分数. 若简单连分数中 a 的个数有限, 则称其为有限简单连分数, 若 a 的个数无限, 则称其为无限简单连分数.

连分数中有一个比较重要的概念——渐进分数:

渐进分数

对于一个连分数 $[a_1, a_2, a_3, \dots, a_n]$, 我们定义其第 $k(1 \leq k \leq n)$ 个渐进分数为:

$$\frac{p_k}{q_k} = [a_1, a_2, a_3, \dots, a_k]$$

其中 p_k 和 q_k 是整数, 且 $(p_k, q_k) = 1$ (即 $\frac{p_k}{q_k}$ 是一个既约分数).

对于无限简单连分数 $[a_1, a_2, a_3, \dots]$, 其第 k 个渐进分数同样按照如上定义, 而且若 $\lim_{k \rightarrow \infty} \frac{p_k}{q_k} = A$, 则称 A 为无限简单连分数 $[a_1, a_2, a_3, \dots]$ 的值.

实际上, 每一个连分数都表示一个实数, 而且每个实数都可以被表示为简单连分数 (有限或无限), 这在密码学应用中有着重要的意义.

下面介绍连分数的一个应用——求解佩尔 (Pell) 方程:

佩尔 (Pell) 方程

形如 $x^2 - dy^2 = 1$ 的方程称为佩尔方程, 其中 d 是一个正整数.

若 d 是一个完全平方数 (即 $\sqrt{d} \in \mathbb{Z}$), 则佩尔方程 $x^2 - dy^2 = 1$ 无正整数解, 否则该方程有无穷多组正整数解, 在下面的讨论中, 我们只讨论 d 不是完全平方数的情况.

对于佩尔方程 $x^2 - dy^2 = 1$, 若 x_0, y_0 是它的一组正整数解, 且 $x_0 + \sqrt{d}y_0$ 是形如 $x + \sqrt{d}y$ (x, y 均为 $x^2 - dy^2 = 1$ 的正整数解) 的最小数, 则方程的一切正整数解可以由:

$$x + \sqrt{d}y = (x_0 + \sqrt{d}y_0)^n$$

确定. 由此我们可以得到通过佩尔方程的最小正整数解 x_0, y_0 来求解佩尔方程的所有正整数解的迭代方法

(其中 $n \geq 1$):

$$\begin{cases} x_n = x_0 x_{n-1} + dy_0 y_{n-1} \\ y_n = x_0 y_{n-1} + y_0 x_{n-1} \end{cases}$$

表示为矩阵形式则为:

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_0 & dy_0 \\ y_0 & x_0 \end{bmatrix} \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} x_0 & dy_0 \\ y_0 & x_0 \end{bmatrix}^n \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

若我们得到了佩尔方程任一组正整数解, 我们均可以通过上面的递推关系得到所有的正整数解, 而如果想要得到佩尔方程的一组正整数解, 我们需要使用到连分数, 实际上, 若将 $\sqrt{d}$ 展开为连分数, 得到 $[a_1, a_2, \dots, a_n, \dots]$, 我们总能找到找到 a_{n+1} 满足 $a_n = 2a_1$, 则 $\sqrt{d}$ 的第 n 个渐进分数 $[a_1, a_2, \dots, a_n] = \frac{x}{y}$ 所确定的 x, y 就可能为佩尔方程 $x^2 - dy^2 = 1$ 的一组解 (注意, 并不是所有满足条件的渐进分数都确定该方程的一组正整数解, 在实际应用中, 对于每一个满足上述条件的渐进分数, 我们均需要将其分子 x , 及分母 y 代入方程中进行测试, 直到符合方程为止).

2.2 线性代数

2.2.1 矩阵与行列式

2.2.2 线性变换

2.2.3 向量空间

2.3 抽象代数

抽象代数 (又称近世代数), 是代数学的一个分支, 其最根本的任务是研究各种抽象的代数结构以及其性质, 其为密码学提供数学基础和工具, 例如在 AES 加密算法中, 我们会用到抽象代数中有限域的知识, 而在椭圆曲线密码学中, 则会经常用到抽象代数中群论的知识.

2.3.1 集合

2.3.2 代数系统

2.3.3 群

2.3.4 环

2.3.5 域

Chapter 3

对称密码

对称密码指的是加密和解密使用相同密钥的密码算法，其加解密过程大致如下图所示：

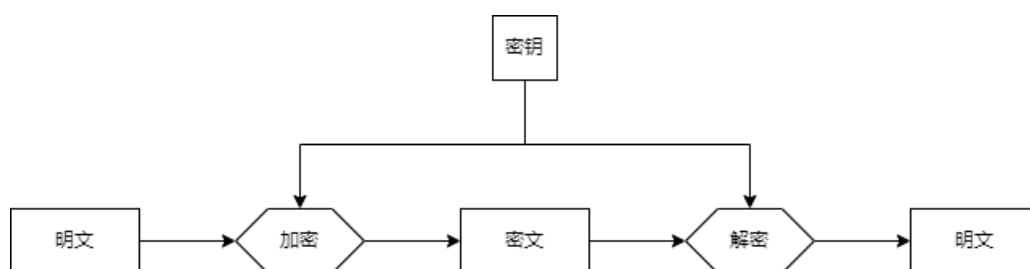


图 3.1: 对称密码加解密过程

在分类上，对称密码可以分为流密码（又称序列密码）和分组密码，

3.1 流密码

3.2 分组密码

Chapter 4

非对称密码

非对称密码指的是加密和解密使用不同密钥的密码算法，其加解密过程大致如下图所示：

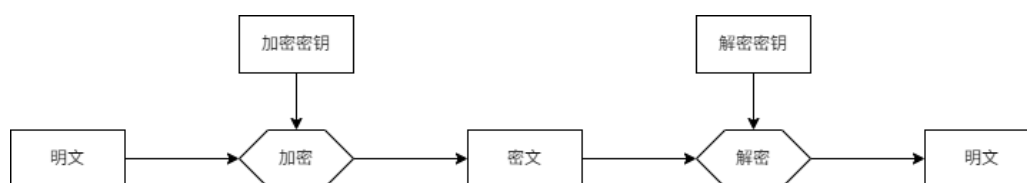


图 4.1: 非对称密码加解密过程

上图中加密密钥与解密密钥不相同，在加密密钥和解密密钥之中必有一个是公开的，另一个是保密的，一般称公开的密钥为公钥，保密的密钥为私钥，故有时候非对称密码也被称为公钥密码。

在 CTF 比赛中常见的非对称密码有 RSA、ECC(Elliptic Curve Cryptography) 等，其中 RSA 是 CTF 中最常见且最基础的非对称密码算法。

4.1 RSA

RSA 算法是由 Ron Rivest、Adi Shamir 以及 Leonard Adleman 三人于 1977 年提出的非对称加密算法，其安全性基于大数分解的困难性，一般情况下，RSA 算法的加解密步骤如下：

加密：

1. 选择两个大质数 p, q ，计算 $n = pq$ ，以及 n 的欧拉函数 $\varphi(n) = (p-1)(q-1)$;
2. 选择一个整数 e (一般选取 $e = 65537$)，使得 $1 < e < \varphi(n)$ 且 $(e, \varphi(n)) = 1$;
3. 计算 d 使得 $ed \equiv 1 \pmod{\varphi(n)}$ ，即 d 是 e 关于模 $\varphi(n)$ 的乘法逆元;
4. 对于明文 m ，计算密文 c ，即 $c \equiv m^e \pmod{n}$;
5. 将公钥对 (n, e) 公开，私钥对 (n, d) 保密。

解密：

1. 对于密文 c ，通过私钥 (n, d) 计算明文 m ，即 $m \equiv c^d \pmod{n}$ 。

RSA 算法的安全性基于大数分解的困难性，在一般情况下，我们已知公钥 (n, e) 的时候，是很难通过分解 n 得到 p, q ，从而计算出私钥 $d \equiv e^{-1} \pmod{\varphi(n)}$ 的。

实际上在 CTF 比赛中，出题人提供的 RSA 加密算法往往是有一定的缺陷的，在本节中，我们将主要介绍一些基础的对 RSA 算法的攻击手段。

4.1.1 直接分解模数 n

尽管先前提到 RSA 算法的安全性基于大数分解的困难性，但是在 CTF 比赛中，RSA 加密过程中选取的素数 p, q 可能较小，或者生成的质数存在某些特殊性质，从而导致直接分解 n 变得可行，常见的分解方法有如下几种：

4.1.1.1 采用工具分解

在 CTF 比赛中，RSA 加密过程中生成的质数可能会比较小，抑或者质因数分解结果被人为地加入了某些数据库中公开，这种情况下，我们可以直接使用一些工具来尝试分解模数 n ，常见的工具有 factordb(<http://www.factordb.com/>)、yafu 等。

factordb 是一个在线的质因数分解数据库，对于一个数 n ，我们可以尝试在 factordb 中搜索 n ，若数据库中不存在该数的分解结果，则可以将其用于求解 RSA，举个例子，对于 $n = 463901$ ，我们可以在 factordb 上搜索这个数：



图 4.2: 使用 factordb 搜索 463901 的质因数

搜索可以得到如下结果：

Search	Sequences	Report results	Factor tables	Status	Downloads	API	Login
463901							
Factorized							
factordb.com - 4 queries to generate this page (0.00 seconds) limits Privacy Policy Imprint Impressum							
Result:							
status (2)	digits	number					
FF	6 show	463901 = 523 * 887					

图 4.3: factordb 搜索结果

此时我们可以得到 463901 的质因数分解结果为 523×887 。但是并非所有的数都能在 factordb 中找到分解结果，若 factordb 中没有该数的分解结果，对于下面这个由两个 512 bits 的质数相乘得到的大整数 n ：

```
8320573729512995489018533734363020219212422510765051110587777028952826122698
8719710929077482516264118525947936654690138203865762947587961213513161052433
7776321996760960256340875966659583104019116897853228345370770890781441937044
8432857596083737050341679106062233488210113949748561973115754159540001230771
9799
```

图 4.4: 两个 512 bits 的质数相乘得到的大整数 n

此时我们尝试使用 factordb 搜索该数，会得到如下结果：

Search	Sequences	Report results	Factor tables	Status	Downloads	API	Login
(832057372951299548901853373436302021921242251076505111058777702895282612269887197109290774825162641) Factorized							
Result:							
status (2)	digits	number					
C*	308 show	8320573729...99_308* = 8320573729...99_308*					

图 4.5: factordb 无法搜索出结果

显然, factordb 无法搜索出该数的分解结果, 一般在求解 RSA 问题的时候遇到这种情况就说明我们可能需要采用其它的方法来求解这道 RSA 问题了.

如果想要更方便地使用 factordb 来分解, 我们可以通过 Python 调用其 API 来实现自动化分解, 有开发者利用 factordb 的 API 开发了一个名为 factordb-python 的库, 安装方式可以参考 factordb-python, 举个例子, 我们以 LitCTF 2023 中 factordb 题目为例, 题目给出如下数据:

```
1 e = 65537
2 n = 87924348264132406875276140514499937145050893665602592992418171647042491658461
3 c = 87677652386897749300638591365341016390128692783949277305987828177045932576708
```

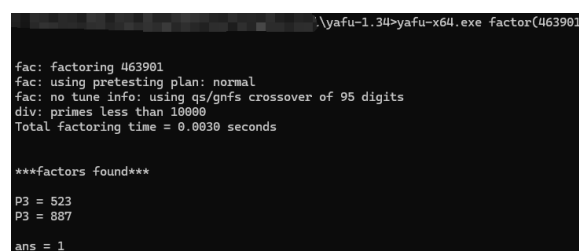
那么我们可以通过 factordb-python 库来分解模数 n , 代码如下:

```
1 from Crypto.Util.number import *
2 from factordb.factordb import FactorDB # 导入factordb-python库
3
4 e = 65537
5 n = 87924348264132406875276140514499937145050893665602592992418171647042491658461
6 c = 87677652386897749300638591365341016390128692783949277305987828177045932576708
7
8 f = FactorDB(n) # 创建factordb对象
9 f.connect() # 连接到factordb
10 factors = f.get_factor_list() # 获取质因数列表
11
12 p = factors[0]
13 q = factors[1]
14
15 phi = (p - 1) * (q - 1)
16 d = inverse(e, phi)
17 m = pow(c, d, n)
18
19 print(long_to_bytes(m))
```

运行代码即可得到 flag.

除 factordb 之外, 我们还可以使用 yafu 等工具来分解模数 n , yafu 是一个开源的质因数分解工具, 支持多种分解算法, 下载地址<https://sourceforge.net/projects/yafu>.

这个工具需要通过命令行来使用, 对于给定的整数 n , 我们可以在 yafu 目录下的命令行中输入 `yafu-x64.exe factor(n)` (其中 n 在实际使用中换成一个具体的数), 例如对于先前的 $n = 463901$, 我们可以在 yafu 目录下的命令行中输入 `yafu-x64.exe factor(463901)`, 得到如下图所示结果:



```

.\yafu-1.34>yafu-x64.exe factor(463901)

fac: factoring 463901
fac: using pretesting plan: normal
fac: no tune info: using qs/gnfs crossover of 95 digits
div: primes less than 10000
Total factoring time = 0.0030 seconds

***factors found***

p1 = 523
p2 = 887

ans = 1

```

图 4.6: yafu 分解结果

如果要分解的数很大，则不能直接使用 `factor(n)` 命令，否则会出现如下图所示的错误：

图 4.7: 长度过长无法直接分解

对于这种情况，我们需要先将这个数写入一个文本文件中（例如将其存入 `n.txt` 中，注意，数的末尾应有换行），然后在命令行中使用 `yafu-x64.exe "factor(@" -batchfile n.txt` 命令来分解，具体流程在此不过多赘述。

4.1.1.2 $p-1$ 是光滑数的情况

在介绍该类题型之前，我们需要了解光滑数的概念：

B-光滑

对于一个正整数 n ，若它的所有质因数均不大于 B ，则称数 n 是 B -光滑的（ B -Smooth），或者称 n 是 B -光滑数（简称光滑数）。

有些文献中也会介绍 B -power-smooth 的概念：

B-power-smooth

对于一个正整数 n ，假设它的质因数分解为：

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

其中 $p_1 < p_2 < \cdots < p_k$ 为质数， $e_1, e_2, \dots, e_k$ 为正整数。若它的所有质因数的幂次 $p_i^{e_i} (i = 1, 2, \dots, k)$ 均不大于 B ，则称数 n 是 B -power-smooth 的。

下面我们默认光滑数的概念为 B -光滑。

在 CTF 中，出题人有时会选取一个满足 $p-1$ 是光滑数的质数 p ，例如 $p = 2^{87} \cdot 3^{23} + 1$ 是满足 $p-1$ 光滑的质数。对于模数 n 的质因数中存在光滑数的情况，我们可以尝试用 Pollard's p-1 算法来对模数 n 进行分解，该算法具体原理如下：根据费马小定理我们可以知道对于一个质数 p ，对于任意与 p 互质的整数 a ，都有 $a^{p-1} \equiv 1 \pmod{p}$ ，那么我们可以知道对于整数 $M = k(p-1) (k \in \mathbb{Z})$ ，有：

$$a^M \equiv a^{k(p-1)} \equiv (a^{p-1})^k \equiv 1^k \equiv 1 \pmod{p}$$

也就是 $a^M - 1 \equiv 0 \pmod{p}$ ，即 $p | (a^M - 1)$ ，若同时有 $p | n$ ，则有：

$$p | (a^M - 1, n)$$

那么对于 RSA 的模数 $n = pq$ ，我们可以通过构造一个可以被 $p-1$ 整除但不会被 $q-1$ 整除的指数 M 来实现对模数 n 的分解，一般选取：

$$M = \prod_{j=2}^B j^{\lfloor \log_j B \rfloor}$$

或者:

$$M = \prod_{j=2}^B j = B!$$

在实际操作中, 我们通常固定取 $a = 2$, 具体分解算法如下:

Algorithm: Pollard's p-1 算法

Input: 模数 n

Output: 模数 n 的一个非平凡因子 p

$a \leftarrow 2$

$M \leftarrow 1$

$j \leftarrow 2$

while *True* **do**

$M \leftarrow M \cdot j$

$g \leftarrow (a^M - 1, n)$

if $1 < g < n$ **then**

return g

end

$j \leftarrow j + 1$

end

通过 Pollard's p-1 算法, 我们可以在 CTF 比赛中对一些构造特殊的模数 n 进行分解, 从而求解 RSA. 以 NCTF 2019 中 childRSA 一题为例, 题目代码如下 (因输出数据太长, 在此不展示):

```
1 from random import choice
2 from Crypto.Util.number import isPrime, sieve_base as primes
3 from flag import flag
4
5 def getPrime(bits):
6     while True:
7         n = 2
8         while n.bit_length() < bits:
9             n *= choice(primes)
10            if isPrime(n + 1):
11                return n + 1
12
13 e = 0x10001
14 m = int.from_bytes(flag.encode(), 'big')
15 p, q = [getPrime(2048) for _ in range(2)]
16 n = p * q
17 c = pow(m, e, n)
```

可以看到这里的质数 p, q 都是通过一个自定义的 `getPrime` 函数来生成的, 该函数会不断地从一个质数表 (即 `sieve_base`, 这是一个包含了 $[2, 104729]$ 中所有质数的数组) 中随机选择质数相乘, 直到得到一个大于等于 2048 bits 的数 n , 然后判断 $n + 1$ 是否为质数, 如果是, 则返回 $n + 1$ 作为 p 或 q , 否则继续循环.

很显然这样生成的质数 p, q 满足 $p - 1$ 和 $q - 1$ 都是光滑数, 所以我们可以使用 Pollard's p-1 算法来分解模数 n , 从而求解 RSA. 具体求解代码如下:

```

1 from Crypto.Util.number import *
2
3 # 数据过长故以省略号代替
4 n = ...
5 c = ...
6 e = 0x10001
7
8 a = 2
9 j = 2
10
11 # Pollard's p-1算法
12 while True:
13     res = GCD(a - 1, n)
14     if res > 1 and res < n:
15         p = res
16         q = n // p
17         break
18     a = pow(a, j, n)
19     j += 1
20
21 assert p * q == n
22
23 phi = (p - 1) * (q - 1)
24 d = inverse(e, phi)
25 m = pow(c, d, n)
26
27 print(long_to_bytes(m))

```

代入题目数据运行即可获得 flag.

4.1.1.3 质数部分位泄露

在一些题目中,可能会出现构成模数 n 的质数 p^1 的部分位被泄露的情况,在这种情况下,我们可以利用这些泄露的信息来恢复完整的质数 p ,进而分解模数 n ,基本的题型有如下几类:

p 高位泄露

在介绍 p 高位泄露之前,我们需要先引入求解这类问题的方法——Coppersmith 方法 [7, 8], 该方法是在 1996 年由 Don Coppersmith 提出的一个基于格基规约²的用于求解首一多项式同余式的小根问题的算法,具体来说,设 N 为一个大整数且其质因数分解未知, $p(x)$ 为一个度为 k 的整系数首一多项式:

$$p(x) = x^k + a_{k-1}x^{k-1} + \cdots + a_1x + a_0$$

若存在一个整数 x_0 满足:

$$p(x_0) \equiv 0 \pmod{N}$$

¹此处的 p 指的是构成模数 n 的任一质数

²格与格基规约相关内容将于第五章介绍

且有 $|x_0| < N^{1/k}$, 则 Coppersmith 方法可以在多项式时间内求出 x_0 , 该方法的具体原理将在第五章进行介绍.

而关于质因数分解, 有如下结论 [9]:

Coppersmith 方法的推论

已知一未知分解的整数 N , 且已知其一因数 $b \geq N^\beta$. 设 $f(x)$ 为一个度为 δ 的一元首一多项式, 若同余式 $f(x) \equiv 0 \pmod{b}$ 存在一个解 x_0 满足:

$$x_0 \leq \frac{1}{2} N^{\frac{\beta^2}{\delta} - \varepsilon}$$

则可以通过 Coppersmith 方法在多项式时间内求解出 x_0 .

在了解了 Coppersmith 方法以及上述结论之后, 我们就可以介绍 p 高位泄露的情况了, 假设 p, q 分别是两个 α 位的大质数且满足 $p < q < 2p$, 若 p 的高 t 位 p_h 被泄露, 那么我们可以将 p 表示为:

$$p = p_h \cdot 2^{\alpha-t} + x$$

其中 x 是一个至多 $\alpha-t$ 位的未知整数, 很显然, 对于多项式 $p(x) = p_h \cdot 2^{\alpha-t} + x$, 存在一个整数 $x_0 < 2^{\alpha-t}$ 使得:

$$p(x_0) \equiv p_h \cdot 2^{\alpha-t} + x_0 \equiv p \equiv 0 \pmod{p}$$

根据上述结论, 若 $x_0 < 2^{\alpha-t} < 0.5N^{\beta^2-\varepsilon}$ (其中 β 一般取 0.4, 可以近似地看作 p 的未知低位少于 p 的位数的一半, 即 $\alpha-t < \alpha/2$), 则可以通过 Coppersmith 方法在多项式时间内分解 N , 在实际操作中, 我们可以使用 Sagemath 中的 `small_roots` 函数¹ 来对这类问题进行求解, 以 HNCCTF 2022 的 partialP 一题为例, 题目代码如下 (因输出数据太长, 在此不展示):

```
1 from Crypto.Util.number import *
2 import uuid
3
4 p = getPrime(512)
5 q = getPrime(512)
6 n = p*q
7 e=65537
8 flag = "flag{" + str(uuid.uuid4())[:20] + "}"
9 m = bytes_to_long(flag.encode())
10 assert(m<n)
11 c=pow(m,e,n)
12
13 print(f"h = {(p>>128)<<128}")
14 print(f"e = 65537")
15 print(f"c = {c}")
16 print(f"n = {n}")
```

分析代码可以知道这里给出了模数 n 以及 n 的一个质因数 p 的高 $512 - 128 = 384$ 位 h , 显然未知位数 $128 < 512/2 = 256$, 根据先前结论, 我们可以通过 Coppersmith 方法在多项式时间内分解 n , 使用 Sagemath 编写求解代码如下:

¹该函数具体用法可以参考官方文档

```

1  # sage 10.4
2  from Crypto.Util.number import *
3
4  h = ...
5  e = 65537
6  c = ...
7  n = ...
8
9  R.<x> = Zmod(n)[]
10
11 f = h + x
12
13 x0 = ZZ(f.small_roots(X=2^128, beta=0.4)[0])
14 p = h + x0
15 q = n // p
16
17 phi = (p - 1) * (q - 1)
18 d = inverse(e, phi)
19
20 m = pow(c, d, n)
21 print(long_to_bytes(m))

```

代入题目数据运行即可获得 flag.

p 低位泄露

低位泄露的处理方式与高位泄露类似：假设 p, q 分别是两个 α 位的大质数且满足 $p < q < 2p$ ，若 p 的低 t 位 p_l 被泄露，那么我们可以将 p 表示为：

$$p = 2^t x + p_l$$

其中 x 是一个至多 $\alpha - t$ 位的未知整数，很显然，对于多项式 $p(x) = 2^t x + p_l$ ，存在一个整数 $x_0 < 2^{\alpha-t}$ 使得：

$$p(x_0) \equiv 2^t x_0 + p_l \equiv p \equiv 0 \pmod{p}$$

但是这个多项式并不是首一多项式，所以我们需要进行处理，使其变为首一多项式¹：

$$p_m(x) = x + p_l \cdot 2^{-t} \tag{4.1.1.3.1}$$

很显然， $p_m(x_0) \equiv p(x_0) \cdot 2^{-t} \equiv 0 \pmod{p}$ ，根据 Coppersmith 方法的推论，若 $x_0 < 2^{\alpha-t} < 0.5N^{\beta^2-\epsilon}$ （其中 β 一般取 0.4，可以近似地看作 p 的未知高位少于 p 的位数的一半，即 $\alpha - t < \alpha/2$ ），则可以通过 Coppersmith 方法在多项式时间内分解 N ，在实际操作中，我们同样可以使用 Sagemath 中的 `small_roots` 函数来对这类问题进行求解，具体实现与高位泄露类似，故不在此举例说明。

除这类题目之外，还有 p 中间部分连续位未知、 p 中间部分连续位已知等题型，这些题型的处理方式与高位泄露和低位泄露类似，都是通过构造一个多项式来表示 p ，然后使用 Coppersmith（一元或多元）方法来求解²。

¹在 Sagemath 中，可以使用 `monic` 函数来把一个多项式变为首一多项式，具体用法可以参考官方文档

²涉及多元 Coppersmith 方法的题目可以使用 `cuso` 进行求解

4.1.1.4 d_p 泄露

在介绍 d_p 泄露之前，我们需要先了解为什么可能会出现 d_p 泄露：

在 RSA 的实际应用中，为了提高解密的效率，通常会使用中国剩余定理 (CRT) 来进行优化，而为了加快解密的速度，通常会在私钥文件中包含下面三个值：

$$\begin{aligned}d_p &= d \pmod{p-1} \\d_q &= d \pmod{q-1} \\q_{inv} &= q^{-1} \pmod{p}\end{aligned}$$

对于密文 c ，因为 $c^d \equiv m \pmod{n}$ ，且 $n = pq$ ，所以有：

$$\begin{aligned}m &\equiv c^d \equiv c^{d_p+k(p-1)} \equiv c^{d_p} \pmod{p} \\m &\equiv c^d \equiv c^{d_q+k'(q-1)} \equiv c^{d_q} \pmod{q}\end{aligned}$$

设 $m_p = c^{d_p} \pmod{p}$ ， $m_q = c^{d_q} \pmod{q}$ ，则可以得到如下的同余式组：

$$\begin{cases} m \equiv m_p \pmod{p} \\ m \equiv m_q \pmod{q} \end{cases}$$

由 $m \equiv m_q \pmod{q}$ 可以得到 $m = m_q + hq$ ，所以可以得到一条关于 h 的线性同余式：

$$m_p \equiv m \equiv m_q + hq \pmod{p}$$

解得 $h = (m_p - m_q) \cdot q^{-1} \pmod{p}$ ，即 $h = (m_p - m_q) \cdot q_{inv} \pmod{p}$ ，最后就可以计算得到 $m = m_q + hq$ ，从而完成解密。很显然，通过上述计算可以将解密所需的时间降低至直接计算 $c^d \pmod{n}$ 的时间的 $1/4$ 左右。

因此，在一些题目中可能会泄露 d_p （可能是直接给出，也可能是泄露 RSA 私钥文件含 d_p 的一小部分），我们可以通过 d_p 来分解模数 n ，具体方法如下：

因为 $d \equiv e^{-1} \pmod{\varphi(n)}$ ，且 $(p-1) | (\varphi(n))$ 所以有：

$$d_p \equiv d \equiv e^{-1} \pmod{p-1}$$

从而有：

$$ed_p \equiv 1 \pmod{p-1}$$

即 $ed_p = 1 + k(p-1) (k \in \mathbb{Z})$ 那么对于一与 p 互质的整数 a （在解决这类问题的时候 a 一般取 2），由费马小定理有：

$$a^{ed_p} \equiv a^{1+k(p-1)} \equiv a \cdot a^{k(p-1)} \equiv a \pmod{p}$$

即：

$$a^{ed_p} - a \equiv 0 \pmod{p}$$

也就是说 $p | (a^{ed_p} - a)$ ，又因为 $p | n$ ，所以可以知道 $(a^{ed_p} - a, n) = p$ ，这样我们就可以借助泄露出的 d_p 来分解模数 n 。以 SWPUCTF 2023 秋季新生赛的 dpdpdpdpdp 一题为例，题目代码如下（因输出数据太长，在此不展示）：

```
1 from Crypto.Util.number import *
2 flag = b'NSSCTF{*****}'
3 p = getPrime(512)
4 q = getPrime(512)
5 n = p*q
```

```

6 e = getPrime(128)
7 d = inverse(e, (p-1)*(q-1))
8 dp = d % (p-1)
9 m = bytes_to_long(flag)
10 c = pow(m, e, n)
11
12 print(f'n = {n}')
13 print(f'e = {e}')
14 print(f'c = {c}')
15 print(f'dp = {dp}')
```

很显然, 这里给出了 n, e, dp , 所以可以通过计算 $(a^{ed_p} - a, n)$ 来分解模数 n , 具体求解代码如下:

```

1 from Crypto.Util.number import *
2
3 n = ...
4 e = ...
5 c = ...
6 dp = ...
7
8 a = 2
9 p = GCD(pow(a, e*dp, n) - a, n)
10 q = n // p
11
12 phi = (p - 1) * (q - 1)
13 d = inverse(e, phi)
14 m = pow(c, d, n)
15
16 print(long_to_bytes(m))
```

代入题目数据运行即可获得 flag.

4.1.2 Wiener 攻击 [10]

Wiener 攻击主要是针对 d 较小的 RSA 的攻击方法, 适用于 $q < p < 2q$ 且 $d < \frac{1}{3}n^{\frac{1}{4}}$ 的情况 (往往这种情况得到的 e 非常大, 所以做题时在遇到 e 非常大的时候可以考虑使用 Wiener 攻击), 在介绍 Wiener 攻击之前, 需要先了解一个连分数相关的重要定理:

Legendre 定理

对于有理数 α , 若存在整数 c, d 满足:

$$\left| \alpha - \frac{c}{d} \right| < \frac{1}{2d^2}$$

那么 $\frac{c}{d}$ 是 α 的一个有理近似 (即渐进分数).

在了解 Legendre 定理之后, 就可以继续介绍 Wiener 攻击了, Wiener 攻击的基本原理如下:

因为在 RSA 中有 $ed \equiv 1 \pmod{\varphi(n)}$, 所以存在整数 k 使得 $ed = 1 + k\varphi(n)$, 即有:

$$ed = 1 + k(p-1)(q-1)$$

两边同时除以 n 可以得到:

$$\frac{e}{n} = \frac{k}{d}(1 - \delta), \text{ 其中 } \delta = \frac{p + q - 1 - \frac{1}{k}}{n}$$

即:

$$\left| \frac{e}{n} - \frac{k}{d} \right| = \frac{|k(p + q - 1) - 1|}{dn}$$

因为 $k < d, p + q - 1 < 3\sqrt{n}$ 所以有:

$$\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{3k\sqrt{n}}{nd} < \frac{3d\sqrt{n}}{nd} = \frac{3}{\sqrt{n}}$$

当 $d < \frac{1}{3}n^{\frac{1}{4}}$ 时, 显然有:

$$\left| \frac{e}{n} - \frac{k}{d} \right| < \frac{3}{\sqrt{n}} < \frac{1}{3d^2} < \frac{1}{2d^2}$$

那么由 Legendre 定理可知, $\frac{k}{d}$ 是 $\frac{e}{n}$ 的一个有理近似, 所以我们可以通过计算 $\frac{e}{n}$ 的连分数 (可以通过 Sagemath 中的 `continued_fraction` 将一个数转换为其对应连分数, 用法可以参考官方文档) 来得到 $\frac{k}{d}$, 从而求出 d . 在获得 d 之后, 我们可以进一步求出 p, q , 因为有 $ed \equiv 1 \pmod{\varphi(n)}$, 所以可以得到:

$$\frac{ed - 1}{k} = \varphi(n) = (p - 1)(q - 1) = n - (p + q) + 1 = n - p - \frac{n}{p} + 1$$

等式两边同时乘以 p 可以得到一个关于 p 的一元二次方程:

$$p^2 - \left(\frac{ed - 1}{k} - n - 1 \right) p + n = 0$$

求得两整数解即为 p 和 q . 以 GHCTF 2025 的 RSA_and_DSA 的前半部分为例, 该部分题目代码如下 (因输出数据太长, 在此不展示):

```

1 from Crypto.Util.Padding import pad
2 from Crypto.Cipher import AES
3 import hashlib
4 import random
5 import gmpy2
6 ink=getPrime(20)
7 p1= getPrime(512)
8 q1= getPrime(512)
9 N = p1* q1
10 phi = (p1-1) * (q1-1)
11 while True:
12     d1= getRandomNBitInteger(200)
13     if GCD(d1, phi) == 1:
14         e = inverse(d1, phi)
15         break
16 c_ink = pow(ink, e, N)
17 print(f'c_ink=',c_ink)
18 print(f'e=',e)
19 print(f'N=',N)

```

分析代码可知这里的 d_1 是一个 200 bits 的随机整数, 而 n 是一个 1024 bits 的整数, 显然满足: $d_1 < \frac{1}{3}n^{\frac{1}{4}}$, 所以可以采用 Wiener 攻击来求解 d_1 , 从而分解模数 n , 具体求解代码如下:

```

1  # sage 10.4
2  from Crypto.Util.number import *
3
4  c_ink = ...
5  e = ...
6  N = ...
7
8  cf = continued_fraction(e / N)
9  convers = cf.convergents()
10
11 for x in convers:
12     k, d = x.as_integer_ratio()
13     if k == 0:
14         continue
15     if (e * d - 1) % k != 0:
16         continue
17     phi = (e * d - 1) // k
18
19     A = 1
20     B = (e*d - 1) // k - N - 1
21     C = N
22
23     p = (-B + int(sqrt(B^2 - 4*A*C))) // (2*A)
24     q = (-B - int(sqrt(B^2 - 4*A*C))) // (2*A)
25
26     if p * q == N:
27         break
28
29 ink = pow(c_ink, d, N)
30 assert ink.bit_length() == 20 and is_prime(ink)
31 print(ink)

```

代入题目数据运行即可求出 `ink`.

4.1.3 共模攻击

共模攻击，顾名思义就是对两次或多次加密的模数 n 相同的时候的攻击，共模攻击并不需要分解 n ，此类题目的特征是一个明文 m ，给出 $k(k \geq 2)$ 个公钥对 $(e_1, n), (e_2, n), \dots, (e_k, n)$ 对 m 进行加密（注意，这里的 n 均为同一个数）得到：

$$c_1 \equiv m^{e_1} \pmod{n}$$

$$c_2 \equiv m^{e_2} \pmod{n}$$

$$\vdots$$

$$c_k \equiv m^{e_k} \pmod{n}$$

在这里先对 $k = 2$ 的情况进行分析, 假设 e_1, e_2 互质, 那么根据裴蜀定理可知存在整数 s, t 使得:

$$s \cdot e_1 + t \cdot e_2 = 1$$

此时我们可以通过扩展欧几里得算法来求出 s, t , 此时可以计算:

$$c_1^s \cdot c_2^t \equiv (m^{e_1})^s \cdot (m^{e_2})^t \equiv m^{se_1+te_2} \equiv m^1 \equiv m \pmod{n}$$

这样就可以在不分解 n 的情况下求出明文 m 了, 以 SWPUCTF 2021 新生赛的crypto2一题为例, 题目代码如下 (因输出数据太长, 在此不展示):

```

1 from gmpy2 import *
2 from Crypto.Util.number import *
3
4 flag = '*****'
5
6 p = getPrime(512)
7 q = getPrime(512)
8 m1 = bytes_to_long(bytes(flag.encode()))
9
10
11 n = p*q
12 e1 = getPrime(32)
13 e2 = getPrime(32)
14 print()
15
16 flag1 = pow(m1,e1,n)
17 flag2 = pow(m1,e2,n)
18 print('flag1= '+str(flag1))
19 print('flag2= '+str(flag2))
20 print('e1= ' +str(e1))
21 print('e2= '+str(e2))
22 print('n= '+str(n))

```

分析代码可知这里分别用两个质数 e_1, e_2 对明文 m 进行加密, 得到 $flag_1$ 和 $flag_2$, 显然 e_1, e_2 是互质的, 故可以使用共模攻击来求解明文 m , 具体求解代码如下:

```

1 from Crypto.Util.number import *
2 from gmpy2 import gcdext
3
4 flag1 = ...
5 flag2 = ...
6 e1 = ...
7 e2 = ...
8 n = ...
9
10 g, s, t = gcdext(e1, e2)
11

```

```

12 m = pow(flag1, s, n) * pow(flag2, t, n) % n
13
14 print(long_to_bytes(m))

```

代入题目数据运行即可获得 flag.

对于 $k > 2$ 的情况, 我们可以将裴蜀定理推广到多个整数的情形:

裴蜀定理的推广

对于 $k(k \geq 2)$ 个非零整数 $a_1, a_2, \dots, a_k$, 对任意整数 $x_1, x_2, \dots, x_k$ 均有:

$$(a_1, a_2, \dots, a_k) | (a_1 x_1 + a_2 x_2 + \dots + a_k x_k)$$

且存在整数 $s_1, s_2, \dots, s_k$ 使得:

$$s_1 a_1 + s_2 a_2 + \dots + s_k a_k = (a_1, a_2, \dots, a_k)$$

通过推广的裴蜀定理我们可以知道, 若 $e_1, e_2, \dots, e_k$ 两两互质 (即 $(e_1, e_2, \dots, e_k) = 1$), 存在整数 $s_1, s_2, \dots, s_k$ 使得:

$$s_1 e_1 + s_2 e_2 + \dots + s_k e_k = (e_1, e_2, \dots, e_k) = 1$$

那么我们可以通过多次利用扩展欧几里得算法来求出 $s_1, s_2, \dots, s_k$, 具体计算过程如下:

Algorithm: 求解 $s_1, s_2, \dots, s_k$

Input: $k(k \geq 2)$ 个非零整数 $e_1, e_2, \dots, e_k$

Output: 整数 $s_1, s_2, \dots, s_k$ 使得 $s_1 e_1 + s_2 e_2 + \dots + s_k e_k = 1$

$g, s_1, s_2 \leftarrow \text{extgcd}(e_1, e_2)$

for $i \leftarrow 3$ **to** k **do**

$g, s, t \leftarrow \text{extgcd}(g, e_i)$

for $j \leftarrow 1$ **to** $i - 1$ **do**

$s_j \leftarrow s_j \cdot s$

end

$s_i \leftarrow t$

end

return $s_1, s_2, \dots, s_k$

使用 Python 实现如下:

```

1 from gmpy2 import gcdext
2
3 def poly_gcdext(*e):
4     g, s, t = gcdext(e[0], e[1])
5     res = [s, t]
6     for i in range(3, len(e) + 1):
7         g, s, t = gcdext(g, e[i - 1])
8         for j in range(len(res)):
9             res[j] = (res[j] * s)
10        res.append(t)
11    return g, res

```

借助这个算法，我们就可以求出 $s_1, s_2, \dots, s_k$ ，从而计算：

$$c_1^{s_1} \cdot c_2^{s_2} \cdots c_k^{s_k} \equiv m^{s_1 e_1 + s_2 e_2 + \cdots + s_k e_k} \equiv m^1 \equiv m \pmod{n}$$

这样就可以在不分解 n 的情况下求出明文 m 了。

4.1.4 相同明文的广播攻击

设 $(e, n_1), (e, n_2), \dots, (e, n_k)$ 是 $k (k \geq e)$ 对合法的 RSA 公钥，其中 $n_1, n_2, \dots, n_k$ 两两互质，若对于同一明文 m ，有：

$$\begin{cases} c_1 \equiv m^e \pmod{n_1} \\ c_2 \equiv m^e \pmod{n_2} \\ \vdots \\ c_k \equiv m^e \pmod{n_k} \end{cases}$$

在已知 $c_1, c_2, \dots, c_k$ 以及 $n_1, n_2, \dots, n_k$ 的情况下，上式显然是一个关于 m^e 的同余式组，所以我们可以使用中国剩余定理来求解得到 $m^e \pmod{n_1 n_2 \cdots n_k}$ ，设 $n_0 = \min\{n_1, n_2, \dots, n_k\}$ ，若有 $m < n_0$ ，则可以知道：

$$m^e < n_0^e \leq n_1 n_2 \cdots n_k$$

那么我们可以对使用 CRT 求解得到的 m^e 进行开 e 次方即可得到明文 m ，这就是广播攻击的基本原理，以鹤城杯 2021 的 Crazy_Rsa_Tech 一题为例，题目代码如下（因输出数据太长，在此不展示）：

```

1 from Crypto.Util.number import *
2 from Crypto.Util.Padding import *
3
4 FLAG = bytes_to_long(pad(b"flag{??????}", 64))
5 def init_key():
6     p, q = getPrime(512), getPrime(512)
7     n = p*q
8     e = 9
9     while(GCD((p-1)*(q-1), e) != 1):
10         p, q = getPrime(512), getPrime(512)
11         n = p*q
12         d = inverse(e, (p-1)*(q-1))
13         return n, e, d
14
15 n_list = list()
16 c_list = list()
17 for i in range(9):
18     N, e, d = init_key()
19     n_list.append(N)
20     c = pow(FLAG, e, N)
21     c_list.append(pow(FLAG, e, N))
22     assert(pow(c, d, N) == FLAG)
23 print("n_list:", n_list)
24 print("c_list:", c_list)

```

可以看到代码使用了 9 组公钥 $(n_1, 9), (n_2, 9), \dots, (n_9, 9)$ 对 flag 进行加密, 而且 $n_1, n_2, \dots, n_9$ 极大概率两两互质且均大于明文, 显然可以使用广播攻击来求解明文 flag, 具体求解代码如下:

```

1 from Crypto.Util.number import *
2 from sympy.ntheory.modular import crt
3 from sympy import integer_nthroot
4
5 n_list = [...]
6 c_list = [...]
7 e = 9
8
9 M = crt(n_list, c_list)[0]
10
11 m = integer_nthroot(M, e)[0]
12 print(long_to_bytes(m))

```

代入题目数据运行即可获得 flag.

4.1.5 相关消息攻击 [11]

对于两明文 m_1, m_2 , 如果有 $m_2 = f(m_1)$ (其中 f 为一已知多项式), 对 m_1, m_2 使用相同的公钥 (n, e) 进行加密得到:

$$\begin{aligned} c_1 &\equiv m_1^e \pmod{n} \\ c_2 &\equiv m_2^e \equiv f(m_1)^e \pmod{n} \end{aligned}$$

很显然, 通过上式可以得到两同余式:

$$\begin{cases} x^e - c_1 \equiv 0 \pmod{n} \\ f(x)^e - c_2 \equiv 0 \pmod{n} \end{cases}$$

显然的, 上述两同余式均有解 $x = m_1$, 所以在 $\mathbb{Z}_n[x]$ 上必然有:

$$(x^e - c_1, f(x)^e - c_2) = p(x)(x - m_1)$$

在绝大多数情况下有 $p(x) = 1$, 即 $(x^e - c_1, f(x)^e - c_2) = x - m_1$, 此时我们只需要提取出公因式 $(x^e - c_1, f(x)^e - c_2)$ 的常系数并在模 n 下计算相反数即可得到明文 m_1 , 从而求出 $m_2 = f(m_1)$. 以 2025 DASCTF 上半年赛的 Excessive Security 一题为例, 题目代码如下 (因输出数据太长, 在此不展示):

```

1 from hashlib import sha256
2 from random import randint
3 from ecdsa import SECP256k1
4 from Crypto.Util.number import inverse, bytes_to_long, getPrime
5 from secret import flag
6
7 curve = SECP256k1
8 G, n = curve.generator, curve.order
9
10 def H(msg): return int.from_bytes(sha256(msg.encode()).digest(), 'big')
11

```

```

12 def sign(h, priv, k):
13     r = (k * G).x() % n
14     s = (inverse(k, n) * (h + r * priv)) % n
15     return s, r
16
17 def split(data):
18     s = data.decode() if isinstance(data, bytes) else data
19     return [s[i::4] for i in range(4)]
20
21 def sign_encrypt(plaintext):
22     blocks = split(plaintext)
23     h = list(map(H, blocks))
24
25     x1 = randint(1, n - 1)
26     x2 = randint(1, n - 1)
27
28     k1 = randint(n // 8, n - 1)
29     k2 = randint(n // 8, n - 1)
30
31     s1, r1 = sign(h[0], x1, k1)
32     s2, _ = sign(h[1], x2, k1)
33     s3, r2 = sign(h[2], x1, k2)
34     s4, _ = sign(h[3], x2, k2)
35
36     m = bytes_to_long(plaintext)
37     p, q = getPrime(512), getPrime(512)
38     N = p * q
39     e = 65537
40     c1 = pow(m, e, N)
41     c2 = pow(x1 * m + x2, e, N)
42
43     print(f"N = {N}")
44     print(f"c1 = {c1}")
45     print(f"c2 = {c2}")
46     print(f"(h1, s1, r1) = ({h[0]}, {s1}, {r1})")
47     print(f"(h2, s2, r1) = ({h[1]}, {s2}, {r1})")
48     print(f"(h3, s3, r2) = ({h[2]}, {s3}, {r2})")
49     print(f"(h4, s4, r2) = ({h[3]}, {s4}, {r2})")
50
51 if __name__ == '__main__':
52     sign_encrypt(flag)

```

这道题虽然涉及到椭圆曲线数字签名算法 (ECDSA)，但是实际上并不需要完全了解 ECDSA 的原理¹，对于签名步骤，我们可以知道对于 `sign(h, priv, k)` 函数，若 k 固定，则返回的 r 是恒定的（即点 kG 的

¹ECDSA 原理参考 4.2

横坐标), 所以对于签名得到的数据 $(h_1, s_1, r_1), (h_2, s_2, r_1), (h_3, s_3, r_2), (h_4, s_4, r_2)$, 我们可以整理出如下的关系:

$$\begin{cases} s_1 \equiv k_1^{-1}(h_1 + r_1 x_1) \pmod{n} \\ s_2 \equiv k_1^{-1}(h_2 + r_1 x_2) \pmod{n} \\ s_3 \equiv k_2^{-1}(h_3 + r_2 x_1) \pmod{n} \\ s_4 \equiv k_2^{-1}(h_4 + r_2 x_2) \pmod{n} \end{cases}$$

可以得到:

$$\begin{cases} k_1 s_1 \equiv h_1 + r_1 x_1 \pmod{n} \\ k_1 s_2 \equiv h_2 + r_1 x_2 \pmod{n} \\ k_2 s_3 \equiv h_3 + r_2 x_1 \pmod{n} \\ k_2 s_4 \equiv h_4 + r_2 x_2 \pmod{n} \end{cases}$$

根据代码我们可以知道 x_1, x_2, k_1, k_2 是未知的, 所以移项可以得到一四元一次同余式组:

$$\begin{cases} s_1 k_1 - r_1 x_1 \equiv h_1 \pmod{n} \\ s_2 k_1 - r_1 x_2 \equiv h_2 \pmod{n} \\ s_3 k_2 - r_2 x_1 \equiv h_3 \pmod{n} \\ s_4 k_2 - r_2 x_2 \equiv h_4 \pmod{n} \end{cases}$$

改写为矩阵形式即为:

$$\begin{pmatrix} s_1 & 0 & -r_1 & 0 \\ s_2 & 0 & 0 & -r_1 \\ 0 & s_3 & -r_2 & 0 \\ 0 & s_4 & 0 & -r_2 \end{pmatrix} \begin{pmatrix} k_1 \\ k_2 \\ x_1 \\ x_2 \end{pmatrix} \equiv \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{pmatrix} \pmod{n}$$

直接采用逆矩阵求解即可得到 x_1, x_2, k_1, k_2 , 此时对于 RSA 加密部分 (即 $c1 = \text{pow}(m, e, N)$ 以及 $c2 = \text{pow}(x1 * m + x2, e, N)$), 有:

$$\begin{cases} c_1 \equiv m^e \pmod{N} \\ c_2 \equiv (x_1 m + x_2)^e \pmod{N} \end{cases}$$

设 $m_1 = m, m_2 = x_1 m + x_2 = x_1 m_1 + x_2$, 显然 m_1, m_2 之间存在线性关系, 所以可以使用相关消息攻击来求解明文 m , 具体求解代码如下:

```

1 # sage 10.4
2 from Crypto.Util.number import *
3 from ecdsa import SECP256k1
4
5 n = SECP256k1.order
6 N = ...
7 c1 = ...
8 c2 = ...
9 (h1, s1, r1) = (... , ... , ...)
10 (h2, s2, r1) = (... , ... , ...)
11 (h3, s3, r2) = (... , ... , ...)
12 (h4, s4, r2) = (... , ... , ...)
13 e = 65537

```

```

14
15 A = matrix(Zmod(n), [[s1, 0, -r1, 0], [s2, 0, 0, -r1], [0, s3, -r2, 0], [0, s4, 0, -r2
    ]])
16 v = vector(Zmod(n), [h1, h2, h3, h4])
17 k1, k2, x1, x2 = A.solve_right(v)
18
19 R.<x> = Zmod(N) []
20 f = x^e - c1
21 g = (ZZ(x1) * x + ZZ(x2))^e - c2
22
23 G = R(f._pari_with_name('x').gcd(g._pari_with_name('x')))
24
25 m = -list(G.monic())[0]
26 print(long_to_bytes(int(m)))

```

代入题目数据运行即可获得 flag.

4.1.5.1 多个明文之间存在隐式多项式关系的相关消息攻击

在先前介绍的相关消息攻击中，我们假设了两个明文之间存在显式的多项式关系，即 $m_2 = f(m_1)$ ，但是在一些题目中，可能会出现多个明文之间存在隐式多项式关系的情况，隐式多项式关系的定义如下：

隐式多项式关系

对数域 $\mathbb{K}$ 中的 n 个变量 $x_1, x_2, \dots, x_n$ ，如果存在一个非零的 n 元多项式：

$$f(x_1, x_2, \dots, x_n) \in \mathbb{K}[x_1, x_2, \dots, x_n]$$

则称方程 $f(x_1, x_2, \dots, x_n) = 0$ 为变量 $x_1, x_2, \dots, x_n$ 的隐式多项式关系。

而对于 t 个明文 $m_1, m_2, \dots, m_t$ ，若它们存在隐式多项式关系 $f(m_1, m_2, \dots, m_t) \equiv 0 \pmod{n}$ （其中 t 元多项式 $f \in \mathbb{Z}_n[x_1, x_2, \dots, x_t]$ 已知），且有密文：

$$c_1 \equiv m_1^e \pmod{n}$$

$$c_2 \equiv m_2^e \pmod{n}$$

$$\vdots$$

$$c_t \equiv m_t^e \pmod{n}$$

则可以得到 $t+1$ 条 t 元同余式：

$$f_0(x_1, x_2, \dots, x_t) \equiv f(x_1, x_2, \dots, x_t) \equiv 0 \pmod{n}$$

$$f_1(x_1, x_2, \dots, x_t) \equiv x_1^e - c_1 \equiv 0 \pmod{n}$$

$$f_2(x_1, x_2, \dots, x_t) \equiv x_2^e - c_2 \equiv 0 \pmod{n}$$

$$\vdots$$

$$f_t(x_1, x_2, \dots, x_t) \equiv x_t^e - c_t \equiv 0 \pmod{n}$$

我们可以通过计算 Groebner 基 $\text{Groebner}([f_0, f_1, f_2, \dots, f_t])$ 来得到 $[x_1 - m_1, x_2 - m_2, \dots, x_t - m_t]$ ，从而获得明文 $m_1, m_2, \dots, m_t$ 。以 NewStar CTF 2024 的学以致用一题为例，题目代码如下（因输出数据太长，在此不展示）：

```

1 import random
2 from Crypto.Util.number import *
3
4 def pad(msg, nbits):
5     # pad了一下, 仔细看看, 别好不容易解出来了却没看到flag
6     pad_length = nbits - len(msg) * 8 - 16
7     assert pad_length >= 0
8     pad = random.getrandbits(pad_length).to_bytes((pad_length + 7) // 8, "big")
9     return pad[:len(pad)//2] + b"*" + msg + b"*" + pad[len(pad)//2:]
10
11 if __name__ == '__main__':
12     p = getPrime(1024)
13     q = getPrime(1024)
14     n = p*q
15     e = 3
16     Nbits = 2048
17     flag = b'flag{?????}'
18     gift = b'GoOd_byE_nEw_5t@r'
19     flag1 = bytes_to_long(pad(flag[:len(flag)//2], Nbits-1))
20     flag2 = bytes_to_long(pad(flag[len(flag)//2:], Nbits-1))
21
22     print('n =', n)
23     print('c1 =', pow(flag1, e, n))
24     print('c2 =', pow(flag2, e, n))
25     print('c3 =', pow(flag1 + flag2 + bytes_to_long(gift), e, n))

```

设 flag1 为 m_1 , flag2 为 m_2 , gift 为 k , 可以知道有:

$$\begin{cases} c_1 &\equiv m_1^3 \pmod{n} \\ c_2 &\equiv m_2^3 \pmod{n} \\ c_3 &\equiv (m_1 + m_2 + k)^3 \pmod{n} \end{cases}$$

那么我们可以得到一个关于 m_1, m_2 的隐式多项式关系:

$$f(m_1, m_2) \equiv (m_1 + m_2 + k)^3 - c_3 \equiv 0 \pmod{n}$$

可以得到 3 个 2 元同余式:

$$\begin{cases} f_0(x_1, x_2) &\equiv f(x_1, x_2) \equiv (x_1 + x_2 + k)^3 - c_3 \equiv 0 \pmod{n} \\ f_1(x_1, x_2) &\equiv x_1^3 - c_1 \equiv 0 \pmod{n} \\ f_2(x_1, x_2) &\equiv x_2^3 - c_2 \equiv 0 \pmod{n} \end{cases}$$

计算 $\text{Groebner}([f_0, f_1, f_2])$ 即可得到 $[x_1 - m_1, x_2 - m_2]$, 从而获得明文 m_1, m_2 , 具体求解代码如下:

```

1 # sage 10.4
2 from Crypto.Util.number import *
3

```

```

4  n = ...
5  c1 = ...
6  c2 = ...
7  c3 = ...
8  e = 3
9  gift = b'Go0d_byE_nEw_5t@r'
10 g = bytes_to_long(gift)
11
12 x,y = Zmod(n)['x,y'].gens()
13
14 p1 = x^e - c1
15 p2 = y^e - c2
16 p0 = (x + y + g)^3 - c3
17
18 I = ideal(p0, p1, p2)
19 GroebnerBasis = I.groebner_basis()
20 f1, f2 = GroebnerBasis
21
22 m1 = n - f1.constant_coefficient()
23 m2 = n - f2.constant_coefficient()
24
25 print(long_to_bytes(int(m1)))
26 print(long_to_bytes(int(m2)))

```

代入题目数据运行即可得到 flag.

4.1.6 e 与 $\varphi(n)$ 不互质

在参数选择不当的情况下, 可能会出现 e 与 $\varphi(n)$ 不互质 (即 $(e, \varphi(n)) \neq 1$) 的情况, 这时根据裴蜀定理我们知道不存在整数 k, d 使得:

$$ed + k\varphi(n) = 1$$

此时即便我们已知 n 的两质因数 p 和 q 也无法通过扩展欧几里得算法来求出 d , 从而无法直接使用 RSA 的解密公式来求出明文 m , 那么我们就需要使用有限域开根结合中国剩余定理来求解明文 m , 具体方法如下:

假定我们已知 n 的两质因数 p 和 q , 则有:

$$c \equiv m^e \pmod{n} \Rightarrow \begin{cases} c \equiv m^e \pmod{p} \\ c \equiv m^e \pmod{q} \end{cases}$$

先考虑质数 p , 假设 $g = (e, p-1) \neq 1$, 那么存在整数 x, y 使得:

$$ex + y\varphi(p) = ex + y(p-1) = g$$

很显然, 我们可以通过拓展欧几里得算法来求解 x 和 y , 此时有

$$c^x \equiv m^{ex} \equiv m^{g-y(p-1)} \equiv m^g \pmod{p}$$

此时我们的目的就是给计算出的 $c^x \bmod p$ 在一个有限域 $\mathbb{Z}_p$ 上开 g 次方, 我们可以使用 Sagemath 中的 `nth_root()` 函数来进行开根, 得到所有的 g 个根 $\{m_1, m_2, \dots, m_g\}$, 同理若 $g' = (e, q-1) \neq 1$, 我们可以仿照上面的方法在有限域 $\mathbb{Z}_q$ 上求出 g' 个根 $\{m'_1, m'_2, \dots, m'_{g'}\}$, 此时我们就得到了 $g \cdot g'$ 个解对 (m_i, m'_j) , 利用中国剩余定理即可得到 $g \cdot g'$ 个明文候选 $m_{i,j}$ 满足:

$$\begin{cases} m_{i,j} \equiv m_i \pmod{p} \\ m_{i,j} \equiv m'_j \pmod{q} \end{cases}$$

以 NCTF 2019 的 easyRSA 一题为例, 题目代码如下 (因输出数据太长, 在此不展示):

```

1 from flag import flag
2
3 e = 0x1337
4 # 因数据过长, p,q具体数值省略
5 p = ...
6 q = ...
7 n = p * q
8
9 assert(flag.startswith('NCTF'))
10 m = int.from_bytes(flag.encode(), 'big')
11 assert(m.bit_length() > 1337)
12
13 c = pow(m, e, n)
14 print(c)

```

本题直接给出了模数 n 的两个质因数 p, q , 但是测试发现 $(e, p-1) = (e, q-1) = e$, 很显然需要使用上述方法, 分别在 $\mathbb{Z}_p$ 和 $\mathbb{Z}_q$ 上对密文 c 开 e 次方根之后再枚举所有根的组合进行 CRT 获得可能的明文, 最后通过判断明文的开头来确定 flag, 具体求解代码如下:

```

1 # sage 10.4
2 from Crypto.Util.number import long_to_bytes
3 from itertools import product
4
5 e = 0x1337
6 p = ...
7 q = ...
8 c = ...
9
10 mps = GF(p)(c).nth_root(e, all=True)
11 mqs = GF(q)(c).nth_root(e, all=True)
12
13 for mp, mq in product(mps, mqs):
14     m = crt([int(mp), int(mq)], [p, q])
15     pt = long_to_bytes(m)
16     if pt.startswith(b'NCTF'):
17         print(pt)
18         break

```

代入题目数据运行即可获得 flag.

若想自己实现在有限域下开根的算法，可以参考 [12] 所提及的 AMM 算法，这是一个比较经典且效率较高的算法.

4.2 椭圆曲线密码学

4.3 数字签名算法

4.4 格密码

格密码 (Lattice-based cryptography) 是一类基于格中困难数学问题的密码体制, 被认为是后量子密码的重要候选方向之一, 格的概念如下:

格

一个 n 维的格 $\mathcal{L}$ 是 $\mathbb{R}^n$ 的一个离散可加的子群. 对于格 $\mathcal{L}$, 其可以被表示为由 $\mathbb{R}^n$ 上 m 个线性无关的基向量 $\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m\}$ 组成的基 $\mathbf{B}$, 向量的个数 m 被称为格 $\mathcal{L}$ 的秩, 所以格 $\mathcal{L}$ 也能表示为:

$$\mathcal{L} = \mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^m a_i \mathbf{b}_i \mid a_i \in \mathbb{Z}, i = 1, 2, \dots, m \right\}$$

格的很多性质与记号与线性空间 (向量空间) 类似.

在实际应用中还有一个比较重要的概念就是格的逐次最小长度:

逐次最小长度

对于一个秩为 n 的格 $\mathcal{L}$, 对于 $i \in \{1, 2, \dots, n\}$, 若 r 是使得格 $\mathcal{L}$ 拥有 i 个长度最大为 r 且线性无关的向量的最小值, 则称 r 为格 $\mathcal{L}$ 的第 i 逐次最小长度, 记为 $\lambda_i(\mathcal{L}) = r$.

关于逐次最小长度, 我们有闵可夫斯基 (Minkowski) 第一定理:

闵可夫斯基 (Minkowski) 第一定理

设 $\mathcal{L}$ 是一个 n 维满秩格, 则:

$$\lambda_1(\mathcal{L}) \leq \sqrt{n} |\text{vol}(\mathcal{L})|^{1/n}$$

闵可夫斯基第一定理表明我们可以以 $\lambda_1(\mathcal{L})$ 为标准来评判格中向量的长度.

除闵可夫斯基第一定理之外, 我们还可以通过高斯启发式 (Gaussian Heuristic) 来估计 $\lambda_1(\mathcal{L})$:

高斯启发式 (Gaussian Heuristic)

设 $\mathcal{L}$ 是一个 n 维满秩格, 则:

$$\lambda_1(\mathcal{L}) \approx \sqrt{\frac{n}{2\pi e}} |\text{vol}(\mathcal{L})|^{1/n}$$

而格密码的安全性主要依赖于格中困难数学问题的难解性, 常见的困难问题有最短向量问题 (Shortest Vector Problem, SVP) 以及最近向量问题 (Closest Vector Problem, CVP) 等, 最短向量问题的叙述如下:

最短向量问题 (Shortest Vector Problem, SVP)

给定格 $\mathcal{L}$ 的基 $\mathbf{B}$, 找到格 $\mathcal{L}$ 上的一个非零向量 $\mathbf{v}$ 满足 $\|\mathbf{v}\| = \lambda_1(\mathcal{L})$.

而最近向量问题的叙述如下:

最近向量问题 (Closest Vector Problem, CVP)

给定格 $\mathcal{L}$ 的基 $\mathbf{B}$ 以及一个目标向量 $\mathbf{t}$ (不一定在格 $\mathcal{L}$ 上), 找到格 $\mathcal{L}$ 上的一个非零向量 $\mathbf{v}$ 满足:

$$\|\mathbf{v} - \mathbf{t}\| = \min_{\mathbf{w} \in \mathcal{L}} \|\mathbf{w} - \mathbf{t}\|$$

上述问题均为 NP-Hard 问题, 而在后续分析中我们主要解决的是它们的宽松版本:

近似最短向量问题 (Approximate Shortest Vector Problem, SVP $_{\gamma}$)

给定格 $\mathcal{L}$ 的基 $\mathbf{B}$ 与近似因子 γ , 找到格 $\mathcal{L}$ 上的一个非零向量 $\mathbf{v}$ 满足 $\|\mathbf{v}\| \leq \gamma \cdot \lambda_1(\mathcal{L})$.

近似最近向量问题 (Approximate Closest Vector Problem, CVP $_{\gamma}$)

给定格 $\mathcal{L}$ 的基 $\mathbf{B}$ 、一个目标向量 $\mathbf{t}$ (不一定在格 $\mathcal{L}$ 上) 以及一个近似因子 γ , 找到格 $\mathcal{L}$ 上的一个非零向量 $\mathbf{v}$ 满足:

$$\|\mathbf{v} - \mathbf{t}\| = \gamma \cdot \min_{\mathbf{w} \in \mathcal{L}} \|\mathbf{w} - \mathbf{t}\|$$

而 SVP $_{\gamma}$ 与 CVP $_{\gamma}$ 对于确定的参数均存在有效的解决方案, 所以我们可以通过解决 SVP $_{\gamma}$ 与 CVP $_{\gamma}$ 来近似地解决 SVP 与 CVP. 我们可以通过格基规约来解决上述两个可计算问题.

4.4.1 格基规约算法

格基规约的目标是将一个任意一个格转化为另外一个基相同但是具有更短、更多的正交向量。由于我们要找的是“短向量”，所以格基规约可能会提供解决近似最短向量问题的方法，最常用的算法就是 LLL 算法，LLL 算法是一个迭代算法，由 Lenstra、Lenstra 与 Lovász 提出。LLL 算法的第一步是通过施密特正交化获得基 $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ 的一组正交基，通过施密特正交化，我们可以将格基进行约减，对于上面的基 $\mathbf{B}$ ，我们知道施密特正交化的过程如下：

$$\begin{cases} \mathbf{b}_i^* = \mathbf{b}_i, & i = 1 \\ \mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{b}_j^*, & 1 < i \leq n \end{cases}, \mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\langle \mathbf{b}_j^*, \mathbf{b}_j^* \rangle}$$

而对于 $\delta \in (\frac{1}{4}, 1)$, 如果基 $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ 满足:

1. (尺寸约减, size-reduction) 对于任意 $i > j$, 有 $|\mu_{i,j}| \leq \frac{1}{2}$
2. (Lovász 条件, Lovász condition) 对于任意 $i \in \{1, 2, \dots, n-1\}$, 有 $(\delta - \mu_{i+1,i}^2) \|\mathbf{b}_i^*\|^2 \leq \|\mathbf{b}_{i+1}^*\|^2$

则称基 $\mathbf{B}$ 为 δ -LLL 约减基，通过结合这两个条件，Lenstra、Lenstra 与 Lovász 给出了 LLL 算法 [13]：通过 LLL 算法，我们可以将一个任意的格基中的向量尽可能地约减为更短的向量，从而解决近似最短向量问题，而对于格中最短向量的下界，我们有如下定理：

格中最短向量下界定理

设格基 $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ 对应的施密特正交基为基 $\mathbf{B}^* = \{\mathbf{b}_1^*, \mathbf{b}_2^*, \dots, \mathbf{b}_n^*\}$, 则有:

$$\lambda_1(\mathcal{L}(\mathbf{B})) \geq \min_{i \in \{1, \dots, n\}} \|\mathbf{b}_i^*\|$$

通过这个定理，我们可以知道，假若 $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ 是一个 δ -LLL 约减基，则必然有：

$$\|\mathbf{b}_1\| \leq \left(\frac{2}{\sqrt{4\delta - 1}} \right)^{n-1} \sqrt{n} \cdot |\text{vol}(\mathcal{L})|^{1/n}$$

Algorithm: LLL($\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}, \delta$)

```

while True do
  for  $i = 2$  to  $n$  do
    for  $j = i - 1$  to  $1$  do
       $\mathbf{b}_i^*, \mu_{i,j} \leftarrow \text{Gram-Schmidt}(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n)$ 
       $\mathbf{b}_i \leftarrow \mathbf{b}_i - \lfloor \mu_{i,j} \rfloor \mathbf{b}_j$ 
    end
  end
  if 存在  $i$  使得  $(\delta - \mu_{i+1,i}^2) \|\mathbf{b}_i^*\|^2 > \|\mathbf{b}_{i+1}^*\|^2$  then
    | 交换  $\mathbf{b}_i$  与  $\mathbf{b}_{i+1}$ 
  end
  else
    | return  $\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ 
  end
end

```

这条式子往往可以帮助我们判断一个格基是否能通过 LLL 来约减出最短向量。

在 Sagemath 中内置了现成的 LLL 算法,例如我们想要对格基 $\mathbf{B} = \{(-2, 2), (-2, 1)\}$ 计算 LLL($\mathbf{B}, 0.75$), 则可以通过如下代码进行计算:

```

1 # sage 10.4
2 b1 = vector(ZZ, [-2, 2])
3 b2 = vector(ZZ, [-2, 1])
4 B = matrix(ZZ, [b1, b2])
5 print(B.LLL(delta=0.75))

```

运行可以得到规约结果为 $\mathbf{b}_1^* = (0, -1), \mathbf{b}_2^* = (-2, 0)$ 。

除 LLL 算法外,还有 BKZ 算法 (Block Korkine-Zolotarev Algorithm) [18], BKZ 算法是 LLL 算法的推广,其核心思想是将格基划分为多个 β 维子格,对每个子格调用 SVP 求解器 (一般直接采用枚举策略) 来找到其中最短的向量并插入到原格基中,再调用 LLL 算法对整个格基进行规约,规约后重复上述步骤直到格基质量无法再进一步提升为止。BKZ 算法的流程如下:

在 Sagemath 中也内置了现成的 BKZ 算法,调用方法与 LLL 算法类似¹。

4.4.2 解决最近向量问题

格基规约算法同样可以用来解决近似最近向量问题,但是需要结合 Babai 最近平面算法或者 Kannan 嵌入法。

Babai 最近平面算法 (Babai's Nearest Plane Algorithm)

Babai 最近平面算法首先要获得输入格的约减基 $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$, 然后对于目标向量 $\mathbf{t}$, 我们令 $\mathbf{t}' = \mathbf{t} - c_n \mathbf{b}_n$, 其中: $c_n = \lceil \langle \mathbf{t}, \mathbf{b}_n^* \rangle / \langle \mathbf{b}_n^*, \mathbf{b}_n^* \rangle \rceil$ (其中 $\lceil \cdot \rceil$ 表示四舍五入), 然后对前 $n-1$ 个向量与 $\mathbf{t}'$ 进行操作, 得到 $\mathbf{t}'' = \mathbf{t}' - c_{n-1} \mathbf{b}_{n-1}$, 其中 $c_{n-1} = \lceil \langle \mathbf{t}', \mathbf{b}_{n-1}^* \rangle / \langle \mathbf{b}_{n-1}^*, \mathbf{b}_{n-1}^* \rangle \rceil$, 以此类推, 就可以得到完整的 Babai 最近平面算法 [14]:

通过该算法我们可以得到一条近似最近向量 $\mathbf{v}$, 从而近似地解决最近向量问题。

¹Sagemath 中 LLL 与 BKZ 函数的参数及其含义可分别参考 Dense matrices over the integer ring-LLL() 和 Dense matrices over the integer ring-BKZ()

Algorithm: BKZ($\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}, \beta, \delta$)

```

LLL( $\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}, \delta$ )
 $z \leftarrow 0, j \leftarrow 0$ 
while  $z < n - 1$  do
     $j \leftarrow (j \bmod (n - 1)) + 1$ 
     $n_j \leftarrow \min(j + \beta - 1, n)$ 
     $h \leftarrow \min(n_j + \beta, n)$ 
    枚举子格  $\mathcal{L}(\{\mathbf{b}_j, \mathbf{b}_{j+1}, \dots, \mathbf{b}_{n_j}\})$  寻找其中最短向量  $\mathbf{v}$ 
    if  $\|\mathbf{b}_j^*\|^2 > \|\pi_j(\mathbf{v})\|^2 / \delta$  then
         $z \leftarrow 0$ 
        LLL( $\{\mathbf{b}_1, \dots, \mathbf{b}_{j-1}, \mathbf{v}, \mathbf{b}_{j+1}, \dots, \mathbf{b}_h\}, \delta$ )
    end
    else
         $z \leftarrow z + 1$ 
        LLL( $\{\mathbf{b}_1, \dots, \mathbf{b}_h\}, 0.99$ )
    end
end
return 更新后格基  $\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ 

```

Algorithm: Babai($\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}, \mathbf{t}$)

```

对格基  $\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$  施行格基规约
 $\mathbf{b}_i^* \leftarrow \text{Gram-Schmidt}(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n)$ 
 $\mathbf{b} \leftarrow \mathbf{t}$ 
for  $i \leftarrow n - 1$  to 1 do
     $c_i \leftarrow \lceil \langle \mathbf{b}, \mathbf{b}_i^* \rangle / \langle \mathbf{b}_i^*, \mathbf{b}_i^* \rangle \rceil$ 
     $\mathbf{b} \leftarrow \mathbf{b} - c_i \mathbf{b}_i$ 
end
return  $\mathbf{t} - \mathbf{b}$ 

```

Kannan 嵌入法 (Kannan's Embedding Method)

Kannan 嵌入法是通过将目标向量嵌入格基，从而将 CVP 转化为 SVP 进行求解，设输入的格基为 $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n\}$ ，而目标向量为 $\mathbf{t} = (t_1, t_2, \dots, t_n)$ ，我们设 CVP 的解为 $c_1 \mathbf{b}_1 + c_2 \mathbf{b}_2 + \dots + c_n \mathbf{b}_n$ ，即：

$$\mathbf{t} \approx \sum_{i=1}^n c_i \mathbf{b}_i \quad (4.4.2.0.1)$$

也就有：

$$\mathbf{t} = \sum_{i=1}^n c_i \mathbf{b}_i + \mathbf{e} \quad (4.4.2.0.2)$$

其中 $\|\mathbf{e}\|$ 很小，所以我们可以构造出一个 $n + 1$ 维的格：

$$\mathbf{B}' = \begin{pmatrix} \mathbf{B} & \mathbf{0} \\ \mathbf{t} & q \end{pmatrix} \quad (4.4.2.0.3)$$

因为有：

$$(-c_1, -c_2, \dots, -c_n, 1) \mathbf{B}' = (\mathbf{e}, q) \quad (4.4.2.0.4)$$

所以这个格显然包含短向量 $(\mathbf{e}, q)$ ，由此我们就可以得到一近似最近向量为 $\mathbf{t} - \mathbf{e}$ 。

在这里, 整数 q 称为嵌入因子, 它会影响到通过格基规约算法寻找正确向量的成功率, 应根据实际情况选取.

4.4.3 背包问题 (Knapsack Problem)

背包问题是 NP 完全问题, 其经常被用于作为公钥密码体系的陷阱门. 在密码学中, 背包问题的常见版本为子集和问题, 即: 给定 n 个正数 $a_1, a_2, \dots, a_n$ 与一个整数 s , 寻找集合 $\{a_1, a_2, \dots, a_n\}$ 的子集使得其和为 s . 也就是找到 $e_1, e_2, \dots, e_n \in \{0, 1\}$ 使得:

$$s = \sum_{i=1}^n e_i a_i$$

4.4.3.1 低密度子集和问题 [17]

在介绍低密度子集和问题之前, 我们需要先了解在该问题中集合密度的定义:

集合的密度

集合 $\{a_1, a_2, \dots, a_n\}$ 的密度为集合中元素的个数与最大元素的对数的比值, 即:

$$d = \frac{n}{\log_2 \max_{i \in \{1, 2, \dots, n\}}(a_i)}$$

研究表明, 当 $d < 0.9408$ 时, 我们可以将子集和问题转换为求解 SVP.

求解该问题的一般策略为构造一个格, 其中有一个短向量包含 $\{e_1, e_2, \dots, e_n\}$, 这样我们就可以构造出下面的格基矩阵:

$$\mathbf{B} = \begin{pmatrix} 1 & & & a_1 \\ & 1 & & a_2 \\ & & \ddots & \vdots \\ & & & 1 & a_n \\ & & & & s \end{pmatrix}$$

显然线性组合 $(e_1, e_2, \dots, e_n, -1)$ 会生成一个短向量 $(e_1, e_2, \dots, e_n, -1)\mathbf{B} = (e_1, e_2, \dots, e_n, 0)$, 所以我们的预期是通过格基规约算法对上述格进行规约得到这个短向量, 但是这个方法在密度比较高的时候会失效, 所以 Coster 等人提出了 CJLOSS 算法, 使得我们可以在 $d < 0.9408$ 的时候通过将子集和问题转换为 SVP 从而在多项式时间内求解这个问题, CJLOSS 算法中构造的格为:

$$\mathbf{B}' = \begin{pmatrix} 1 & & & & Na_1 \\ & 1 & & & Na_2 \\ & & \ddots & & \vdots \\ & & & 1 & Na_n \\ \frac{1}{2} & \frac{1}{2} & \cdots & \frac{1}{2} & Ns \end{pmatrix}$$

其中整数 $N > \sqrt{n}$, 在这个格内, 线性组合 $(e_1, e_2, \dots, e_n, -1)$ 会生成另外一个短向量:

$$(e_1, e_2, \dots, e_n, -1)\mathbf{B}' = (e_1 - \frac{1}{2}, e_2 - \frac{1}{2}, \dots, e_n - \frac{1}{2}, 0)$$

然这条短向量的模为 $\frac{\sqrt{n}}{2}$, 我们很容易可以通过 LLL 算法或者 BKZ 算法求解出这条短向量 (这个短向量一般是规约后的格的第一个向量, 且前 n 个元素往往为 $\frac{1}{2} - e_i$). 使用 Sagemath 通过 CJLOSS 算法求解低密度子集和的代码如下¹:

¹当 $n = 100$ 时使用 BKZ 作为规约算法的 CJLOSS 算法在不同密度和 `block_size` 下平均求解时间和求解成功率可以参考附录 A

```

1  # sage 10.4
2  def CJLOSS(A, s, algorithm="LLL", delta=0.99, block_size=None):
3      if algorithm == "LLL":
4          if block_size is not None:
5              raise ValueError("LLL does not support block size.")
6      elif algorithm == "BKZ":
7          if block_size is None:
8              block_size = 10
9      else:
10         raise ValueError("Unknown algorithm or invalid parameters.")
11
12     n = len(A)
13     N = ceil(sqrt(n))
14     L = matrix(QQ, n + 1)
15     for i in range(n):
16         L[i, i] = 1
17         L[n, i] = 1/2
18         L[i, n] = N * A[i]
19     L[n, n] = N * s
20
21     if algorithm == "LLL":
22         res = L.LLL(delta=delta)
23     elif algorithm == "BKZ":
24         res = L.BKZ(block_size=block_size)
25
26     for v in res:
27         if all(x in [-1/2, 1/2] for x in v[:-1]):
28             out = [int(1/2 - a) for a in v[:-1]]
29             return out

```

4.4.3.2 低密度多重子集和问题 [16]

通过对子集和问题进行扩展，我们可以得到多重子集和问题：给定 kn 个正整数 $a_{1,1}, a_{1,2}, \dots, a_{k,n}$ 以及 k 个整数 $s_1, \dots, s_k$ ，找到 $e_1, e_2, \dots, e_n \in \{0, 1\}$ 满足对任意 $j \in 1, 2, \dots, k$ ：

$$s_j = \sum_{i=1}^n e_i a_{j,i}$$

实际上多重子集和问题总体与普通子集和问题差不多，但是集合更多，此时我们通过下式对密度 d 进行计算：

$$d = \frac{n}{k \log_2 \max(a_{j,i})}$$

根据潘彦斌等人研究可以得出，当 $d < 0.9408$ 时，我们可以通过构造如下格来将低密度多重子集和问题转换为 SVP 进行求解：

$$\mathbf{B} = \begin{pmatrix} 1 & & & 0 & Na_{1,1} & Na_{2,1} & \cdots & Na_{k,1} \\ & 1 & & 0 & Na_{1,2} & Na_{2,2} & \cdots & Na_{k,2} \\ & & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ & & & 1 & 0 & Na_{1,n} & Na_{2,n} & \cdots & Na_{k,n} \\ \frac{1}{2} & \frac{1}{2} & \cdots & \frac{1}{2} & \frac{1}{2} & Ns_1 & Ns_2 & \cdots & Ns_k \end{pmatrix}$$

其中整数 $N > \sqrt{\frac{n+1}{4}}$ ，此时在这个格内，线性组合 $(e_1, e_2, \dots, e_n, -1)$ 会生成一个短向量：

$$(e_1, e_2, \dots, e_n, -1)\mathbf{B} = (e_1 - \frac{1}{2}, e_2 - \frac{1}{2}, \dots, e_n - \frac{1}{2}, -\frac{1}{2}, 0, \dots, 0)$$

通过格基规约算法我们就可以得到这个短向量。

以 HGAME 2025 的 ezBag 一题为例，题目代码如下（因输出数据太长，在此不展示）：

```

1 from Crypto.Util.number import *
2 import random
3 from Crypto.Cipher import AES
4 import hashlib
5 from Crypto.Util.Padding import pad
6 from secrets import flag
7
8 list = []
9 bag = []
10 p=random.getrandbits(64)
11 assert len(bin(p)[2:])==64
12 for i in range(4):
13     t = p
14     a=[getPrime(32) for _ in range(64)]
15     b=0
16     for i in a:
17         temp=t%2
18         b+=temp*i
19         t=t>>1
20     list.append(a)
21     bag.append(b)
22 print(f'list={list}')
23 print(f'bag={bag}')
24
25 key = hashlib.sha256(str(p).encode()).digest()
26 cipher = AES.new(key, AES.MODE_ECB)
27 flag = pad(flag,16)
28 ciphertext = cipher.encrypt(flag)
29 print(f"ciphertext={ciphertext}")

```

很显然本题是一个多重子集和问题，我们可以通过给出的数据计算出该问题的密度满足：

$$d = \frac{64}{4 \cdot \log_2 \max(a_{j,i})} < 0.9408$$

满足通过格基规约求解多重子集和问题的条件，所以设四个大小为 n 的集合分别为 a_1, a_2, a_3, a_4 ，对应的子集和分别为 b_1, b_2, b_3, b_4 ，构造格：

$$\mathbf{B} = \begin{pmatrix} 1 & & & 0 & Ns_{1,1} & Ns_{2,1} & \cdots & Ns_{4,1} \\ & 1 & & 0 & Ns_{1,2} & Ns_{2,2} & \cdots & Ns_{4,2} \\ & & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ & & & 1 & 0 & Ns_{1,n} & Ns_{2,n} & \cdots & Ns_{4,n} \\ \frac{1}{2} & \frac{1}{2} & \cdots & \frac{1}{2} & \frac{1}{2} & Nb_1 & Nb_2 & \cdots & Nb_4 \end{pmatrix}$$

其中 $a_{i,j}$ 表示第 i 个集合中第 j 个元素，整数 $N > \sqrt{\frac{n+1}{4}}$ （可以直接取 $N = \left\lceil \sqrt{\frac{n+1}{4}} \right\rceil$ ），这样我们就可以通过 LLL 算法规约出目标向量：

$$(e_1, e_2, \dots, e_n, -1)\mathbf{B} = (e_1 - \frac{1}{2}, e_2 - \frac{1}{2}, \dots, e_n - \frac{1}{2}, -\frac{1}{2}, 0, 0, 0, 0)$$

根据上述分析，我们可以通过如下代码求解出目标向量，从而恢复整数 p ，从而获得 flag：

```

1 # sage 10.4
2 from Crypto.Util.number import *
3 import random
4 from Crypto.Cipher import AES
5 import hashlib
6
7 list = [...]
8 bag = [...]
9 ciphertext = b'...'
10
11 n = len(list[0])
12 N = ceil((isqrt(n) + 1)/4)
13
14 L = matrix(QQ, n + 1, n + 5)
15 for i in range(n):
16     L[i, i] = 1
17     for j in range(4):
18         L[i, n + 1 + j] = list[j][i]*N
19
20 for i in range(n+1):
21     L[n, i] = 1/2
22
23 for i in range(4):
24     L[n, n + 1 + i] = bag[i]*N
25
26 res = L.LLL()
27
28 for v in res:

```

```

29     v = v[:n]
30     if all([v[i] in [-1/2, 1/2] for i in range(n)]):
31         s = ""
32         for x in v:
33             if x == 1/2:
34                 s += "1"
35             else:
36                 s += "0"
37         p = int(s[::-1], 2)
38         key = hashlib.sha256(str(p).encode()).digest()
39         cipher = AES.new(key, AES.MODE_ECB)
40         plaintext = cipher.decrypt(ciphertext)
41         print(plaintext)

```

代入数据运行即可得到 flag.

4.4.3.3 低密度模子集和问题与低密度多重模子集和问题

通过对子集和问题的扩充，我们可以得到模子集和问题：给定 n 个模 M 下的正整数 $a_1, a_2, \dots, a_n$ ，再给出一个整数 s ，找到 $e_1, e_2, \dots, e_n \in \{0, 1\}$ ，使得：

$$s \equiv \sum_{i=1}^n e_i a_i \pmod{M}$$

从而可以推广为多重模子集和问题：给定 kn 个模 M 下的正整数 $a_{1,1}, a_{1,2}, \dots, a_{k,n}$ 以及 k 个整数 $s_1, \dots, s_k$ ，找到 $e_1, e_2, \dots, e_n \in \{0, 1\}$ 满足对任意 $j \in 1, 2, \dots, k$ ：

$$s_j \equiv \sum_{i=1}^n e_i a_{j,i} \pmod{M}$$

实际上，模子集和问题可以视作 $k = 1$ 的多重模子集和问题，我们可以通过下式对多重模子集和问题的密度 d 进行计算：

$$d = \frac{n}{k \log_2 M}$$

根据潘彦斌等人的研究可以知道，当 $d < 0.9408$ 且 $k = o\left(\frac{n}{\log_2((n+1)\sqrt{n+1})}\right)$ 时，我们可以通过构造如下格来将低密度多重模子集和问题转换为 SVP 进行求解：

$$B = \begin{pmatrix} 1 & & 0 & Na_{1,1} & Na_{2,1} & \cdots & Na_{k,1} \\ & 1 & 0 & Na_{1,2} & Na_{2,2} & \cdots & Na_{k,2} \\ & & \ddots & \vdots & \vdots & \ddots & \vdots \\ & & & 1 & 0 & Na_{1,n} & Na_{2,n} & \cdots & Na_{k,n} \\ & & & & NM & & & & \\ & & & & & NM & & & \\ & & & & & & \ddots & & \\ & & & & & & & NM & \\ \frac{1}{2} & \frac{1}{2} & \cdots & \frac{1}{2} & \frac{1}{2} & Ns_1 & Ns_2 & \cdots & Ns_k \end{pmatrix}$$

使用格基规约算法进行规约就可以得到目标短向量 $(e_1 - \frac{1}{2}, e_2 - \frac{1}{2}, \dots, e_n - \frac{1}{2}, -\frac{1}{2}, 0, \dots, 0)$.

4.4.4 隐藏数问题 (Hidden Number Problem, HNP)

隐藏数问题的形式一般为给定一个质数 p , 令 $\alpha \in [1, p-1]$ 作为“被隐藏的数字”, 给定 m 个数对 $\{(t_i, a_i)\}_{i=1}^m$ 使得:

$$\beta_i - t_i \alpha + a_i \equiv 0 \pmod{p}$$

其中 $|\beta_i| < K < p$, 我们需要从中恢复出 α .

为解决这类问题, 我们可以重写 $\beta_i - t_i \alpha + a_i \equiv 0 \pmod{p}$ 为 $\beta_i + a_i = t_i \alpha + k_i p$, 这样我们就可以构造出下面的格:

$$\mathbf{B} = \begin{pmatrix} p & & & & \\ & p & & & \\ & & \ddots & & \\ & & & p & \\ t_1 & t_2 & \cdots & t_m & \frac{1}{p} \end{pmatrix}$$

对于向量 $\mathbf{v} = (k_1, \dots, k_m, \alpha)$ 有 $\mathbf{vB} = (\beta_1 + a_1, \dots, \beta_m + a_m, \alpha/p)$, 显然这个向量并不是很短, 但是我们注意到:

$$\mathbf{vB} = (a_1, \dots, a_m, 0) + (\beta_1, \dots, \beta_m, \alpha/p)$$

那么令 $\mathbf{t} = (a_1, \dots, a_m, 0)$, $\mathbf{u} = (\beta_1, \dots, \beta_m, \alpha/p)$, 我们就可以得到:

$$\mathbf{vB} - \mathbf{t} = \mathbf{u}$$

其中 $\|\mathbf{u}\| < \sqrt{m+1}K$, 显然我们可以通过将求解 HNP 转换为求解 CVP, 通过 Kannan 嵌入法, 令嵌入因子为 K , 我们就可以得到下面的格基:

$$\mathbf{B}' = \begin{pmatrix} p & & & & \\ & p & & & \\ & & \ddots & & \\ & & & p & \\ t_1 & t_2 & \cdots & t_m & \frac{K}{p} \\ a_1 & a_2 & \cdots & a_m & K \end{pmatrix}$$

其包含向量 $\mathbf{u}' = (\beta_1, \dots, \beta_m, K\alpha/p, -K)$ (也有可能是 $(-\beta_1, \dots, -\beta_m, -K\alpha/p, K)$, 在编写代码求解时需要根据具体情况调整), 且 $\|\mathbf{u}'\| < \sqrt{m+2}K$, $\text{vol}(\mathcal{L}(\mathbf{B}')) = \det(\mathbf{B}') = p^{m-1}K^2$, 显然有:

$$\|\mathbf{u}'\| < \sqrt{m+2}K < \left(\frac{2}{\sqrt{4\delta-1}}\right)^{m+1} \sqrt{m+2} |\det(\mathbf{B}')|^{1/(m+2)}$$

所以我们可以通过 LLL 来找到短向量 $\mathbf{u}'$, 但是这个向量并不是最短的, 因为格上存在另一个向量 $(0, \dots, 0, K, 0)$, 它比 $\mathbf{u}'$ 更短, 所以 $\mathbf{u}'$ 一般在规约后的第二个向量.

以 HGAME 2023 week4 的 LLLCG 一题为例, 题目代码如下:

```
1 from Crypto.Util.number import *
2 from random import randint
3 from sage.all import next_prime
4 from flag import flag
5
6 class LCG():
7     def __init__(self) -> None:
8         self.n = next_prime(2**360)
```

```

9         self.a = bytes_to_long(flag)
10        self.seed = randint(1, self.n-1)
11
12    def next(self):
13        self.seed = self.seed * self.a + randint(-2**340, 2**340) % self.n
14        return self.seed
15
16    lcg = LCG()
17
18    outputs = []
19    for i in range(40):
20        outputs.append(lcg.next())
21
22    with open('output.txt', 'w') as f:
23        f.write(str(outputs))

```

可以看到本题实现了一个 LCG 的变体，迭代公式为：

$$x_{n+1} \equiv ax_n + k_n \pmod{n}$$

其中 n 为已知质数， $|k_n| < 2^{340}$ 是随机的，且 $a < n$ 未知，我们要通过 40 个输出来恢复 a 。对于变体 LCG 的 40 个输出 $x_1, x_2, \dots, x_{40}$ ，我们有：

$$\begin{aligned}
 k_1 - ax_1 + x_2 &\equiv 0 \pmod{n} \\
 k_2 - ax_2 + x_3 &\equiv 0 \pmod{n} \\
 &\dots \\
 k_{39} - ax_{39} + x_{40} &\equiv 0 \pmod{n}
 \end{aligned}$$

显然这是一个 HNP 问题，我们可以构造如下格基：

$$\mathbf{B} = \begin{pmatrix} n & & & & \\ & n & & & \\ & & \ddots & & \\ & & & n & \\ x_1 & x_2 & \cdots & x_{39} & \frac{2^{340}}{n} \\ x_2 & x_3 & \cdots & x_{40} & 2^{340} \end{pmatrix}$$

通过格基规约算法对上述格基进行规约可以得到短向量 $(k_1, k_2, \dots, k_{39}, 2^{340}a/n, -2^{340})$ ，从而恢复出 a ，进而得到 flag。求解代码如下：

```

1 from Crypto.Util.number import *
2 import ast
3
4 x = ast.literal_eval(open("./output.txt").read())
5 n = next_prime(2**360)
6 K = 2**340
7
8 L = matrix(QQ, 41, 41)

```

```

9  for i in range(39):
10     L[i, i] = n
11     L[-2, i] = x[i]
12     L[-1, i] = x[i + 1]
13 L[-2, -2] = K/n
14 L[-1, -1] = K
15
16 res = L.LLL()[1]
17 alpha = int(abs(res[-2] * n / K))
18
19 print(long_to_bytes(alpha))

```

代入题目数据运行即可获得 flag.

前面讲到的隐藏数问题一般称为线性隐藏数问题 (Linear Hidden Number Problem, LHNP), 而隐藏数问题有多种变体, 在这里我们介绍两种变体——扩展隐藏数问题以及椭圆曲线隐藏数问题:

4.4.4.1 扩展隐藏数问题 (Extended Hidden Number Problem, EHNP) [19]

通过对线性隐藏数问题的扩展我们可以得到扩展隐藏数问题: 给定素数 p , 对整数 $x \in [1, p-1]$, 有:

$$x = \bar{x} + \sum_{j=1}^m 2^{\pi_j} x_j$$

其中 $\bar{x}$ 以及 π_j 均已知, 而未知整数 x_j 对于一个已知的整数 ν_j 满足 $0 \leq x_j < 2^{\nu_j}$, 给出 d 条方程:

$$\alpha_i \sum_{j=1}^m 2^{\pi_j} x_j + \sum_{j=1}^{l_i} \rho_{i,j} k_{i,j} \equiv \beta_i - \alpha_i \bar{x} \pmod{p}$$

其中对于 $1 \leq i \leq d$, $\alpha_i \not\equiv 0 \pmod{p}$, $\rho_{i,j}$ 以及 β_i 均为已知的整数, 未知正整数 $k_{i,j}$ 的上界为 $2^{\mu_{i,j}}$ 且 $\mu_{i,j}$ 已知, 则扩展隐藏数问题 (EHNP) 则是通过上述条件恢复 x .

为解决 EHNP, 我们可以构造格:

$$B = \begin{pmatrix} pI_d & & \\ \mathbf{A} & \mathbf{X} & \\ \mathbf{R} & & \mathbf{K} \end{pmatrix}$$

其中:

$$\begin{aligned}
\mathbf{A} &= \begin{pmatrix} \alpha_1 2^{\pi_1} & \alpha_2 2^{\pi_1} & \cdots & \alpha_d 2^{\pi_1} \\ \alpha_1 2^{\pi_2} & \alpha_2 2^{\pi_2} & \cdots & \alpha_d 2^{\pi_2} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1 2^{\pi_m} & \alpha_2 2^{\pi_m} & \cdots & \alpha_d 2^{\pi_m} \end{pmatrix}, \\
\mathbf{X} &= \text{diag} \left(\frac{\delta}{2^{\nu_1}}, \frac{\delta}{2^{\nu_2}}, \dots, \frac{\delta}{2^{\nu_m}} \right), \\
\mathbf{R} &= \text{diag} \left(\boldsymbol{\rho}_1^T, \boldsymbol{\rho}_2^T, \dots, \boldsymbol{\rho}_d^T \right), \\
\boldsymbol{\rho}_i &= (\rho_{i,1}, \dots, \rho_{i,l_i}) (i = 1, \dots, d), \\
\mathbf{K} &= \text{diag} \left(\frac{\delta}{2^{\mu_{1,1}}}, \dots, \frac{\delta}{2^{\mu_{1,l_1}}}, \dots, \frac{\delta}{2^{\mu_{d,1}}}, \dots, \frac{\delta}{2^{\mu_{d,l_d}}} \right)
\end{aligned}$$

设 $L = l_1 + l_2 + \dots + l_d$, $D = d + m + L$, 计算:

$$\kappa_D = \frac{2^{D/4}(m+L)^{1/2} + 1}{2}$$

通过 κ_D , 我们可以得到 δ 满足 $0 < \delta \cdot \kappa_D < 1$ 。令

$$\mathbf{v} = (\beta_1 - \alpha_1 \bar{x}, \dots, \beta_d - \alpha_d \bar{x}, \frac{\delta}{2}, \dots, \frac{\delta}{2})$$

我们需要根据上述的 δ 构造格 $\mathcal{L} = \mathcal{L}(\mathbf{B}, \delta)$, 在格 $\mathcal{L}$ 中找到向量 $\mathbf{u}$ 使得:

$$\|\mathbf{u} - \mathbf{v}\| \leq 2^{\frac{D}{4}} \min_{\mathbf{t} \in \mathcal{L}} \|\mathbf{v} - \mathbf{t}\|$$

通过格基规约算法对我们构造的格进行规约可以得到:

$$\mathbf{u} = \left(\beta_1 - \alpha_1 \bar{x}, \dots, \beta_d - \alpha_d \bar{x}, \frac{x_1 \delta}{2^{\nu_1}}, \dots, \frac{x_m \delta}{2^{\nu_m}}, \frac{k_{1,1} \delta}{2^{\mu_{1,1}}}, \dots, \frac{k_{1,l_1} \delta}{2^{\mu_{1,l_1}}}, \dots, \frac{k_{d,1} \delta}{2^{\mu_{d,1}}}, \dots, \frac{k_{d,l_d} \delta}{2^{\mu_{d,l_d}}} \right)$$

然后令 $x'_j = \frac{1}{\delta}(\mathbf{u}_{d+j} 2^{\nu_j})$ ($1 \leq j \leq m$), 最后计算:

$$x' = \bar{x} + \sum_{j=1}^m 2^{\pi_j} x'_j \pmod{p}$$

这样得到的 x' 就是我们需要的 x 。

双洞隐藏数问题 (Hidden Number Problem with Two Holes, HNP-2H)

HNP-2H 是 EHNP 的一个特例, 设 N 是一个质数, 并设 $x \in \mathbb{Z}_N$ 是一个部分未知的整数满足以下 d 条同余式:

$$\alpha_i x + \rho_{i,1} k_{i,1} + \rho_{i,2} k_{i,2} \equiv \beta_i \pmod{N}, \quad 1 \leq i \leq d$$

其中 α_i 满足 $\alpha_i \pmod{N} \neq 0$, $\alpha_i, \rho_{i,1}, \rho_{i,2}$ 以及 β_i ($1 \leq i \leq d$) 为已知量, 未知量 $k_{i,1}$ 与 $k_{i,2}$ 满足 $0 \leq k_{i,1} \leq 2^{\mu_1}$ 及 $0 \leq k_{i,2} \leq 2^{\mu_2}$ ($1 \leq i \leq d$), 其中 μ_1, μ_2 为已知有理常量, 则 HNP-2H 就是通过以上条件求出 x 。

在参考文献 [19] 中, 作者给出了一种通过狄利克雷近似来解决 HNP-2H 的方法, 在此之前我们需要先了解狄利克雷近似定理:

狄利克雷近似定理

设 $\alpha \in \mathbb{R}$ 以及 $0 < \varepsilon \leq 1$ 为两已知量, 那么存在 $p, q \in \mathbb{Z}$ 使得 $1 \leq q \leq \frac{1}{\varepsilon}$ 以及 $|\alpha - \frac{p}{q}| < \frac{\varepsilon}{q}$ 成立。

根据狄利克雷近似定理, 我们可以得到一推论: 假定我们已知 $A, N \in \mathbb{Z}$ 与 $B \in \mathbb{R}$ 满足 $B \geq 1$ 且 $N > 0$ 那么存在 $\lambda \in \mathbb{Z}$ 满足 $1 \leq \lambda \leq B$ 且 $|\lambda A|_N < \frac{N}{B}$. ($|a|_N$ 表示 $\min_{k \in \mathbb{Z}} |a - kN|$).

通过狄利克雷近似定理以及其推论, 我们很容易可以知道, 在已知 B 的情况下, 有:

$$|\lambda A|_N = |\lambda A - kN| < \frac{N}{B}$$

不等式两边同时除以 λN 可以得到:

$$\left| \frac{A}{N} - \frac{k}{\lambda} \right| < \frac{N}{\lambda B N} = \frac{1}{\lambda B} < \frac{1}{\lambda^2}$$

可以知道 $\frac{k}{\lambda}$ 为 $\frac{A}{N}$ 的一渐进分数, 所以我们可以通过连分数在多项式时间内求出 λ 。

在了解上述理论之后, 我们可以借助狄利克雷近似定理来解决 HNP-2H, 首先令 $A_i = (\rho_{i,1})^{-1} \rho_{i,2} \pmod{N}$, $\gamma_i = k_{i,1} + A_i k_{i,2}$, $\alpha'_i = (\rho_{i,1})^{-1} \alpha_i \pmod{N}$ 以及 $\beta'_i = (\rho_{i,1})^{-1} \beta_i \pmod{N}$ ($1 \leq i \leq d$), 则同余式 $\alpha_i x + \rho_{i,1} k_{i,1} + \rho_{i,2} k_{i,2} \equiv \beta_i \pmod{N}$ 可以转化为:

$$\alpha'_i x + \gamma_i \equiv \beta'_i \pmod{N}$$

我们选定一个大于 1 的实数 B ，由狄利克雷近似的推论，我们可以通过连分数找到一个非零整数 $\lambda_{i,B}$ 满足 $|\lambda_{i,B}A_i| < \frac{N}{B}$ ，且有 $1 \leq \lambda_{i,B} \leq B$ ($1 \leq i \leq d$)，使得下面的关系成立：

$$\begin{aligned} |\lambda_{i,B}\gamma_i|_N &= |\lambda_{i,B}k_{i,1} + \lambda_{i,B}A_i k_{i,2}|_N \\ &\leq |\lambda_{i,B}|_N k_{i,1} + |\lambda_{i,B}A_i|_N k_{i,2} \\ &< B2^{\mu_1} + \frac{N}{B}2^{\mu_2} \end{aligned}$$

选择 $B_{min} = N^{\frac{1}{2}}2^{\frac{\mu_2-\mu_1}{2}}$ 可以最大限度地减少 $B2^{\mu_1} + \frac{N}{B}2^{\mu_2}$ 的上界（此时有 $B_{min}2^{\mu_1} + \frac{N}{B_{min}}2^{\mu_2} = N^{\frac{1}{2}}2^{\frac{\mu_1+\mu_2+2}{2}}$ ）。

在此之后，我们设 $k'_i = (\lambda_{i,B_{min}}\gamma_i + \lfloor N^{\frac{1}{2}}2^{\frac{\mu_1+\mu_2+2}{2}} \rfloor) \bmod N$ ，可以知道： $k'_i < N^{\frac{1}{2}}2^{\frac{\mu_1+\mu_2+4}{2}}$ ，那么通过下述步骤，我们就可以将 HNP-2H 转化为 HNP，从而通过一般 HNP 的解决方法来解决 HNP-2H 了：

$$\begin{aligned} \alpha'_i x + \gamma_i &\equiv \beta'_i \pmod{N} \\ (\lambda_{i,B_{min}}\alpha'_i)x + \lambda_{i,B_{min}}\gamma_i &\equiv \lambda_{i,B_{min}}\beta'_i \pmod{N} \\ (\lambda_{i,B_{min}}\alpha'_i)x + k'_i &\equiv \lambda_{i,B_{min}}\beta'_i + \lfloor N^{\frac{1}{2}}2^{\frac{\mu_1+\mu_2+2}{2}} \rfloor \pmod{N} \\ \alpha''_i x + k'_i &\equiv \beta''_i \pmod{N}, (1 \leq i \leq d) \end{aligned}$$

以 Crypto CTF 2024 的Honey一题为例，题目代码如下（因输出数据太长，在此不展示）：

```

1  #!/usr/bin/env python3
2
3  from Crypto.Util.number import *
4  from math import sqrt
5  from flag import flag
6
7  def gen_params(nbit):
8      p, Q, R, S = getPrime(nbit), [], [], []
9      d = int(sqrt(nbit << 1))
10     for _ in range(d):
11         Q.append(getRandomRange(1, p - 1))
12         R.append(getRandomRange(0, p - 1))
13         S.append(getRandomRange(0, p - 1))
14     return p, Q, R, S
15
16 def encrypt(m, params):
17     p, Q, R, S = params
18     assert m < p
19     d = int(sqrt(p.bit_length() << 1))
20     C = []
21     for _ in range(d):
22         r, s = [getRandomNBitInteger(d) for _ in '01']
23         c = Q[_] * m + r * R[_] + s * S[_]
24         C.append(c % p)
25     return C
26
27

```

```

28 nbit = 512
29 params = gen_params(512)
30 m = bytes_to_long(flag)
31 C = encrypt(m, params)
32 f = open('params_enc.txt', 'w')
33 f.write(f'p = {params[0]}\n')
34 f.write(f'Q = {params[1]}\n')
35 f.write(f'R = {params[2]}\n')
36 f.write(f'S = {params[3]}\n')
37 f.write(f'C = {C}')
38 f.close()

```

整理可得方程组:

$$C_i \equiv Q_i m + R_i r_i + S_i s_i \pmod{p}$$

其中 $i = 1, 2, \dots, d$, 而 C_i, Q_i, R_i, S_i 以及 p 均已知, 而且知道 $0 \leq r_i, s_i \leq 2^d = 2^{32}$, 要求 m , 显然, 这是 HNP-2H, 那么我们可以通过上面讲的方法来将其约化为 HNP 来求解, 有 $A_i = R_i^{-1} S_i \pmod{p}$, $\gamma_i = r_i + A_i s_i$, $\alpha'_i = R_i^{-1} Q_i \pmod{p}$, $\beta'_i = R_i^{-1} C_i \pmod{p}$ ($1 \leq i \leq d$), 可以将方程 $C_i \equiv Q_i m + R_i r_i + S_i s_i \pmod{p}$ 变为:

$$\alpha'_i m + \gamma_i \equiv \beta'_i \pmod{p}$$

由于 $0 \leq r_i, s_i \leq 2^d = 2^{32}$, 那么我们可以取 $\mu_1 = \mu_2 = 32$, 那么我们取 $B_{min} = p^{\frac{1}{2}} 2^{\frac{\mu_2 - \mu_1}{2}} = p^{\frac{1}{2}}$, 则可以利用连分数计算出满足条件的 $\lambda_{i, B_{min}}$, 算法如下:

```

1 def getLambda(A):
2     B = p.isqrt()
3     cf = (A/p).continued_fraction()
4     for i in range(len(cf)):
5         if cf.denominator(i) < B and cf.denominator(i+1) > B:
6             return cf.denominator(i)
7     return None

```

在求出 $\lambda_{i, B_{min}}$ 之后, 我们可以得到:

$$k'_i = \left(\lambda_{i, B_{min}} \gamma_i + \lfloor p^{\frac{1}{2}} 2^{\frac{\mu_1 + \mu_2 + 2}{2}} \rfloor \right) \pmod{p} = \left(\lambda_{i, B_{min}} \gamma_i + \lfloor p^{\frac{1}{2}} 2^{33} \rfloor \right) \pmod{p}$$

可以得到 $\{k_1, k_2, \dots, k_d\}$ 的上界 $K = p^{\frac{1}{2}} 2^{\frac{\mu_1 + \mu_2 + 4}{2}} = p^{\frac{1}{2}} 2^{34}$. 这样我们就可以将这个问题约化为一般的 HNP 进行求解, 代码如下:

```

1 # sage 10.4
2 from Crypto.Util.number import*
3
4 p = ...
5 Q = [...]
6 R = [...]
7 S = [...]
8 C = [...]
9
10 def getLambda(A):

```

```
11     B = p.isqrt()
12     cf = (A/p).continued_fraction()
13     for i in range(len(cf)):
14         if cf.denominator(i) < B and cf.denominator(i+1) > B:
15             return cf.denominator(i)
16     return None
17
18 d = len(Q)
19 A = []
20 B = p.isqrt()
21
22 for i in range(d):
23     a = inverse(R[i], p) * S[i] % p
24     A.append(a)
25
26 alpha = []
27 beta = []
28
29 for i in range(d):
30     lambda_i = getLambda(A[i])
31     a = inverse(R[i], p) * Q[i] * lambda_i % p
32     b = (inverse(R[i], p) * C[i] * lambda_i + floor(B * 2^33)) % p
33     alpha.append(a)
34     beta.append(b)
35
36 K = p.isqrt() * 2^34
37
38 L = matrix(QQ, d+2)
39
40 for i in range(d):
41     L[i, i] = p
42     L[-2, i] = alpha[i]
43     L[-1, i] = beta[i]
44
45 L[-2, -2] = K / p
46 L[-1, -1] = K
47
48 res = L.LLL()
49
50 v = res[1]
51 m = int(abs(v[-2] * p / K))
52 print(long_to_bytes(m))
```

代入题目数据即可得到 flag.

4.4.4.2 椭圆曲线隐藏数问题 (Elliptic Curve Hidden Number Problem, ECHNP)

ECHNP 是一种经典的 HNP 变体, 其定义如下:

ECHNP

给定质数 p 、一正整数 m 、一 $\mathbb{F}_{p^m}$ 上的椭圆曲线 E , 点 $Q \in E$, 以及由 Q 生成的点 $R \in \langle Q \rangle$, 设 f 为一个 E 上的函数, P 为 E 上一未知点, 并设一预言机 $\mathcal{O}_{P,R}(t)$, 其输入为 t , 输出为:

$$\mathcal{O}_{P,R}(t) = f(P + tR)$$

那么椭圆曲线隐藏数问题 (ECHNP) 则是通过预言机 $\mathcal{O}_{P,R}(t)$ 的若干个输出来恢复点 P .

而对于 $m = 1$, 且函数 $f(X) = \text{MSB}_k(X_\psi)$ (即取点 X 的 ψ -坐标的高 k 位, 其中 $\psi \in \{x, y\}$), 则有 ECHNP_ψ :

ECHNP_ψ

给定质数 p 、正整数 k 、一 $\mathbb{F}_p$ 上的椭圆曲线 E , 点 $Q \in E$, 以及由 Q 生成的点 $R \in \langle Q \rangle$, 设 P 为 E 上一未知点, 并设一预言机 $\mathcal{O}_{P,R}(t)$, 其输入为 t , 输出为:

$$\mathcal{O}_{P,R}(t) = \text{MSB}_k((P + tR)_\psi)$$

那么 ECHNP_ψ 则是通过预言机 $\mathcal{O}_{P,R}(t)$ 的若干个输出来恢复点 P .

对于 $k > \frac{5}{6} \log p$ 情况下的 ECHNP_x , 存在一种多项式时间的算法来求解这个问题 (以下默认椭圆曲线为 Weierstrass 方程定义的椭圆曲线):

根据椭圆曲线相异点的加法可以知道, 对于曲线上两相异点 $P(x_P, y_P), Q(x_Q, y_Q)$, 其和 $P + Q$ 的横坐标计算如下:

$$x_{P+Q} = \left(\frac{y_Q - y_P}{x_Q - x_P} \right)^2 - x_P - x_Q$$

所以可以知道:

$$\mathcal{O}_{P,R}(t) = \text{MSB}_k((P + tR)_x) = \text{MSB}_k \left(\left(\frac{y_{tR} - y_P}{x_{tR} - x_P} \right)^2 - x_P - x_{tR} \mod p \right)$$

而 MSB 运算也可以等价于减去一个较小量 e , 那么可以得到:

$$\mathcal{O}_{P,R}(t) = \text{MSB}_k \left(\left(\frac{y_{tR} - y_P}{x_{tR} - x_P} \right)^2 - x_P - x_{tR} \mod p \right) = \left(\frac{y_{tR} - y_P}{x_{tR} - x_P} \right)^2 - x_P - x_{tR} - e \mod p$$

其中 $|e| \leq p/2^{k+1}$, 我们令 $h = \mathcal{O}_{P,R}(t)$, 则可以得到:

$$h \equiv \left(\frac{y_{tR} - y_P}{x_{tR} - x_P} \right)^2 - x_P - x_{tR} - e \pmod{p}$$

可将 x 坐标与 y 坐标分别移到等号两边:

$$(h + x_P + x_{tR} + e)(x_{tR} - x_P)^2 \equiv (y_{tR} - y_P)^2 \equiv y_{tR}^2 - 2y_{tR}y_P + y_P^2 \pmod{p}$$

而因为 $y_P^2 \equiv x_P^3 + ax_P + b \pmod{p}$, 则有:

$$(h + x_P + x_{tR} + e)(x_{tR} - x_P)^2 \equiv (y_{tR} - y_P)^2 \equiv y_{tR}^2 - 2y_{tR}y_P + x_P^3 + ax_P + b \pmod{p}$$

然而上面有我们并不期望出现的 y_P , 故我们需要进行消元, 对于任一正整数 t , 令 $Q = tR$ 我们可以通过预言机得到 $(P + Q)_x$ 以及 $(P - Q)_x$ 的 MSB, 而在 $\mathbb{F}_p$ 上有:

$$\begin{aligned}(P + Q)_x + (P - Q)_x &= \left(\frac{y_P - y_Q}{x_P - x_Q}\right)^2 - x_P - x_Q + \left(\frac{y_P + y_Q}{x_P - x_Q}\right)^2 - x_P - x_Q \\ &= \frac{2(y_P^2 + y_Q^2)}{(x_P - x_Q)^2} - 2x_P - 2x_Q \\ &= 2 \left(\frac{x_Q x_P^2 + (a + x_Q^2)x_P + ax_Q + 2b}{(x_P - x_Q)^2} \right)\end{aligned}$$

我们令 $h = \mathcal{O}_{P,R}(t) = (P + Q)_x - e$, $h' = \mathcal{O}_{P,R}(-t) = (P - Q)_x - e'$, 并令 $\tilde{h} = h + h'$, $\tilde{e} = e + e'$, 由上面的推导可以得到: $\tilde{h} + \tilde{e} = (P + Q)_x + (P - Q)_x$, 又因为当 $t = 0$ 的时候, 有 $\mathcal{O}_{P,R}(0) = x_P - e_0$ (其中 e_0 为一整数), 那么我们记 $h_0 = \mathcal{O}_{P,R}(0)$, 则有 $x_P = h_0 + e_0$, 综合上述推导就可以得到:

$$\begin{aligned}\tilde{h} + \tilde{e} &= (P + Q)_x + (P - Q)_x \\ &= 2 \left(\frac{x_Q x_P^2 + (a + x_Q^2)x_P + ax_Q + 2b}{(x_P - x_Q)^2} \right) \\ &= 2 \left(\frac{x_Q(h_0 + e_0)^2 + (a + x_Q^2)(h_0 + e_0) + ax_Q + 2b}{(h_0 + e_0 - x_Q)^2} \right)\end{aligned}$$

显然的, 上式中只有 $e_0, \tilde{e}$ 是未知的, 我们令两边乘上 $(h_0 + e_0 - x_Q)^2$ 可以得到:

$$(\tilde{h} + \tilde{e})(h_0 + e_0 - x_Q)^2 = 2(x_Q(h_0 + e_0)^2 + (a + x_Q^2)(h_0 + e_0) + ax_Q + 2b)$$

从而可以从中得到一个多项式:

$$F(X, Y) = (\tilde{h} + Y)(h_0 - x_Q + X)^2 - 2(x_Q(h_0 + X)^2 + (a + x_Q^2)(h_0 + X) + ax_Q + 2b)$$

展开整理则有:

$$\begin{aligned}F(X, Y) &= X^2Y + (\tilde{h} - 2x_Q)X^2 + 2(h_0 - x_Q)XY + 2[\tilde{h}(h_0 - x_Q) - 2h_0x_Q - a - x_Q^2]X \\ &\quad + (h_0 - x_Q)^2Y + [\tilde{h}(h_0 - x_Q)^2 - 2h_0^2x_Q - 2(a + x_Q)h_0 - 2ax_Q - 4b]\end{aligned}$$

很显然, 该多项式满足 $F(e_0, \tilde{e}) \equiv 0 \pmod{p}$, 根据上述方法, 我们取 n 组 $(\mathcal{O}_{P,R}(t_i), \mathcal{O}_{P,R}(-t_i)) = (h_i, h'_i)$ 进行上述操作就可以得到 n 个多项式:

$$F_i(X, Y) = X^2Y + A_iX^2 + A_{0,i}XY + B_iX + B_{0,i}Y + C_i$$

满足 $F_i(e_0, \tilde{e}_i) \equiv 0 \pmod{p}$, 其中:

$$\begin{aligned}A_i &= \tilde{h}_i - 2x_{Q_i} \\ A_{0,i} &= 2(h_0 - x_{Q_i}) \\ B_i &= 2[\tilde{h}_i(h_0 - x_{Q_i}) - 2h_0x_{Q_i} - a - x_{Q_i}^2] \\ B_{0,i} &= (h_0 - x_{Q_i})^2 \\ C_i &= \tilde{h}_i(h_0 - x_{Q_i})^2 - 2h_0^2x_{Q_i} - 2(a + x_{Q_i}^2)h_0 - 2ax_{Q_i} - 4b\end{aligned}$$

此时可以构造矩阵:

$$M = \begin{pmatrix} \Delta_1 & 0 & M_1 \\ 0 & \Delta_2 & M_2 \\ 0 & 0 & pI \end{pmatrix}$$

其中 Δ_1, Δ_2 的定义如下:

$$\Delta_1 = \text{diag}(2^{3k}, \underbrace{2^{2k}, \dots, 2^{2k}}_{n+1}), \Delta_2 = 2^k I_{n+1}$$

而 M_1, M_2 定义如下:

$$M_1 = \begin{pmatrix} -C_1 & -C_2 & \cdots & -C_n \\ -B_1 & -B_2 & \cdots & -B_n \\ -B_{0,1} & & & \\ & -B_{0,2} & & \\ & & \ddots & \\ & & & -B_{0,n} \end{pmatrix}, M_2 = \begin{pmatrix} -A_1 & -A_2 & \cdots & -A_n \\ -A_{0,1} & & & \\ & -A_{0,2} & & \\ & & \ddots & \\ & & & -A_{0,n} \end{pmatrix}$$

则存在向量 $v = (1, e_0, \tilde{e}_1, \dots, \tilde{e}_n, e_0^2, e_0 \tilde{e}_1, \dots, e_0 \tilde{e}_n, k_1, \dots, k_n)$ 使得:

$$vM = (2^{3k}, 2^{2k}e_0, 2^{2k}\tilde{e}_1, \dots, 2^{2k}\tilde{e}_n, 2^k e_0^2, 2^k e_0 \tilde{e}_1, \dots, 2^k e_0 \tilde{e}_n, e_0^2 \tilde{e}_1, \dots, e_0^2 \tilde{e}_n)$$

很容易可以证明上述向量可以通过规约得出.

4.4.5 近似公约数问题 (Approximate Common Divisor Problem, ACD Problem) [20]

近似公约数问题 (Approximate Common Divisor Problem, ACD Problem)

对于给定的 $\gamma, \eta, \rho \in \mathbb{N}$, 对于一个 η -bit 的奇数 p , 定义如下高效采样分布 $\mathcal{D}_{\gamma, \rho}(p)$:

$$\mathcal{D}_{\gamma, \rho}(p) = \left\{ pq + r \mid q \in \mathbb{Z} \cap \left[0, \frac{2^\gamma}{p}\right), r \in \mathbb{Z} \cap (2^{-\rho}, 2^\rho) \right\}$$

对于该采样分布, 有 $\rho \ll \eta$, 所以任取 $x \in \mathcal{D}_{\gamma, \rho}(p)$, 都有极大概率满足 $x < 2^\gamma$, 那么对于一个充分大的整数 t , 从 $\mathcal{D}_{\gamma, \rho}(p)$ 中取 t 个样本 $x_1, x_2, \dots, x_t$, 那么所谓近似公约数问题 (Approximate Common Divisor Problem, ACD) 即为从给定的 t 个样本 $\{x_1, x_2, \dots, x_t\}$ 中恢复出 p .

若在 ACD 基础上增加一个特殊样本 $x_0 = pq_0$ (其中 q_0 与 $\mathcal{D}_{\gamma, \rho}(p)$ 中 q 的取值范围一致), 则从给定的从给定的 $t+1$ 个样本 $\{x_0, x_1, x_2, \dots, x_t\}$ 中恢复出 p 就是所谓的部分近似公约数问题 (Partial Approximate Common Divisor Problem, PACD).

求解 ACD 主流的方法有三种: SDA (Simultaneous Diophantine approximation approach), OL (Orthogonal based approach) 以及 MP (Multivariate polynomial approach), 在此仅介绍 SDA¹.

4.4.5.1 同步丢番图逼近方法 (Simultaneous Diophantine approximation approach, SDA)

这是求解 ACD 的方法中最简单的一种, 大致思想就是对于从 $\mathcal{D}_{\gamma, \rho}(p)$ 采样出的 $\{x_0, x_1, \dots, x_t\}$ (此处 x_0 为一般样本), 当 ρ 很小的时候 (也就是 $r_0, r_1, \dots, r_t$ 都很小的时候), 对于 $1 \leq i \leq t$, 总有:

$$\frac{x_i}{x_0} = \frac{pq_i + r_i}{pq_0 + r_0} \approx \frac{q_i}{q_0}$$

很显然, q_i/q_0 是 x_i/x_0 的同步丢番图近似 (simultaneous Diophantine approximation), 事实上有:

$$q_i x_0 - q_0 x_i = q_i(pq_0 + r_0) - q_0(pq_i + r_i) = pq_0 q_i + q_i r_0 - pq_0 q_i - q_0 r_i = q_i r_0 - q_0 r_i$$

所以必然有 $|q_i x_0 - q_0 x_i| \approx 2^{\rho+\gamma-\eta+1}$, 那么可以通过这个关系构造格基:

$$\mathbf{B} = \begin{pmatrix} 2^{\rho+1} & x_1 & x_2 & \cdots & x_t \\ & -x_0 & & & \\ & & -x_0 & & \\ & & & \ddots & \\ & & & & -x_0 \end{pmatrix}$$

¹MP 方法将在多元 CopperSmith 方法部分介绍

对于这个格有如下关系：

$$\begin{aligned}(q_0, q_1, \dots, q_t)\mathbf{B} &= (2^{\rho+1}q_0, q_0x_1 - q_1x_0, \dots, q_0x_t - q_tx_0) \\ &= (2^{\rho+1}q_0, q_0r_1 - q_1r_0, \dots, q_0r_t - q_tr_0)\end{aligned}$$

显然对目标向量 $(2^{\rho+1}p_0, q_0r_1 - q_1r_0, \dots, q_0r_t - q_tr_0)$ 有：

$$|(2^{\rho+1}p_0, q_0r_1 - q_1r_0, \dots, q_0r_t - q_tr_0)| \approx 2^{\rho+\gamma-\eta+1}\sqrt{t+1}$$

又对于格基矩阵，有：

$$|\det(\mathbf{B})| = 2^{\rho+1}x_0^t \approx 2^{\rho+\gamma t+1}$$

那么由高斯启发式可知当：

$$2^{\rho+\gamma-\eta+1}\sqrt{t+1} \leq \sqrt{\frac{t+1}{2\pi e}} \cdot |\det(\mathbf{B})|^{1/(t+1)} \approx 2^{(\rho+\gamma t+1)/(t+1)}\sqrt{t+1}$$

成立，即：

$$\begin{aligned}\rho + \gamma - \eta + 1 &\leq \frac{\rho + \gamma t + 1}{t + 1} \\ \Rightarrow \rho - \eta + 1 &\leq \frac{\rho + \gamma t + 1 - \gamma t - \gamma}{t + 1} = \frac{\rho - \gamma + 1}{t + 1} \\ \Rightarrow t + 1 &\leq \frac{\gamma - \rho - 1}{\eta - \rho - 1} < \frac{\gamma - \rho}{\eta - \rho} \\ \Rightarrow t + 1 &< \frac{\gamma - \rho}{\eta - \rho}\end{aligned}$$

的时候，可以从这个格中规约出目标向量 $(2^{\rho+1}p_0, q_0r_1 - q_1r_0, \dots, q_0r_t - q_tr_0)$ ，此时我们可以从目标向量中获得 q_0 ，因为 r_0 很小，所以有 $x_0 \equiv pq_0 + r_0 \equiv r_0 \pmod{q_0}$ ，那么我们可以得到 $r_0 = x_0 \pmod{q_0}$ ，从而计算出 $p = \frac{x_0 - r_0}{q_0}$ 。

4.4.6 容错学习问题 (Learning with Errors, LWE)

容错学习问题 (Learning with Errors, LWE) 是密码学中一个非常重要的问题，其中最经典的搜索 LWE 问题定义如下：

搜索 LWE 问题 (Search-LWE)

设 n, m 以及 q 为正整数， χ 为一个 $\mathbb{Z}^m$ 上的概率分布，并设 $\mathbf{s} \in (\mathbb{Z}/q\mathbb{Z})^n$ 是一条“秘密向量”，在 $(\mathbb{Z}/q\mathbb{Z})^{m \times n}$ 上均匀随机选取矩阵 $\mathbf{A}$ ，并在 $\mathbb{Z}^m$ 上选取一个分布服从 χ 的小向量 $\mathbf{e}$ 作为噪声，计算：

$$\mathbf{b} \equiv \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$$

并给出 $(\mathbf{A}, \mathbf{b})$ ，那么搜索 LWE 问题即为给定 $(\mathbf{A}, \mathbf{b})$ ，还原出 $\mathbf{s}$ 。

可以看到，因为 $\mathbf{e}$ 是一个小向量，所以搜索 LWE 问题显然是一个 CVP，首先我们可以将 $\mathbf{b} \equiv \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$ 改写为：

$$\mathbf{b} + q\mathbf{k} = \mathbf{A}\mathbf{s} + \mathbf{e}$$

即：

$$q\mathbf{k} - \mathbf{A}\mathbf{s} + \mathbf{b} = \mathbf{e}$$

显然可以构造出如下格基：

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_m & 0 & 0 \\ -\mathbf{A}^T & \mathbf{I}_n & 0 \\ \mathbf{b} & 0 & 1 \end{pmatrix}$$

(实际上是利用 Kannan 嵌入法来将 CVP 转换为 SVP 求解)。对上述格基有如下关系:

$$(\mathbf{k}^T, \mathbf{s}^T, 1)\mathbf{B} = (\mathbf{e}^T, \mathbf{s}^T, 1)$$

通过格基规约算法 (如 LLL, BKZ 等) 即可得到短向量 $(\mathbf{e}^T, \mathbf{s}^T, 1)$, sage 实现如下:

```

1 # sage 10.4
2 q = ...
3
4 L = block_matrix(ZZ, 3, 3, [[q, 0, 0], [-A.transpose(), 1, 0], [matrix(b), 0, 1]])
5 res = L.LLL()
6
7 v = res[0]
```

测试发现, m 越大这种方法就越容易规约出目标向量, 但是这种算法实际效果并不好, 在参考文献 [21] 中提及了另外一种通过 Kannan 嵌入法求解 LWE 的方法, 论文中提及算法如下:

Algorithm: 利用 Kannan 嵌入法求解 LWE

Input: 一组 LWE 实例 $\mathbf{A}, \mathbf{b}$

Output: 秘密向量 $\mathbf{s}$ 与 i 及噪声向量 $\mathbf{e}$

构造基 $\mathbf{B} = \begin{pmatrix} \mathbf{A}^T \\ q\mathbf{I}_m \end{pmatrix} \in \mathbb{Z}^{(m+n) \times m}$

计算 $\mathbf{B}$ 的埃尔米特标准型 (Hermite Normal Form) $\mathbf{B}_{\text{HNF}} \in \mathbb{Z}^{m \times m}$

构造 Kannan 嵌入格:

$$\mathbf{B}' = \begin{pmatrix} \mathbf{B}_{\text{HNF}} & 0 \\ \mathbf{b} & M \end{pmatrix} \in \mathbb{Z}^{(m+1) \times (m+1)}$$

对 $\mathbf{B}'$ 进行格基规约, 得到短向量 $(\mathbf{e}^T, M)$

提取出 $\mathbf{e}$ 后通过对方程 $\mathbf{A}\mathbf{s} \equiv \mathbf{b} - \mathbf{e} \pmod{q}$ 使用高斯消元法求解得到 $\mathbf{s}$

在之前的讨论中我们构造出的格基大小为 $(n + m + 1) \times (n + m + 1)$, 当 n, m 都比较大的时候如果想要规约出目标向量是比较困难的, 即使可以规约出目标向量也需要比较长时间, 如果利用这种算法那么得到的格基大小将是 $(m + 1) \times (m + 1)$, 会相对较容易规约出目标向量, 该算法的 sage 实现如下:

```

1 # sage 10.4
2 m = len(b[0])
3 B = block_matrix(ZZ, 2, 1, [[A.transpose()], [q]])
4 B_HNF = B.hermite_form(include_zero_rows=False)
5
6 L = block_matrix(ZZ, 2, 2, [[B_HNF, 0], [matrix(b), 1]])
7
8 res = L.LLL()[0]
9
10 if res[-1] == -1:
11     e = -vector(res[:-1])
12 else:
13     e = vector(res[:-1])
14
15 cvp = vector(b) - e
16
```

```

17 AA = matrix(Zmod(q), A)
18 cvp = vector(Zmod(q), cvp)
19
20 s = AA.solve_right(cvp)

```

上述代码中得到求解出的 s 即为我们所需要的。经测试发现，相较于之前讨论得到的算法这种算法求解搜索 LWE 的效果更好。

4.4.6.1 环 LWE (Ring-LWE, RLWE)

LWE 的有一个常见变体是环 LWE 问题 (Ring-LWE, RLWE)，其定义如下：

环 LWE 问题 (Ring-LWE, RLWE)

设 n 为一个大于等于 1 的整数， q 为一质数，得到商环：

$$R = \frac{\mathbb{Z}[x]}{(x^n + 1)}, R_q = \frac{(\mathbb{Z}/q\mathbb{Z})[x]}{(x^n + 1)}$$

而 χ 为 R 上的一个概率分布，选取随机秘密多项式 $s(x) \in R_q$ 以及随机多项式 $A(x) \in R_q$ ，并在 R_q 上选取一个分布服从 χ 的多项式 e 作为噪声，计算：

$$b(x) = A(x)s(x) + e(x)$$

搜索 RLWE 问题即为给出若干组 $(A_i(x), b_i(x))$ ，并从中恢复出 $s(x)$ 。

我们可以利用与求解一般搜索 LWE 问题类似的方法来求解搜索 RLWE 问题，我们可以考虑构造如下格基：

$$\mathbf{B} = \begin{pmatrix} q & 0 & 0 \\ -A(x) & 1 & 0 \\ b(x) & 0 & 1 \end{pmatrix}$$

很显然在商环 R 中有如下关系：

$$(k(x), s(x), 1)\mathbf{B} = (e(x), s(x), 1)$$

但是暂时不存在一种有效的方法对商环上的格基进行规约，所以我们需要找到一种方法将商环上的格基矩阵转换为整数格基矩阵，进而利用整数格基规约算法来求解搜索 RLWE 问题。设一商环 $R = \mathbb{Z}[x]/f(x)$ (其中 $f(x)$ 为 $\mathbb{Z}$ 中的一个首一 n 次多项式)，设 $f(x)$ 的系数分别为： $(a_0, a_1, \dots, a_{n-1}, 1)$ ，那么很显然可以知道在该商环中有：

$$x^n = -(a_0 + a_1x + \dots + a_{n-1}x^{n-1})$$

那么对于该商环中任意两多项式：

$$g(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$$

$$h(x) = c_0 + c_1x + c_2x^2 + \dots + c_{n-1}x^{n-1}$$

设对 $\mathbb{Z}[x]$ 中有 $g(x) \cdot h(x) = k_0 + k_1x + k_2x^2 + \dots + k_{2n-2}x^{2n-2}$ ，则有如下关系：

$$k_i = \sum_{s+t=i} b_s c_t$$

那么可以得到上述多项式乘法的矩阵表示:

$$(b_0, b_1, b_2, \dots, b_{n-1}) \begin{pmatrix} c_0 & c_1 & c_2 & \cdots & c_{n-1} \\ & c_0 & c_1 & \cdots & c_{n-2} & c_{n-1} \\ & & c_0 & \cdots & c_{n-3} & c_{n-2} & c_{n-1} \\ & & & \ddots & \vdots & \vdots & \vdots & \ddots \\ & & & & c_0 & c_1 & c_2 & \cdots & c_{n-1} \end{pmatrix} = (k_0, k_1, k_2, \dots, k_{2n-2})$$

但是这只是对一般的多项式乘法生效的, 对于商环 R 中的多项式, 则还需要对 $(k_0, k_1, \dots, k_{2n-2})$ 进行进一步处理, 已知在 R 中有:

$$x^n = -(a_0 + a_1x + \cdots + a_{n-1}x^{n-1})$$

所以对 x^{n+1} , 有:

$$\begin{aligned} x^{n+1} &= -(a_0x + a_1x^2 + \cdots + a_{n-2}x^{n-1} + a_{n-1}x^n) \\ &= -[a_0x + a_1x^2 + \cdots + a_{n-2}x^{n-1} - a_{n-1}(a_0 + a_1x + \cdots + a_{n-1}x^{n-1})] \end{aligned}$$

展开就可以计算出各项系数, 同理, 有:

$$x^{n+2} = -(a_0x^2 + a_1x^3 + \cdots + a_{n-2}x^n + a_{n-1}x^{n+1})$$

将先前求出的 x^{n+1} 与 x^n 代进去即可求出 x^{n+2} , 以此类推可以求出 $x^{n+3}, x^{n+4}, \dots, x^{2n-2}$ 在 R 中的表示, 设在 R 中 $g(x) \cdot h(x) = s_0 + s_1x + s_2x^2 + \cdots + s_{n-1}x^{n-1}$, 有如下关系:

$$(k_0, k_1, k_2, \dots, k_{2n-2}) \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ & & & 1 & \\ t_{n,0} & t_{n,1} & \cdots & t_{n,n-1} & \\ t_{n+1,0} & t_{n+1,1} & \cdots & t_{n+1,n-1} & \\ \vdots & \vdots & & \vdots & \\ t_{2n-2,0} & t_{2n-2,1} & \cdots & t_{2n-2,n-1} & \end{pmatrix} = (s_0, s_1, \dots, s_{n-1})$$

其中 $t_{n,0}$ 到 $t_{2n-2,n-1}$ 需要通过前面所说步骤进行计算, 具体代码如下:

```

1 # sage 10.4
2 MR = Matrix(ZZ, 2*n-1, n)
3 for i in range(n):
4     MR[i, i] = 1
5
6 for i in range(n, 2*n-1):
7     for j in range(i-n, n):
8         MR[i, j] = -a[j-(i-n)]
9
10    tmp = vector(ZZ, n*[0])
11    for j in range(i-n):
12        tmp2 = -a[n-1-j] * vector(ZZ, MR[i-j-1])
13        tmp += tmp2
14    for j in range(n):
15        MR[i, j] += tmp[j]
```

获得两矩阵之后只需要通过如下计算就可以得到商环 R 中多项式 $g(x) \cdot h(x)$ 的矩阵表示:

$$(b_0, b_1, b_2, \dots, b_{n-1}) \mathbf{M}_L \mathbf{M}_R = (s_0, s_1, \dots, s_{n-1})$$

其中:

$$\mathbf{M}_L = \begin{pmatrix} c_0 & c_1 & c_2 & \cdots & c_{n-1} & & \\ & c_0 & c_1 & \cdots & c_{n-2} & c_{n-1} & \\ & & c_0 & \cdots & c_{n-3} & c_{n-2} & c_{n-1} \\ & & & \ddots & \vdots & \vdots & \vdots & \ddots \\ & & & & c_0 & c_1 & c_2 & \cdots & c_{n-1} \end{pmatrix}$$

$$\mathbf{M}_R = \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ & & & 1 & \\ t_{n,0} & t_{n,1} & \cdots & t_{n,n-1} & \\ t_{n+1,0} & t_{n+1,1} & \cdots & t_{n+1,n-1} & \\ \vdots & \vdots & & \vdots & \\ t_{2n-2,0} & t_{2n-2,1} & \cdots & t_{2n-2,n-1} & \end{pmatrix}$$

事实上在 sage 中可以通过商环多项式的 ‘matrix()’ 方法来获得多项式 $g(x)$ 的乘法矩阵 $\mathbf{M} = \mathbf{M}_L \mathbf{M}_R$, 在获得这个矩阵之后, RLWE 问题就可以转换为一般的 LWE 问题了:

$$b(x) = A(x)s(x) + e(x) \Leftrightarrow \mathbf{b} = \mathbf{s}\mathbf{M} + \mathbf{e}$$

其中 $\mathbf{b}, \mathbf{s}, \mathbf{e}$ 分别为 $b(x), s(x), e(x)$ 的系数 (行) 向量. 以 NSSRound#18 Basic 的 New Year Ring2 一题为例, 题目代码如下:

```

1 from Crypto.Util.number import *
2 from random import *
3 from secret import flag
4
5 p = getPrime(128)
6 n = 64
7 assert len(flag) < n
8
9 PRp.<x> = PolynomialRing(Zmod(p))
10 f = x^n+2*x^3+0*x^2+2*x+4 #welcome to 2024!
11 PR = PRp.quo(f)
12 assert f.is_irreducible()
13
14 A = [randint(0, p) for i in range(n)]
15 E = [randint(-4, 4) for i in range(n)]
16 S = [ord(flag[i]) for i in range(len(flag))]
17
18 B = PR(A)*PR(S)+PR(E)
19 print(A)
20 print(B.list())
21 print(p)

```

可以看到这很显然是 RLWE 问题，这里的环是：

$$R = \frac{\mathbb{Z}_p[x]}{x^{64} + 2x^3 + 2x + 4}$$

可以利用前面提到的方法来构造格基并进行规约获得目标向量并获得 flag：

```

1 # sage 10.4
2 A = [...]
3 b = [...]
4 p = 171384865635734387982308861436753436427
5 n = 64
6
7 PR.<x> = ZZ[]
8 f = x^n + 2*x^3 + 2*x + 4
9 R = PR.quotient(f)
10
11 g = R(A)
12 M = g.matrix()
13
14 L = block_matrix(ZZ, 3, 3, [[p, 0, 0], [M, 1, 0], [matrix(b), 0, 1]])
15 res = L.LLL()
16
17 flag = ""
18 for i in range(n, 2*n):
19     if res[i, i] != 0:
20         flag += chr(abs(res[0, i]))
21
22 print(flag)

```

代入题目数据运行即可得到 flag.

需要注意的是，对于上述矩阵 $\mathbf{M}$ 我们并不能通过求矩阵 $\begin{pmatrix} \mathbf{M} \\ q\mathbf{I} \end{pmatrix}$ 的 HNF 来减小格的规模，故若需要加速运算可能需要使用更高效的规约算法进行规约.

4.4.6.2 决定 LWE (Decision-LWE)

4.4.7 NTRU

在了解 NTRU 之前,我们需要先了解三个概念——卷积多项式环、center-lift 以及三元多项式 (ternary polynomial):

卷积多项式环 (convolution polynomial rings)

对于一个整数 N , 秩为 N 的卷积多项式环即为如下商环:

$$R = \frac{\mathbb{Z}[x]}{(x^N - 1)}$$

类似的我们也可以定义出模 q 的卷积多项式环为如下商环:

$$R_q = \frac{\mathbb{Z}_q[x]}{(x^N - 1)}$$

可以知道卷积多项式环中每一个多项式 $a(x)$ 都有唯一的表示 (在这之后, 我们默认卷积多项式环内的多项式都是如下表示):

$$a(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{N-1}x^{N-1}$$

其中 $a_0, a_1, \dots, a_{N-1} \in \mathbb{Z}$ (或者 $\mathbb{Z}_q$, 这取决于这个卷积多项式环是一般的卷积多项式环还是模 q 的卷积多项式环) 卷积多项式环是 NTRU 密码体系的基础.

center-lift

对于 R_q 上一个多项式 $a(x)$, $a(x)$ 到 R 上的 center-lift 即为一个多项式 $a'(x)$ 满足:

$$a'(x) \equiv a(x) \pmod{q}$$

其中 $a'(x)$ 的系数 $a'_i \in (-\frac{q}{2}, \frac{q}{2}]$.

对于一个多项式 $a(x) \in R_q$, 要获得其 center-lift 有如下算法:

$$a'_i = \begin{cases} a_i & , 0 \leq a_i \leq \frac{q}{2} \\ a_i - q & , \frac{q}{2} < a_i < q \end{cases}$$

center-lift 可以通过如下代码实现:

```
1 def center_lift(Rm, R, f):
2     modulo = ZZ(Rm(list(f)).base_ring()(-1)) + 1
3     l = [ZZ(x) if x <= modulo // 2 else ZZ(x) - modulo for x in list(f)]
4     return R(l)
```

三元多项式 (ternary polynomial)

对于卷积多项式环 $R = \frac{\mathbb{Z}[x]}{(x^N-1)}$, 对于任意正数 d_1, d_2 , 若多项式 $a(x) \in R$ 满足以下条件:

1. $a(x)$ 中有 d_1 个系数为 1;
2. $a(x)$ 中有 d_2 个系数为 -1;
3. $a(x)$ 中其余系数均为 0

则称这个多项式为一个三元多项式 (ternary polynomial, 或称 trinary polynomial), 对于一个卷积多项式环 R , 其上的所有三元多项式的集合记为:

$$\mathcal{T}(d_1, d_2) = \{a(x) \in R | a(x) \text{ is a ternary polynomial}\}$$

在了解上面三个概念之后, 我们就可以进一步了解 NTRU 体系的算法细节了: 首先取整数 $N \geq 1$ 以及两个模数 p, q (其中 N 为质数, 且 N 与 q 互质, p 与 q 互质), 得到如下三个卷积多项式环:

$$R = \frac{\mathbb{Z}[x]}{(x^N-1)}, \quad R_p = \frac{(\mathbb{Z}/p\mathbb{Z})[x]}{(x^N-1)}, \quad R_q = \frac{(\mathbb{Z}/q\mathbb{Z})[x]}{(x^N-1)}$$

在此我们将消息传递的双方分别称为 Alice 和 Bob, Alice 选取公共参数 (N, p, q, d) (d 也是个整数, 公共参数的选择应满足 $q > (6d+1)p$), 并在 R 上随机选取三元多项式 $f(x) \in \mathcal{T}(d+1, d)$ 以及 $g(x) \in \mathcal{T}(d, d)$, 计算:

$$F_p \equiv f^{-1} \pmod{p}$$

$$F_q \equiv f^{-1} \pmod{q}$$

这样就可以得到一个 R_p 上的多项式 F_p 以及一个 R_q 上的多项式 F_q , 在此之后 Alice 在 R_q 上计算:

$$h(x) = F_q(x) \star g(x)$$

在这里 “ $\star$ ” 表示卷积多项式环上多项式的乘法, 对于两个卷积多项式环 R 上的多项式 $a(x), b(x)$, 其乘积定义如下:

$$a(x) \star b(x) = c(x), \quad c_k = \sum_{i+j \equiv k \pmod{N}} a_i b_j$$

由此计算出的多项式 $h(x)$ 即为**公钥**, 前面生成的 (f, F_p) 就是**私钥** (实际上私钥只需要 f , Alice 可以通过 f 来计算出 F_p). 在此之后 Bob 需要通过公钥 $h(x)$ 来加密明文 $m(x)$ (其中明文 $m(x) \in R$, 而且其中系数 m_i 满足 $-\frac{p}{2} < m_i \leq \frac{p}{2}$, 实际上 $m(x)$ 就是 R_p 上某个多项式的 center-lift), Bob 先选取一个随机三元多项式 $r(x) \in \mathcal{T}(d, d)$, 计算:

$$e(x) \equiv ph(x) \star r(x) + m(x) \pmod{q}$$

得到的多项式 $e(x)$ 就是 Bob 的密文, Alice 得到 Bob 发送的密文之后要进行解密, 则需先在 R_q 上计算:

$$a(x) \equiv f(x) \star e(x) \pmod{q}$$

然后得到 $a(x)$ 的 center-lift, 然后再在 R_p 中计算:

$$b(x) \equiv F_p(x) \star a(x) \pmod{p}$$

这样得到的 R_p 上的多项式 $b(x)$ 就是明文 $m(x)$.

4.4.7.1 NTRU 的实现

在这里通过 Sagemath¹实现一个简单的例子 [22]:

取 $(N, p, q, d) = (7, 3, 41, 2)$, 显然有 $(6d + 1)p = 39 < q$, 而且 $(p, q) = 1, (N, q) = 1$, 符合参数选择的条件, 那么可以通过如下代码构造出三个卷积多项式环:

```
1 _R = PolynomialRing(ZZ, 'x')
2 R = _R.quotient(x^N - 1, 'x')
3
4 _Rp = PolynomialRing(Zmod(p), 'x')
5 Rp = _Rp.quotient(x^N - 1, 'x')
6
7 _Rq = PolynomialRing(Zmod(q), 'x')
8 Rq = _Rq.quotient(x^N - 1, 'x')
```

对固定的卷积多项式环 R , 可以通过如下代码来获得随机三元多项式:

```
1 def T(R, N, d1, d2):
2     from random import shuffle
3     coef = [1] * d1 + [-1] * d2 + [0] * (N - d1 - d2)
4     shuffle(coef)
5     return R(coef)
```

随机选取两个 R 上的三元多项式 $f(x) \in \mathcal{T}(3, 2)$ 以及 $g(x) \in \mathcal{T}(2, 2)$ (并保证 $f(x)$ 在 R_p 以及 R_q 上均可逆), 选取的两个多项式如下:

$$f(x) = x^6 - x^4 + x^3 + x^2 - 1$$

$$g(x) = x^6 + x^4 - x^2 - x$$

然后分别在 R_p, R_q 上计算 $f(x)$ 的逆, 对于 R_m 的模数是质数的情况, 我们可以通过如下代码来求多项式 $f(x)$ 在任意模卷积多项式环 R_m 的逆:

```
1 def inv(Rm, f):
2     return Rm(list(f)).inverse()
```

但是如果环 R_m 的模数为合数的时候上述方法就不适用了, 因为在合数模下, 多项式的逆可能不存在或者计算复杂度很高, 在<https://latticehacks.cr.yp.to/ntru.html>提及了一种名为 `invertmodpowerof2` 的算法可以解决模数为 2^k 的情况², 经过修改可以得到如下版本:

```
1 def invertmodpowerof2(Rm, R, f):
2     modulo = ZZ(Rm(list(f)).base_ring()(-1)) + 1
3     assert modulo.is_power_of(2)
4     _R2 = PolynomialRing(Zmod(2), 'x')
5     R2 = _R2.quotient(x^N - 1, 'x')
6
7     g = R2(list(f)).inverse()
8     while True:
9         r = center_lift(Rm, R, Rm(list(f * g)))
10        if r == 1:
```

¹在本节中默认使用 Sagemath 10.4

²对于模数为一般合数的情况, 目前还没有找到有效的解决方案

```

11         return g
12     g = center_lift(Rm, R, Rm(list(g * (2 - r))))

```

回到例子，通过如下代码可以计算出 F_p 以及 F_q ：

```

1 Fp = inv(Rp, f)
2 Fq = inv(Rq, f)

```

可以得到：

$$F_p = x^6 + 2x^5 + x^3 + x^2 + x + 1 \in R_p$$

$$F_q = 8x^6 + 26x^5 + 31x^4 + 21x^3 + 40x^2 + 2x + 37 \in R_q$$

可以得到 (f, F_p) 作为私钥，并通过如下代码计算公钥 $h(x) = F_q(x) \star g(x)$ ：

```

1 h = Fq * Rq(list(g))

```

得到：

$$h(x) = F_q(x) \star g(x) = 20x^6 + 40x^5 + 2x^4 + 38x^3 + 8x^2 + 26x + 30 \in R_q$$

欲加密消息 $m(x) = -x^5 + x^3 + x^2 - x + 1$ ，取 R 上随机三元多项式 $r(x) = x^6 - x^5 + x - 1 \in \mathcal{T}(2, 2)$ ，通过 $e = Rq(\text{list}(p \star r \star h + m))$ 在 R_q 上计算 $e(x) \equiv pr(x) \star h(x) + m(x) \pmod{q}$ ，得到密文：

$$e(x) = 31x^6 + 19x^5 + 4x^4 + 2x^3 + 40x^2 + 3x + 25$$

综上可得参数选取以及加密全流程代码如下：

```

1 (N, p, q, d) = (7, 3, 41, 2)
2
3 _R = PolynomialRing(ZZ, 'x')
4 R = _R.quotient(x^N - 1, 'x')
5
6 _Rp = PolynomialRing(Zmod(p), 'x')
7 Rp = _Rp.quotient(x^N - 1, 'x')
8
9 _Rq = PolynomialRing(Zmod(q), 'x')
10 Rq = _Rq.quotient(x^N - 1, 'x')
11
12 def T(R, N, d1, d2):
13     from random import shuffle
14     coef = [1] * d1 + [-1] * d2 + [0] * (N - d1 - d2)
15     shuffle(coef)
16     return R(coef)
17
18 def center_lift(Rm, R, f):
19     modulo = ZZ(Rm(list(f)).base_ring()(-1)) + 1
20     l = [ZZ(x) if x <= modulo // 2 else ZZ(x) - modulo for x in list(f)]
21     return R(l)
22
23 def inv(Rm, f):
24     return Rm(f).inverse()

```

```

25
26 def invertmodpowerof2(Rm, R, f):
27     modulo = ZZ(Rm(list(f)).base_ring()(-1)) + 1
28     assert modulo.is_power_of(2)
29     _R2 = PolynomialRing(Zmod(2), 'x')
30     R2 = _R2.quotient(x^N - 1, 'x')
31
32     g = R2(list(f)).inverse()
33     while True:
34         r = center_lift(Rm, R, Rm(list(f * g)))
35         if r == 1:
36             return g
37         g = center_lift(Rm, R, Rm(list(g * (2 - r))))
38
39 # f = T(R, N, d + 1, d)
40 f = R("x^6 - x^4 + x^3 + x^2 - 1") # private key
41 # g = T(R, N, d, d)
42 g = R("x^6 + x^4 - x^2 - x")
43
44 Fp = inv(Rp, f)
45 Fq = inv(Rq, f)
46
47 h = Fq * Rq(list(g)) # public key
48
49 m = R("-x^5+x^3+x^2-x+1")
50 # r = T(R, N, d, d)
51 r = R("x^6-x^5+x-1")
52
53 e = Rq(list(p*r*h + m))

```

在获取密文之后，需要首先在 R_q 上计算 $f(x) \star e(x)$ 并获取其 center-lift 得到 $a(x)$:

```
1 a = center_lift(Rq, R, Rq(list(f * e)))
```

得到:

$$a(x) = x^6 + 10x^5 - 8x^4 - x^3 - x^2 + x - 1$$

然后在 R_p 上计算 $F_p \star a(x)$ 并获取其 center-lift 得到 $b(x)$:

```
1 b = center_lift(Rp, R, Rp(list(Fp * a)))
```

此时可以得到:

$$b(x) = -x^5 + x^3 + x^2 - x + 1$$

可以看到 $b(x) = m(x)$ ，说明此时计算出来的就是明文，如此可以得到解密流程如下:

```

1 a = center_lift(Rq, R, Rq(list(f * e)))
2 b = center_lift(Rp, R, Rp(list(Fp * a)))

```

4.4.7.2 针对 NTRU 的格攻击

在了解了 NTRU 的加解密流程以及 Sagemath 实现之后, 我们就可以进一步了解针对 NTRU 的格攻击了。让我们回到公钥计算这一步, 在这一步我们需要在 R_q 上计算 $h(x) = F_q(x) \star g(x)$, 两边乘上 $f(x)$ 可以得到:

$$f(x) \star h(x) \equiv g(x) \pmod{q}$$

与对数的处理方法类似的, 可以将上式变为:

$$f(x) \star h(x) = g(x) + q \cdot u(x)$$

其中 $u(x) \in R$, 我们想要的是私钥 $f(x)$, 那么可以构造出如下方程:

$$(f(x), -u(x)) \begin{pmatrix} 1 & h(x) \\ 0 & q \end{pmatrix} = (f(x), g(x)) \quad (4.4.7.2.1)$$

但是直接对多项式矩阵进行规约应该是不太可能的, 那么需要找到一种方法来将多项式矩阵转换为一般的矩阵, 先前提到对于两个卷积多项式环 R 上的多项式 $a(x), b(x)$, 其乘积定义为:

$$a(x) \star b(x) = c(x), \quad c_k = \sum_{i+j \equiv k \pmod{N}} a_i b_j$$

显然有:

$$c_k = (a_0, a_1, \dots, a_{N-1}) \cdot (b_k \pmod{N}, b_{k-1} \pmod{N}, \dots, b_{k-N+1} \pmod{N})^T$$

那么可以得到:

$$(c_0, c_1, \dots, c_{N-1}) = (a_0, a_1, \dots, a_{N-1}) \begin{pmatrix} b_0 & b_1 & \dots & b_{N-1} \\ b_{N-1} & b_0 & \dots & b_{N-2} \\ \vdots & \vdots & & \vdots \\ b_1 & b_2 & \dots & b_0 \end{pmatrix}$$

实际上等式右边的矩阵是 $b(x)$ 的系数的循环排列, 这样我们就可以将矩阵内的 $h(x)$ 转换为一个 $n \times n$ 的矩阵:

$$H = \begin{pmatrix} h_0 & h_1 & \dots & h_{N-1} \\ h_{N-1} & h_0 & \dots & h_{N-2} \\ \vdots & \vdots & & \vdots \\ h_1 & h_2 & \dots & h_0 \end{pmatrix}$$

同理, 由于 1 以及 q 也可以看作是仅有常数项不为 0 的多项式, 所以可以将它们分别转换为单位矩阵 I 以及 q 倍的单位矩阵 qI , 而行向量内的多项式 $f(x)$ 可以转换为其对应的系数向量 $\mathbf{f} = (f_0, f_1, \dots, f_{N-1})$, 同理 $u(x), g(x)$ 也可以转换为对应的系数向量 $\mathbf{u}$ 以及 $\mathbf{g}$, 那么我们就可以得到方程 (4.4.7.2.1) 的等价表示:

$$(\mathbf{f}, -\mathbf{u}) \begin{pmatrix} I & H \\ 0 & qI \end{pmatrix} = (\mathbf{f}, \mathbf{g})$$

其中 $2N \times 2N$ 矩阵 $\begin{pmatrix} I & H \\ 0 & qI \end{pmatrix}$ 所确定的格被称为 NTRU 格 (NTRU Lattice) 一般记为 $\mathcal{L}_h^{\text{NTRU}}$, 方便起见, 后续将其记作 $\mathcal{L}_h$, 显然向量 $(\mathbf{f}, \mathbf{g}) \in \mathcal{L}_h$, 接下来分析通过对 $\mathcal{L}_h$ 进行格基规约得到向量 $(\mathbf{f}, \mathbf{g})$ 的可能性:

因为 $\det(\mathcal{L}_h) = q^N$, 所以有:

$$\left(\frac{2}{\sqrt{4\delta - 1}} \right)^{2N-1} \sqrt{2N} \cdot |\det(\mathcal{L}_h)|^{1/2N} = \left(\frac{2}{\sqrt{4\delta - 1}} \right)^{2N-1} \sqrt{2Nq}$$

取 $\delta = \frac{3}{4}$, 有:

$$\left(\frac{2}{\sqrt{4\delta-1}}\right)^{2N-1} \sqrt{2Nq} = (\sqrt{2})^{2N-1} \cdot \sqrt{2Nq} = 2^N \sqrt{Nq}$$

而因为 $f(x), g(x)$ 都是三元多项式, 所以:

$$\|(\mathbf{f}, \mathbf{g})\| = \sqrt{4d+1} \leq 2^N \sqrt{Nq} = \left(\frac{2}{\sqrt{4\delta-1}}\right)^{2N-1} \sqrt{2N} \cdot |\det(\mathcal{L}_h)|^{1/2N}$$

很显然, $(\mathbf{f}, \mathbf{g})$ 可以通过 BKZ 算法对格 $\mathcal{L}_h$ 进行格基规约得到 (LLL 有时候不行), 通过如下代码就可以构造出格并进行规约:

```

1 L = matrix(ZZ, 2*N, 2*N)
2
3 h_coeff = [ZZ(x) for x in list(h)] # 有时候可能h要乘上一个pow(q, -1, p)
4 for i in range(N):
5     L[i, i] = 1
6     L[N + i, N + i] = q
7     for j in range(N):
8         L[i, N + j] = h_coeff[(j - i) % N]
9
10 res = L.BKZ()

```

但是目标向量并不一定在第一条, 所以需要遍历每一行, 判断前 N 个分量表示的多项式是否属于 $\mathcal{T}(d+1, d)$ 再尝试求解, 经过测试发现, 一个公钥可以通过格攻击获取多个私钥, 并且这些私钥都可以成功进行解密, 例如对于前面的例子, 假设我们只知道公钥:

$$h(x) = 20x^6 + 40x^5 + 2x^4 + 38x^3 + 8x^2 + 26x + 30$$

以及密文:

$$e(x) = 31x^6 + 19x^5 + 4x^4 + 2x^3 + 40x^2 + 3x + 25$$

通过构造格进行私钥恢复:

$$\mathcal{L}_h = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 30 & 26 & 8 & 38 & 2 & 40 & 20 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 20 & 30 & 26 & 8 & 38 & 2 & 40 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 40 & 20 & 30 & 26 & 8 & 38 & 2 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 2 & 40 & 20 & 30 & 26 & 8 & 38 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 38 & 2 & 40 & 20 & 30 & 26 & 8 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 8 & 38 & 2 & 40 & 20 & 30 & 26 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 26 & 8 & 38 & 2 & 40 & 20 & 30 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 41 \end{pmatrix}$$

代码如下:

```

1 L = matrix(ZZ, 2*N, 2*N)
2
3 h_coeff = [ZZ(x) for x in list(h)]
4 for i in range(N):
5     L[i, i] = 1
6     L[N + i, N + i] = q
7     for j in range(N):
8         L[i, N + j] = h_coeff[(j - i) % N]
9
10 res = L.BKZ()[1]
11 ff = R([ZZ(x) for x in res[:N]])
12 print(ff)
13
14 fp = inv(Rp, ff)
15 a = center_lift(Rq, R, Rq(list(ff * e)))
16 b = center_lift(Rp, R, Rp(list(fp * a)))
17 print(b)

```

得到多项式 $f' = -x^6 - x^5 + x^3 - x^2 + 1$ ，显然其不等于前面的私钥 $f = x^6 - x^4 + x^3 + x^2 - 1$ ，但是计算出来的 $b = -x^5 + x^3 + x^2 - x + 1$ 却等于前面选择的明文。

以 SCTF 2020 的 Lattice 一题为例，题目代码如下：

```

1 from base64 import b16encode
2
3 Zx.<x> = ZZ[]
4 n = 109
5 q = 2048
6 p = 3
7 Df = 9
8 Dg = 10
9 Dr = 11
10
11 def mul(f,g):
12     return (f * g) % (x^n-1)
13
14 def bal_mod(f,q):
15     g = list(((f[i] + q//2) % q) - q//2 for i in range(n))
16     return Zx(g)
17
18 def random_poly(d):
19     assert d <= n
20     result = n*[0]
21     for j in range(d):
22         while True:
23             r = randrange(n)

```

```

24         if not result[r]: break
25         result[r] = 1-2*randrange(2)
26     return Zx(result)
27
28 def inv_mod_prime(f,p):
29     T = Zx.change_ring(Integers(p)).quotient(x^n-1)
30     return Zx(lift(1 / T(f)))
31
32 def inv_mod_powerof2(f,q):
33     assert q.is_power_of(2)
34     g = inv_mod_prime(f,2)
35     while True:
36         r = bal_mod(mul(g,f),q)
37         if r == 1: return g
38         g = bal_mod(mul(g,2 - r),q)
39
40 def keygen():
41     f = random_poly(Df)
42     while True:
43         try:
44             fp = inv_mod_prime(f,p)
45             fq = inv_mod_powerof2(f,q)
46             break
47         except:
48             f = random_poly(Df)
49     g = random_poly(Dg)
50     h = bal_mod(p * mul(fq,g),q)
51     pub_key = h
52     pri_key = [f,fp]
53     return pub_key,pri_key
54
55 def encrypt(m,h):
56     r = random_poly(Dr)
57     e = bal_mod(mul(h,r) + m,q)
58     return e
59
60 if __name__ == '__main__':
61     pub_key,pri_key = keygen()
62     flag=b'SCTF{*****}'[5:-1]
63     m = Zx(list(bin(int(b16encode(flag), 16))[2:]))
64     print(m)
65     e = encrypt(m,pub_key)
66     print('pub_key=')
67     print(pub_key)

```

```

68     print('e=')
69     print(e)

```

显然这是 NTRU 加密，题目代码将 flag 转换为二进制然后转化为一个多项式 $m(x)$ ，给出了公钥 $h(x)$ 以及密文 $e(x)$ ，要恢复出明文 $m(x)$ ，则需要先恢复出私钥，通过下面的代码就可以通过格攻击恢复出明文 $m(x)$ ，并恢复出 flag：

```

1  from Crypto.Util.number import *
2
3  N = 109
4  q = 2048
5  p = 3
6
7  _R = PolynomialRing(ZZ, 'x')
8  R = _R.quotient(x^N - 1, 'x')
9
10 _Rp = PolynomialRing(Zmod(p), 'x')
11 Rp = _Rp.quotient(x^N - 1, 'x')
12
13 _Rq = PolynomialRing(Zmod(q), 'x')
14 Rq = _Rq.quotient(x^N - 1, 'x')
15
16 def center_lift(Rm, R, f):
17     modulo = ZZ(Rm(list(f)).base_ring()(-1)) + 1
18     l = [ZZ(x) if x <= modulo // 2 else ZZ(x) - modulo for x in list(f)]
19     return R(l)
20
21 def inv(Rm, f):
22     return Rm(f).inverse()
23
24 h = R("...")
25 e = R("...")
26
27 L = matrix(ZZ, 2*N, 2*N)
28
29 h_coeff = [ZZ(x) for x in list(h)]
30 for i in range(N):
31     L[i, i] = 1
32     L[N + i, N + i] = q
33     for j in range(N):
34         L[i, N + j] = h_coeff[(j - i) % N]
35
36 res = L.BKZ(blocksize=24)
37 for v in res:
38     fc = list(v[:N])
39     c0, c1, c_1 = fc.count(0), fc.count(1), fc.count(-1)

```

```

40     if c0 == 100 and c1 + c_1 == 9:
41         f = R(fc)
42         Fp = inv(Rp, f)
43         a = center_lift(Rq, R, Rq(list(f * e)))
44         b = center_lift(Rp, R, Rp(list(Fp * a)))
45         s = "".join([str(x) for x in ct])
46         pad = 8 - len(s) % 8
47         for i in range(pad + 1):
48             print(long_to_bytes(int('0' * i + s + '0' * (pad - i), 2)))
49         break

```

代入题目数据运行即可得到 flag.

4.4.8 隐子集和问题 (Hidden Subset Sum Problem) [23]

隐子集和问题 (Hidden Subset Sum Problem, HSSP)

设 M 为一整数, 设 $\alpha_1, \alpha_2, \dots, \alpha_n$ 是 $\mathbb{Z}/M\mathbb{Z}$ 上随机选取的 n 个整数, $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ 为 $\mathbb{Z}^m$ 上随机选取的 m 个 0-1 向量 (即 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \{0, 1\}^m$), 令

$$\mathbf{h} \equiv \sum_{i=1}^n \alpha_i \mathbf{x}_i \equiv \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2 + \dots + \alpha_n \mathbf{x}_n \pmod{M}$$

隐子集和问题 (Hidden Subset Sum Problem, HSSP) 即为已知整数 M 以及向量 $\mathbf{h}$, 恢复 $(\alpha_1, \alpha_2, \dots, \alpha_n)$ 以及 $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$.

在了解如何通过格方法求解 HSSP 之前, 我们需要了解正交格和完备格的概念:

正交格 (Orthogonal Lattice) 与完备格 (Complete Lattice)

对于 $\mathbb{Z}^m$ 上一个格 $\mathcal{L}$, 定义其正交格为:

$$\mathcal{L}^\perp = \{\mathbf{v} \in \mathbb{Z}^m \mid \forall \mathbf{b} \in \mathcal{L}, \langle \mathbf{v}, \mathbf{b} \rangle = 0\}$$

定义其完备格为 $\bar{\mathcal{L}} = (\mathcal{L}^\perp)^\perp$.

显然的, $\mathcal{L}$ 为 $\bar{\mathcal{L}}$ 的一个满秩子格.

求解 HSSP 的一种比较行之有效的方法就是 Nguyen-Stern 算法, 这个算法主要分为两步:

1. 从 $\mathbf{h}$ 中得到由 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ 所生成的格 $\mathcal{L}_x$ 的完备格 $\bar{\mathcal{L}}_x$.
2. 从 $\bar{\mathcal{L}}_x$ 中恢复向量 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, 从而根据 $\mathbf{h}, M, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ 恢复出 $\alpha_1, \alpha_2, \dots, \alpha_n$.

第一步

首先我们需要通过正交格攻击 (Orthogonal Lattice Attack) 来获得 $\bar{\mathcal{L}}_x$, 我们设所有模 M 下与向量 $\mathbf{h}$ 正交的向量所构成的格为:

$$\mathcal{L}_0 = \{\mathbf{u} \in \mathbb{Z}^m \mid \langle \mathbf{u}, \mathbf{h} \rangle \equiv 0 \pmod{M}\}$$

显然的, 对于 $\mathcal{L}_0$ 中任一向量 $\mathbf{u}$, 必然有:

$$\langle \mathbf{u}, \mathbf{h} \rangle \equiv \left\langle \mathbf{u}, \sum_{i=1}^n \alpha_i \mathbf{x}_i \right\rangle \equiv \sum_{i=1}^n \alpha_i \langle \mathbf{u}, \mathbf{x}_i \rangle \equiv \alpha_1 \langle \mathbf{u}, \mathbf{x}_1 \rangle + \alpha_2 \langle \mathbf{u}, \mathbf{x}_2 \rangle + \dots + \alpha_n \langle \mathbf{u}, \mathbf{x}_n \rangle \equiv 0 \pmod{M}$$

可以知道,向量 $(\langle \mathbf{u}, \mathbf{x}_1 \rangle, \langle \mathbf{u}, \mathbf{x}_2 \rangle, \dots, \langle \mathbf{u}, \mathbf{x}_n \rangle)$ 与向量 $(\alpha_1, \alpha_2, \dots, \alpha_n)$ 在模 M 下是正交的,因为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \{0, 1\}^m$, 那么如果 $\mathbf{u}$ 足够短的话,向量 $(\langle \mathbf{u}, \mathbf{x}_1 \rangle, \langle \mathbf{u}, \mathbf{x}_2 \rangle, \dots, \langle \mathbf{u}, \mathbf{x}_n \rangle)$ 也会比较短,而如果向量 $(\langle \mathbf{u}, \mathbf{x}_1 \rangle, \langle \mathbf{u}, \mathbf{x}_2 \rangle, \dots, \langle \mathbf{u}, \mathbf{x}_n \rangle)$ 比与 $(\alpha_1, \alpha_2, \dots, \alpha_n)$ 在模 M 下正交的非零向量都短的话,那么它只能是零向量,也就是有:

$$(\langle \mathbf{u}, \mathbf{x}_1 \rangle, \langle \mathbf{u}, \mathbf{x}_2 \rangle, \dots, \langle \mathbf{u}, \mathbf{x}_n \rangle) = (0, 0, \dots, 0) \Rightarrow \langle \mathbf{u}, \mathbf{x}_i \rangle = 0 (i = 1, 2, \dots, n)$$

那么我们可以通过一些方法构造 $\mathcal{L}_0$ 并进行规约, 取出约减基的前 $m - n$ 个短向量 $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{m-n}$ 可以构成 $\mathcal{L}_x^\perp$, 再求其正交格即可得到完备格 $\bar{\mathcal{L}}_x = (\mathcal{L}_x^\perp)^\perp$.

在 [23] 中给出了一种通过 $\mathbf{h}$ 来构造 $\mathcal{L}_0$ 的方法, 我们设 $\mathbf{h} = (h_1, h_2, \dots, h_n)$, 如果 $\gcd(h_1, M) = 1$, 那么对于 $\mathbf{u} = (u_1, u_2, \dots, u_m) \in \mathcal{L}_0$ 我们可以得到:

$$\langle \mathbf{u}, \mathbf{h} \rangle \equiv \sum_{i=1}^m u_i h_i \equiv u_1 h_1 + \sum_{i=2}^m u_i h_i \equiv 0 \pmod{M}$$

两边同乘 h_1^{-1} 有:

$$u_1 + \sum_{i=2}^m u_i h_1^{-1} h_i \equiv 0 \pmod{M}$$

即:

$$u_1 + \sum_{i=2}^m (u_i h_1^{-1} h_i \pmod{M}) = kM$$

那么可以得到一线性关系:

$$u_1 = kM - \sum_{i=2}^m (u_i h_1^{-1} h_i \pmod{M})$$

通过这一线性关系我们可以构造出格:

$$\mathcal{L}_0 = \begin{pmatrix} M & 0 & 0 & \dots & 0 \\ -h_1^{-1}h_2 \pmod{M} & 1 & 0 & \dots & 0 \\ -h_1^{-1}h_3 \pmod{M} & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -h_1^{-1}h_m \pmod{M} & 0 & 0 & \dots & 1 \end{pmatrix}$$

根据前面的分析可以知道这个格规约后得到的约减基的前 $m - n$ 个短向量 $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{m-n}$ 可以构成 $\mathcal{L}_x^\perp$, 在获得 $\mathcal{L}_x^\perp$ 之后, 只要求 $\mathcal{L}_x^\perp$ 的核空间即可得到 $\bar{\mathcal{L}}_x = (\mathcal{L}_x^\perp)^\perp$.

第二步

因为 $\mathcal{L}_x$ 为 $\bar{\mathcal{L}}_x$ 的一个子格, 所以 $\bar{\mathcal{L}}_x$ 中必然是包含 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ 的, 又因为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \{0, 1\}^m$, 那么直接对 $\bar{\mathcal{L}}_x$ 进行规约大概率是可以得到 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ 的, 也可以构造格:

$$\mathcal{L}'_x = \begin{pmatrix} 2\mathcal{L}_x \\ -\mathbf{e} \end{pmatrix}$$

来规约求得 $2\mathbf{x}_1 - \mathbf{e}, 2\mathbf{x}_2 - \mathbf{e}, \dots, 2\mathbf{x}_n - \mathbf{e} \in \{-1, 1\}^m$ (其中 $\mathbf{e} = (1, 1, \dots, 1)$, 要去掉规约后的格基里面全为 1 或全为 -1 的行向量, 且有部分向量求出是 $\mathbf{e} - 2\mathbf{x}_i$). 在获得 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ 之后, 只需求解方程:

$$\mathbf{h} = (\alpha_1, \alpha_2, \dots, \alpha_n) \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{pmatrix}$$

就可以恢复出 $\alpha_1, \alpha_2, \dots, \alpha_n$.

代码实现

根据上述分析，通过如下代码就可以求解 HSSP（因为格较大，所以在正交格攻击一步中使用 flatter 进行加速）

```

1  # sage 10.4
2
3  from Crypto.Util.number import *
4
5  def flatter(M):
6      from subprocess import check_output
7      from re import findall
8      global count
9      z = "[" + "]" * M.nrows() + "]"
10     ret = check_output(["flatter"], input=z.encode())
11     return matrix(M.nrows(), M.ncols(), map(int, findall(b"-?\\d+", ret)))
12
13 m = ...
14 n = ...
15 M = ...
16 h = [...]
17
18 L0 = matrix(ZZ, m, m)
19 L0[0, 0] = M
20 inv_h0 = inverse(h[0], M)
21 for i in range(1, m):
22     L0[i, 0] = -h[i] * inv_h0
23     L0[i, i] = 1
24
25 Lx_ort = matrix(ZZ, flatter(L0)[0: m-n])
26
27 Lx = Lx_ort.right_kernel(algorithm='pari').matrix()
28
29 e = matrix(ZZ, [1 for _ in range(m)])
30 L = block_matrix(ZZ, [[2*Lx], [-e]])
31
32 e = vector(ZZ, [1 for _ in range(m)])
33 tmpMat = L.BKZ()
34
35 B = []
36 space = Lx.row_space()
37 for v in tmpMat:
38     if len(set(v)) == 1:
39         continue
40     else:
41         tmp = (v + e) / 2

```

```
42
43     if tmp in space:
44         B.append(tmp)
45     elif e - tmp in space:
46         B.append(e - tmp)
47
48 L_x = matrix(Zmod(M), B)
49 H = vector(Zmod(M), h)
50
51 print(L_x.solve_left(H))
```

4.5 同源密码学

基于同源的密码学 (isogeny based cryptography) 是一个十分年轻的领域, 它是后量子密码学的一个分支, 利用椭圆曲线之间的同源映射构造密码方案。在这一节中, 我们主要关注同源密码学中的两个密钥交换方案: 超奇异同源 Diffie-Hellman (Supersingular Isogeny Diffie-Hellman, SIDH) 以及可交换超奇异同源 Diffie-Hellman (Commutative Supersingular Isogeny Diffie-Hellman, CSIDH)。这两个密钥交换方案都是基于超奇异椭圆曲线 (Supersingular Elliptic Curves) 的同源映射构造的, 具有较高的安全性和抗量子攻击能力。因为超奇异椭圆曲线以及同源映射相关知识在先前都没有介绍过, 所以在开始介绍 SIDH 和 CSIDH 之前, 我们需要先了解一些相关的基础知识。

4.5.1 前置知识 [24, 25, 26]

我们最先需要了解的是超奇异椭圆曲线的定义:

超奇异椭圆曲线 (Supersingular Elliptic Curve)

对于 $GF(p^r)$ 下的椭圆曲线 E , 若 $|E| \equiv 1 \pmod{p}$, 则称该曲线为超奇异椭圆曲线。

在同源密码学中, 我们主要使用 j -不变量 (j -invariant) 来对超奇异椭圆曲线进行分类, j -不变量是一个椭圆曲线的代数不变量, 它可以唯一地确定一个椭圆曲线的同构类。其定义如下:

j -不变量 (j -invariant)

对于一条由方程 $y^2 = x^3 + ax + b$ 定义的椭圆曲线 E , 其 j -不变量的定义为:

$$j(E) = 1728 \cdot \frac{4a^3}{4a^3 + 27b^2}$$

当且仅当两条定义在代数闭包 $\bar{k}$ 上的曲线 E 与 E' 的 j -不变量相同时这两条曲线同构, 即存在点到点之间的双射 ϕ 使得:

$$\phi : E \mapsto E'$$

对于二次扩展域 $GF(p^2)$ 上的椭圆曲线, 定义在 $GF(p^2)$ 上的超奇异 j -不变量的个数为 $\lfloor p/12 \rfloor + z$, 这些超奇异 j -不变量每一个都对应一条同构意义下的超奇异曲线, 其中 $z \in \{0, 1, 2\}$, z 的取值取决于 $p \pmod{12}$, 即:

$$z = \begin{cases} 0 & , p \equiv 1 \pmod{12} \\ 1 & , p \equiv 5 \pmod{12} \text{ or } p \equiv 7 \pmod{12} \\ 2 & , p \equiv 11 \pmod{12} \end{cases}$$

例如对于 $p = 431$, $GF(p^2)$ 中的 $\lfloor p/12 \rfloor + 2 = 37$ 个超奇异 j -不变量分别对应一条同构意义下的超奇异椭圆曲线, 这些 j -不变量如下表所示:

表 4.1: $GF(431^2)$ 中的 37 个超奇异 j -不变量

0	4	19	61	67	102	107
125	143	150	189	234	241	242
316	319	356	358	381	419	422
$81i + 65$	$350i + 65$	$209i + 118$	$222i + 118$	$42i + 141$	$389i + 141$	$87i + 190$
$344i + 190$	$67i + 304$	$364i + 304$	$132i + 315$	$299i + 315$	$106i + 379$	$325i + 379$
$125i + 426$	$306i + 426$					

在先前提到的椭圆曲线密码学中我们主要使用 Weierstrass 方程（即形如 $y^2 = x^3 + ax + b$ 的方程）所定义的曲线，这类曲线一般称为 Weierstrass 形式的椭圆曲线，而还有另外一种形式更简单的椭圆曲线是由方程 $y^2 = x^3 + Ax^2 + x$ 所定义的，这类椭圆曲线被称为 Montgomery 曲线，对于一条蒙哥马利曲线 $E : y^2 = x^3 + Ax^2 + x$ ，其 j -不变量的定义为：

$$j(E) = \frac{256(A^2 - 3)^3}{A^2 - 4}$$

任何一条椭圆曲线都可以转换为 j -不变量相同（亦即同构）的 Montgomery 曲线，在 Sagemath 中，我们可以通过 `montgomery_model` 来将一条椭圆曲线转换为 Montgomery 曲线，例如我们要将 Weierstrass 形式的曲线：

$$E : y^2 = x^3 + 312589632x + 654443578 \pmod{1912812599}$$

转换为对应的蒙哥马利曲线，则可以通过如下代码进行：

```
1 # sage 10.4
2
3 p = 1912812599
4 a = 312589632
5 b = 654443578
6
7 E = EllipticCurve(GF(p), [a, b])
8 E_M = print(E.montgomery_model())
```

可以得到其对应的蒙哥马利曲线为：

$$E_M : y^2 = x^3 + 723347356x^2 + x \pmod{1912812599}$$

在后续的讨论中，我们主要使用 Montgomery 曲线来进行同源密码学的相关讨论。

4.5.2 同源

同源（Isogeny）本质上就是一条曲线到另一条曲线或者一条曲线到自身的映射，其本质为同态（Homomorphism）映射，可以简单地表达为 $(x, y) \mapsto (f(x, y), g(x, y))$ ，其中 f, g 是两个函数。在先前我们讲到在后续的讨论中，我们主要使用 Montgomery 曲线来进行同源密码学的相关讨论，这是因为蒙哥马利曲线之间的映射可以单纯通过对 x 进行映射 $x \mapsto f(x)$ 来表示出曲线间完整的映射，其完整映射形式为：

$$(x, y) \mapsto (f(x), cyf'(x))$$

其中 c 是固定常数， f' 为 f 的导函数。例如对于一条蒙哥马利曲线 $E : y^2 = x^3 + ax^2 + x$ ，可以得到一个 E 到 E 的同构映射：

$$[2] : E \mapsto E, x \mapsto \frac{(x^2 - 1)^2}{4x(x^2 + ax + 1)}$$

事实上，我们令分母 $4x(x^2 + ax + 1) = 0$ ，可以得到三个根 $0, \alpha, \frac{1}{\alpha}$ （其中 α 满足 $\alpha^2 + a\alpha + 1 = 0$ ），我们就可以得到曲线上阶为 2 的点： $(0, 0), (\alpha, 0), (\frac{1}{\alpha}, 0)$ ，这三个点的集合就是映射 $[2]$ 的核，记为 $\ker([2])$ ，它是椭圆曲线群 E 的一个子群，同构于 $\mathbb{Z}_2 \times \mathbb{Z}_2$ ，这上面的点称为 2-torsion，事实上对于任意一条椭圆曲线都有其对应的 $\ker([2])$ 例如对于 $GF(431^2)$ 下的椭圆曲线 $E : y^2 = x^3 + x$ ，可以通过如下代码得到对应的上述映射：

```
1 # sage 10.4
2
```

```

3 p = 431
4 R.<i> = GF(p^2, modulus=[1, 0, 1])
5 a = 1
6 E = EllipticCurve(R, [1, 0])
7 print(E._multiple_x_numerator(2)) # 分式分子部分
8 print(E._multiple_x_denominator(2)) # 分式分母部分

```

可以得到:

$$[2] : E \mapsto E, x \mapsto \frac{x^4 + 429x^2 + 1}{4x^3 + 4x}$$

我们令分母 $4x^3 + 4x = 4(x^3 + x) = 0$, 得到三个解 $0, i, -i$, 据此我们就可以得到一个 E 的一个子群 $\ker([2]) = \{(0, 0), (i, 0), (-i, 0), \mathcal{O}\}$, 而且这三个点都可以确定一个二阶循环子群, 我们可以通过 Sagemath 的 `division_points` 方法对 E 的零元开二次根来达到这一目的: `E(0).division_points(2)`, 文献 [25] 中给出了这样一个图来描述 $E : y^2 = x^3 + ax^2 + x$ 的 $\ker([2])$:

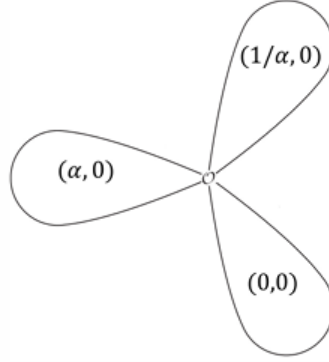


图 4.8: $E : y^2 = x^3 + ax^2 + x$ 的 $\ker([2])$

在这里每个“花瓣”对应一个二阶循环子群。相应的, 还有三倍点映射 [3], 对于蒙哥马利曲线 $E : y^2 = x^3 + ax^2 + x$, 其对应三倍点映射为:

$$[3] : E \mapsto E : x \mapsto \frac{x(x^4 - 6x^2 - 4ax^3 - 3)^2}{(3x^4 + 4ax^3 + 6x^2 - 1)^2}$$

通过令分母等于 0 我们可以得到四个根 $\beta, \delta, \zeta, \theta$, 通过这四个根我们可以得到八个阶为 3 的点: $(\beta, \gamma), (\beta, -\gamma), (\delta, \epsilon), (\delta, -\epsilon), (\zeta, \eta), (\zeta, -\eta), (\theta, \iota), (\theta, -\iota)$, 这八个点加上无穷远点可以构成子群 $\ker([3]) \simeq \mathbb{Z}_3 \times \mathbb{Z}_3$, 也可以叫做 3-torsion, 3-torsion 的结构如下图所示:

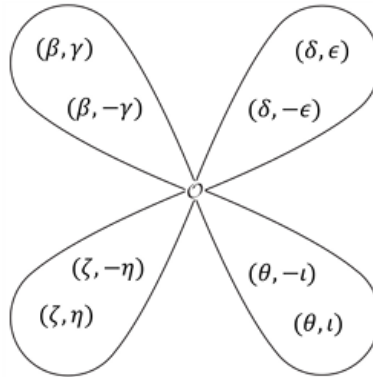


图 4.9: $E : y^2 = x^3 + ax^2 + x$ 的 $\ker([3])$

这里每个“花瓣”对应一个三阶循环子群。推广到一般情况，所有阶为 l 的点 ($p \nmid l$) 与 $\mathcal{O}$ 构成的子群就是 $\ker([l]) \simeq \mathbb{Z}_l \times \mathbb{Z}_l$ (或者称为 l -torsion)，而若 l 为质数，则可以得到 $l+1$ 个 l 阶循环子群。

在了解上面的概念之后，我们就可以给出可分同源的定义了：

可分同源 (Separable Isogeny)

对于一条椭圆曲线 E 上的椭圆曲线群以及它的一个子群 G ，可以构造出唯一的同源 $\phi : E \mapsto E'$ ，使得 $\ker(\phi) = G$ 且有 $\deg(\phi) = |G|$ ，这样的同源称为可分同源 (separable isogeny)，这样得到的曲线 E' 称为该同源的陪域 (codomain)，可以表示为 E/G 。

我们可以使用 Vélu 公式来计算可分同源的陪域以及同源映射，Vélu 公式可以接受一个椭圆曲线 E 以及它的一个有限子群 G 作为输入，输出该同源的陪域 E/G 以及同源映射 $\phi : E \mapsto E/G$ 。

以 Montgomery 曲线 $E : y^2 = x^3 + ax^2 + x$ 为例，设其上一个二阶子群为 $G = \{\mathcal{O}, (\alpha, 0)\}$ ，我们可以以 G, E 作为输入，得到映射：

$$\phi : x \mapsto \frac{x(\alpha x - 1)}{x - \alpha}$$

以及陪域 $E' : y^2 = x^3 + 2(1 - 2\alpha^2)x^2 + x$ ，这个映射被用于计算 Montgomery 曲线的 2-同源¹，对 $GF(431^2)$ 下的超奇异椭圆曲线 $E : y^2 = x^3 + (208i + 161)x^2 + x$ ，我们知道 $j(E) = 364i + 304$ ，我们可以通过如下 Sagemath 代码求得其中一个 2-同源并确保其输出的陪域为 Montgomery 曲线：

```

1 # sage 10.4
2
3 p = 431
4 R.<i> = GF(p^2, modulus=[1, 0, 1])
5 E = EllipticCurve(R, [0, 208*i+161, 0, 1, 0])
6
7 ker2 = E(0).division_points(2)
8 # [(0 : 1 : 0), (0 : 0 : 1), (350*i + 68 : 0 : 1), (304*i + 202 : 0 : 1)]
9
10 alpha = ker2[2]
11 phi = E.isogeny(alpha, model = "montgomery")
12 print(phi.rational_maps())
13 print()
14 E_ = phi.codomain()
15 print(E_)
16 print(E_.j_invariant())

```

可以得到映射 ϕ ：

$$\phi : x \mapsto \frac{(-81i + 68)x^2 - x}{x + (81i - 68)}$$

以及陪域 $E' : y^2 = x^3 + (102i + 423)x^2 + x$ ，其 j -不变量 $j(E') = 344i + 190$ ，由此我们也可以知道同源会使得 j -不变量发生变化。

Sagemath 上的同源

我们一般用 Sagemath 的 `isogeny` 方法来计算同源，其函数原型如下：

¹ 对于一个可分同源 ϕ ，设其度 $\deg(\phi) = d$ ，则称这个同源为 d -同源 (d -isogeny)

通过同源图，我们可以知道超奇异曲线的 j -不变量在同源时的变化路线。在 Sagemath 中，我们可以通过如下算法来绘制 $GF(p^2)$ 下的 $\ker([l])$ 的同源图：

```

1  # sage 10.4
2
3  def IsogenyGraph(p, l=2, vertex_size=3750, size=[20, 20]):
4      R.<i> = GF(p^2, modulus=[1,0,1])
5
6      jlist = {}
7      Elist = []
8
9      E = EllipticCurve(R, [1, 0])
10     assert E.is_supersingular()
11     jlist[E.j_invariant()] = set()
12     Elist.append(E)
13
14     while Elist:
15         tmp = Elist.pop()
16         kerl = tmp(0).division_points(l)
17         for P in kerl:
18             if P != tmp(0):
19                 phi = tmp.isogeny(P, model = "montgomery")
20                 E2 = phi.codomain()
21                 j = E2.j_invariant()
22                 # print(tmp.j_invariant(), j)
23                 if j not in jlist:
24                     jlist[j] = set()
25                 if j not in jlist[tmp.j_invariant()]:
26                     jlist[tmp.j_invariant()].add(j)
27                 Elist.append(E2)
28
29     Tab = {}
30     for x in jlist:
31         Tab[x] = list(jlist[x])
32     G = Graph(Tab)
33
34     G.set_pos(G.layout_circular())
35     G.plot(vertex_labels=True, vertex_size=vertex_size).show(figsize=size)

```

通过这个算法我们可以计算出 $GF(p^2)$ 下的 $\ker([l])$ 的同源图。

4.5.3 超奇异同源 Diffie-Hellman 密钥交换体系 (SIDH)

了解同源的基本概念之后，我们就可以介绍超奇异同源 Diffie-Hellman 密钥交换体系 (Supersingular Isogeny Diffie-Hellman, SIDH) 了。

4.5.3.1 SIDH 协议细节

SIDH 的步骤与一般的 Diffie-Hellman 密钥交换协议类似，主要分为四个部分：参数生成、Alice 侧的密钥生成、Bob 侧的密钥生成以及密钥交换。

参数生成

首先密钥交换双方（以下称为 Alice 和 Bob）协商选取模数 $p = 2^a 3^b - 1$ ，其中 $2^a \approx 3^b$ ，然后选取一条 $GF(p^2)$ 下的超奇异椭圆曲线 E ，在此之后，Alice 选取 E 上阶为 2^a 的两点 P_A, Q_A 并公开，Bob 选取 E 上阶为 3^b 的两点 P_B, Q_B 并公开（在选取的时候，需要保证 P_A, Q_A 线性无关， P_B, Q_B 线性无关）。

Alice 侧的密钥生成

Alice 随机选取秘密值 $k_A \in \{0, 1, \dots, 2^a - 1\}$ ，计算 $S_A = P_A + k_A Q_A$ ，并通过 S_A 计算同源 $\phi_A : E \mapsto E_A$ ，其中 $E_A = E / \langle S_A \rangle$ ， ϕ_A 由 a 个 2-同源组合而成，最后使用 $(E_A, \phi_A(P_B), \phi_A(Q_B))$ 作为公钥， (k_A, S_A) 作为私钥；

Bob 侧的密钥生成

Bob 随机选取秘密值 $k_B \in \{0, 1, \dots, 3^b - 1\}$ ，计算 $S_B = P_B + k_B Q_B$ ，通过 S_B 计算同源 $\phi_B : E \mapsto E_B$ ，其中 ϕ_B 由 b 个 3-同源组合而成，使用 $(E_B, \phi_B(P_A), \phi_B(Q_A))$ 作为公钥， (k_B, S_B) 作为私钥。

密钥交换

通过上述计算得到的公钥，Alice 可以计算出 $S'_A = \phi_B(P_A) + k_A \phi_B(Q_A)$ ，计算同源 $\phi'_A : E_B \mapsto E_{AB}$ ，就可以计算出 $j_{AB} = j(E_{AB})$ ，同样的，Bob 可以计算出 $S'_B = \phi_A(P_B) + k_B \phi_A(Q_B)$ ，然后计算同源 $\phi'_B : E_A \mapsto E_{BA}$ ，从而可以计算出 $j_{BA} = j(E_{BA})$ ，有 $j_{AB} = j_{BA}$ ，所以共享密钥值为 $j = j_{AB} = j_{BA}$ 。

4.5.3.2 SIDH 实例 [25]

在这里举一个简单的例子来说明 SIDH 的具体计算过程：

Alice 和 Bob 协商选取模数 $p = 2^4 3^3 - 1 = 431$ ，并选取 $GF(p^2)$ 上的一条超奇异椭圆曲线：

$$E : y^2 = x^3 + (329i + 423)x^2 + x$$

有 $j(E) = 87i + 190$ ，Alice 从中选取两个阶为 2^4 的点：

$$P_A = (100i + 248, 304i + 199), Q_A = (426i + 394, 51i + 79)$$

同时 Bob 从中选取两个阶为 3^3 的点：

$$P_B = (358i + 275, 410i + 104), Q_B = (20i + 185, 281i + 239)$$

对应 Sagemath 代码如下：

```

1 # sage 10.4
2
3 p = 2^4 * 3^3 - 1
4 R.<i> = GF(p^2, modulus=[1, 0, 1])
5 a = 329*i + 423
6
7 E = EllipticCurve(R, [0, a, 0, 1, 0])

```

```

8
9 PA = E(100*i+248, 304*i+199)
10 QA = E(426*i+394, 51*i+79)
11 assert PA.order() == QA.order() == 2^4
12
13 PB = E(358*i+275, 410*i+104)
14 QB = E(20*i+185, 281*i+239)
15 assert PB.order() == QB.order() == 3^3

```

然后 Alice 从 $\{0, 1, \dots, 2^4 - 1\}$ 中选择 $k_A = 11$, 据此计算 $S_A = P_A + k_A Q_A = (271i + 79, 153i + 430)$, 我们可以通过如下算法计算同源路径从而得到 S_A 对应的同源:

Algorithm: 同源路径算法 (Alice 侧)

Input: 椭圆曲线 E , 点 S_A, P_B, Q_B

Output: 同源 ϕ_A , 公钥 (E', P', Q')

$S', P', Q', E' \leftarrow S_A, P_B, Q_B, E$

$\phi_A \leftarrow e$ **for** $i = a - 1$ **to** 0 **do**

$R \leftarrow 2^i S'$

$\phi' \leftarrow \text{Isogeny}(E', R)$

$S', E', P', Q' \leftarrow \phi'(S'), \phi'(E'), \phi'(P'), \phi'(Q')$

$\phi_A \leftarrow \phi' \circ \phi_A$

end

return $\phi_A, (E', P', Q')$

其中 e 表示恒等映射, 对应 Sagemath 代码如下:

```

1 # sage 10.4
2
3 phis = []
4
5 SA_, EA_, PB_, QB_ = SA, E, PB, QB
6
7 for e in range(a-1, -1, -1):
8     RA_ = SA_ * 2^e
9     phi = EA_.isogeny(-RA_, model="montgomery")
10    phis.append(phi)
11    SA_, EA_, PB_, QB_ = phi(SA_), phi.codomain(), phi(PB_), phi(QB_)
12
13 phiA = phis[-1]
14 for i in range(len(phis) - 2, -1, -1):
15     phiA = phiA * phis[i]

```

计算得到 $E_A : y^2 = x^3 + (128i + 19)x^2 + x$, $\phi_A(P_B) = (130i + 170, 428i + 290)$, $\phi_A(Q_B) = (235i + 209, 126i + 15)$.

同理, Bob 在 $\{0, 1, \dots, 3^3 - 1\}$ 中选择 $k_B = 2$, 计算 $S_B = P_B + k_B Q_B = (122i + 309, 291i + 374)$, 那么我们可以通过如下算法计算出 S_B 对应的同源:

Algorithm: 同源路径算法 (Bob 侧)**Input:** 椭圆曲线 E , 点 S_B, P_A, Q_A **Output:** 同源 ϕ_B , 公钥 (E', P', Q') $S', P', Q', E' \leftarrow S_B, P_A, Q_A, E$ $\phi_B \leftarrow e$ **for** $i = b - 1$ **to** 0 **do** $R \leftarrow 3^i S'$ $\phi' \leftarrow \text{Isogeny}(E', R)$ $S', E', P', Q' \leftarrow \phi'(S'), \phi'(E'), \phi'(P'), \phi'(Q')$ $\phi_B \leftarrow \phi' \circ \phi_B$ **end****return** $\phi_B, (E', P', Q')$ 其中 e 表示恒等映射, 对应 Sagemath 代码如下:

```

1 # sage 10.4
2
3 phis = []
4 SB_, EB_, PA_, QA_ = SB, E, PA, QA
5
6 for e in range(b-1, -1, -1):
7     RB_ = SB_ * 3^e
8     phi = EB_.isogeny(RB_, model="montgomery")
9     phis.append(phi)
10    SB_, EB_, PA_, QA_ = phi(SB_), phi.codomain(), phi(PA_), phi(QA_)
11
12 phiB = phis[-1]
13 for i in range(len(phis) - 2, -1, -1):
14     phiB = phiB * phis[i]
```

计算得到 $E_B : y^2 = x^3 + (329i+8)x^2 + x$, $\phi_B(P_A) = (160i+421, 246i+252)$, $\phi_B(Q_A) = (119i+14, 246i+138)$.

这样双方的公钥均已经计算出来, 最后 Alice 就可以通过自己的私钥计算出:

$$S'_A = \phi_B(P_A) + k_A \phi_B(Q_A)$$

再通过上述算法计算出 Alice 侧 S'_A 对应的同源的陪域 $E_A / \langle S'_A \rangle$, 那么这个陪域的 j -不变量就是共享密钥, 同样的, Bob 可以通过自己的私钥计算出:

$$S'_B = \phi_A(P_B) + k_B \phi_A(Q_A)$$

再通过上述算法计算出 Bob 侧 S'_B 对应的同源的陪域 $E_B / \langle S'_B \rangle$, 那么这个陪域的 j -不变量就是共享密钥, 可以通过下列代码计算出共享密钥:

```

1 # sage 10.4
2
3 SAB = phiA(PB) + kB * phiA(QB)
4 EAB = EA_.isogeny(SAB, model="montgomery").codomain()
5 Bob_shared_secret = EAB.j_invariant()
6
7 print(Bob_shared_secret)
```

```

8 SBA = phiB(PA) + kA * phiB(QA)
9 EBA = EB_.isogeny(SBA, model="montgomery").codomain()
10 Alice_shared_secret = EBA.j_invariant()
11
12 print(Alice_shared_secret)

```

可以计算出共享密钥为 234，整理并优化代码后可以得到 SIDH 的完整实现代码如下：

```

1 # sage 10.4
2
3 a = 4
4 b = 3
5 p = 2^a * 3^b - 1
6 R.<i> = GF(p^2, modulus=[1, 0, 1])
7 A = 329*i + 423
8
9 E = EllipticCurve(R, [0, A, 0, 1, 0])
10
11 PA = E(100*i+248, 304*i+199)
12 QA = E(426*i+394, 51*i+79)
13
14 PB = E(358*i+275, 410*i+104)
15 QB = E(20*i+185, 281*i+239)
16 assert PA.order() == QA.order() == 2^4 and PB.order() == QB.order() == 3^3
17
18 kA = 11
19 SA = PA + kA * QA
20 phiA = E.isogeny(SA, model="montgomery", algorithm="factored")
21 EA = phiA.codomain()
22 PB_ = phiA(PB)
23 QB_ = phiA(QB)
24
25 kB = 2
26 SB = PB + kB * QB
27 phiB = E.isogeny(SB, model="montgomery", algorithm="factored")
28 EB = phiB.codomain()
29 PA_ = phiB(PA)
30 QA_ = phiB(QA)
31
32 SAB = phiA(PB) + kB * phiA(QB)
33 EAB = EA.isogeny(SAB, model="montgomery", algorithm="factored").codomain()
34 Bob_shared_secret = EAB.j_invariant()
35
36 SBA = phiB(PA) + kA * phiB(QA)
37 EBA = EB.isogeny(SBA, model="montgomery", algorithm="factored").codomain()
38 Alice_shared_secret = EBA.j_invariant()

```

```

39
40 assert Bob_shared_secret == Alice_shared_secret
41 print(Alice_shared_secret)

```

或者可以将封装公钥和计算私钥封装为函数：

```

1 # sage 10.4
2
3 def gen_public_key(E, P1, Q1, s, P2, Q2):
4     S = P1 + s*Q1
5     phi = E.isogeny(S, model='montgomery', algorithm="factored")
6     return phi, (phi.codomain(), phi(P2), phi(Q2))
7
8 def gen_shared_secret(E, P, Q, s):
9     S = P + s*Q
10    phi1 = E.isogeny(S, model='montgomery', algorithm="factored")
11    return phi1.codomain().j_invariant()

```

4.5.3.3 SIDH 的安全性以及攻击方案 [28]

理论上 SIDH 的安全性主要依赖于通过公钥中给出的曲线来求出它对应的同源是困难的，但是在 2022 年 SIDH 就被宣布完全破解，攻击代码在 Github 上开源：<https://github.com/GiacomoPope/Castryck-Decru-SageMath>。

如果我们要使用上述开源项目对给定 SIDH 体系进行攻击，我们仅需要构造一个初始曲线（一般为方程 $y^2 = x^3 + x$ 或 $y^2 = x^3 + 6x^2 + x$ 所定义的超奇异椭圆曲线）上的自同态 $[2i]$ 使得 $[2i] \circ [2i] = [-4]$ （实际上就是一个 $[2i]$ 倍点映射），但是 Sagemath 并不能直接给 $GF(p^2)$ 上的椭圆曲线的点乘上 $2i$ ，所以我们需要自行针对特定曲线构造 $[2i]$ ，在这里仅介绍 $E_0 : y^2 = x^3 + x$ 和 $E_6 : y^2 = x^3 + 6x^2 + x$ 这两条曲线上的 $[2i]$ 映射。

首先是曲线 $E_0 : y^2 = x^3 + x$ ，在文献 [28] 中提到该曲线上的自同态映射 $[2i]$ 为：

$$[2i] : (x, y) \mapsto [2](-x, iy)$$

我们可以通过下述 Sagemath 代码来实现这个自同态映射：

```

1 # sage 10.4
2
3 def two_i(P):
4     E = P.curve()
5     if P == E(0):
6         return E(0)
7     else:
8         x, y = P.xy()
9         return 2*E(-x, i*y)

```

而对于 $E_6 : y^2 = x^3 + 6x^2 + x$ ，因为存在一个 2-同源 $\phi : E_6 \rightarrow E_0$ ，根据文献 [28]，我们可以利用这个同源来构造 $[2i]$ 映射。在此之前我们需要了解对偶同源的概念：

对偶同源 (Dual Isogeny)

设 $\phi: E \rightarrow E'$ 是一个 d -同源 (其中 $d > 0$)，那么必然存在唯一同源 $\hat{\phi}: E' \rightarrow E$ 满足：

$$\hat{\phi} \circ \phi = [d]_E, \quad \phi \circ \hat{\phi} = [d]_{E'}$$

其中 $[d]_E$ 和 $[d]_{E'}$ 分别表示 E 和 E' 上的 d 倍点映射，那么 $\hat{\phi}$ 称为 ϕ 的对偶同源 (dual isogeny)。

那么对于上述 2-同源 $\phi: E_6 \rightarrow E_0$ ，我们可以得到其对偶同源 $\hat{\phi}: E_0 \rightarrow E_6$ ，再结合 E_0 上的 $[i]$ 映射：

$$[i]_{E_0}: E_0 \rightarrow E_0: (x, y) \mapsto (-x, iy)$$

可以得到 E_6 上的 $[2i]$ 映射：

$$[2i]_{E_6} = \hat{\phi} \circ [i]_{E_0} \circ \phi$$

可以验证：

$$\begin{aligned} [2i]_{E_6} \circ [2i]_{E_6} &= \hat{\phi} \circ [i]_{E_0} \circ \phi \circ \hat{\phi} \circ [i]_{E_0} \circ \phi \\ &= \hat{\phi} \circ [i]_{E_0} \circ [2]_{E_0} \circ [i]_{E_0} \circ \phi \\ &= \hat{\phi} \circ [-2]_{E_0} \circ \phi \\ &= [-2]_{E_6} \circ \hat{\phi} \circ \phi \\ &= [-2]_{E_6} \circ [2]_{E_6} \\ &= [-4]_{E_6} \end{aligned}$$

上述推导说明了 $[2i]_{E_6}$ 的本质，实际上在开源代码中给出了如下 E_6 上的 $[2i]$ 映射的 Sagemath 实现代码：

```
1 def generate_distortion_map(E):
2     if E.a_invariants() != (0,6,0,1,0):
3         raise NotImplementedError
4     return E.isogeny(E.lift_x(ZZ(1)), codomain=E)
```

直接使用即可。在这里以 CryptoCTF 2023 的 Shevid 一题为例，题目给出代码如下：

```
1 #!/usr/bin/env sage
2
3 from Crypto.Util.number import *
4 from Crypto.Cipher import AES
5 from hashlib import md5
6 from flag import flag
7
8 def gen_param(B):
9     while True:
10         a = randint(B >> 1, B)
11         b = randint(B >> 2, B >> 1)
12         p = 2**a * 3**b - 1
13         if is_prime(p):
14             return a, b
15
16 def gen_dmap(E):
17     return E.isogeny(E.lift_x(ZZ(1)), codomain = E)
```

```

18
19 def gen_tpt(E, a, b):
20     P, Q = [((p + 1) // 2**a) * _ for _ in E.gens()]
21     R, S = [((p + 1) // 3**b) * _ for _ in E.gens()]
22     return P, Q, R, S
23
24 def keygen(EC, b, P, Q, R, S):
25     skey = randint(1, 3**b)
26     T = R + skey * S
27     phi = EC.isogeny(T, algorithm = "factored")
28     _phi_dom, _phi_P, _phi_Q = phi.codomain(), phi(P), phi(Q)
29     return skey, _phi_dom, _phi_P, _phi_Q
30
31 a, b = gen_param(190)
32 p = 2**a * 3**b - 1
33
34 F.<x> = GF(p^2, modulus = x**2 + 1)
35 EC = EllipticCurve(F, [0, 6, 0, 1, 0])
36 P, Q, R, S = gen_tpt(EC, a, b)
37
38 print(f'P = {P.xy()}')
39 print(f'Q = {Q.xy()}')
40 print(f'R = {R.xy()}')
41 print(f'S = {S.xy()}')
42
43 skey, _phi_dom, _phi_P, _phi_Q = keygen(EC, b, P, Q, R, S)
44
45 print(f'_phi_dom = {_phi_dom}')
46 print(f'_phi_P = {_phi_P.xy()}')
47 print(f'_phi_Q = {_phi_Q.xy()}')
48
49 key = md5(long_to_bytes(skey)).digest()
50 iv = md5(str(skey).encode()).digest()
51
52 cipher = AES.new(key, AES.MODE_CFB, iv=iv)
53 enc = cipher.encrypt(flag)
54
55 print(f'enc = {enc.hex()}')

```

这里给出了一个初始曲线为 $E_6: y^2 = x^3 + 6x^2 + x$ 的 SIDH 的实例，并且给出了 Alice 的公开点 P, Q 和 Bob 的公钥 $(\phi(E), \phi(P), \phi(Q))$ ，我们需要通过这些信息去还原 Bob 的私钥 k_B ，从而获得 flag，那么我们就可以使用上述开源项目进行攻击，具体攻击代码如下：

```

1 # sage 10.4
2
3 from sage.all import *

```

```

4 from public_values_aux import *
5 from Crypto.Util.number import *
6 from Crypto.Cipher import AES
7 from hashlib import md5
8
9 load('castryck_decru_shortcut.sage')
10
11 p = ...
12 Fd.<i> = GF(p^2, modulus=[1, 0, 1])
13 E = EllipticCurve(Fd, [0, 6, 0, 1, 0])
14 E.set_order((p+1)^2)
15
16 a = 142
17 b = 69
18
19 a2 = 6
20 a4 = 2070374075904221348548347954672740119972290047985052548426161483408084160515*i
    +896749506444795652787374405713981306103783874504413776724865996633074195878
21 a6 = 2497300913991521538985990865799426081199023429830552981773916386651958830387*i
    +4243320791854592301388975795466391442631117041175807728844738724691845270777
22
23 _phi_dom = EllipticCurve(Fd, [0, a2, 0, a4, a6])
24
25 _phi_P = _phi_dom(...)
26 _phi_Q = _phi_dom(...)
27
28 P2 = E(...)
29 Q2 = E(...)
30 P3 = E(...)
31 Q3 = E(...)
32 two_i = generate_distortion_map(E)
33 check_torsion_points(E, a, b, P2, Q2, P3, Q3)
34
35 enc = "..."
36
37 recovered_key = CastryckDecruAttack(E, P2, Q2, _phi_dom, _phi_P, _phi_Q, two_i,
    num_cores=1)
38
39 key = md5(long_to_bytes(int(recovered_key))).digest()
40 iv = md5(str(recovered_key).encode()).digest()
41
42 cipher = AES.new(key, AES.MODE_CFB, iv=iv)
43 flag = cipher.decrypt(bytes.fromhex(enc))
44

```

```
45 print(flag)
```

上述代码中的 P_2, Q_2, P_3, Q_3 分别对应题目代码中的 P, Q, R, S （即 Alice 的公开点 P, Q 以及 Bob 的公开点 R, S ）

附录 A

附录

参考文献

- [1] Joseph H. Silverman. 数论概论（第四版）[M]. 北京：机械工业出版社, 2020.
- [2] 闵嗣鹤, 严士健. 初等数论（第四版）[M]. 北京：高等教育出版社, 2020.
- [3] 张苍. 九章算术 [M]. 邹涌, 译解. 重庆：重庆出版社, 2016.
- [4] Michael O Rabin. Probabilistic algorithm for testing primality[J]. Journal of Number Theory, 1980, 12(1):128-138. [DOI: 10.1016/0022-314X(80)90084-0]
- [5] 杨子胥. 近世代数（第四版）[M]. 北京：高等教育出版社, 2020.
- [6] Christof Parr, Jan Pelzl. 深入浅出密码学——常用加密技术原理及应用 [M]. 马小婷, 译. 北京：清华大学出版社, 2012.
- [7] Coppersmith, D. Small Solutions to Polynomial Equations, and Low Exponent RSA Vulnerabilities[J]. Cryptology 10, 233-260 (1997). [DOI: 10.1007/s001459900030]
- [8] Coppersmith, D. Finding a Small Root of a Univariate Modular Equation[J]. Advances in Cryptology — EUROCRYPT' 96. Lecture Notes in Computer Science, vol 1070. Springer, Berlin, Heidelberg, 1996 [DOI: 10.1007/3-540-68339-9_14]
- [9] Joseph Surin, Shaanan Cohney. A Gentle Tutorial for Lattice-Based Cryptanalysis[J]. Cryptology ePrint Archive, Paper 2023/032, 2023. [URL: <https://eprint.iacr.org/2023/032>]
- [10] M. J. Wiener. Cryptanalysis of short RSA secret exponents[J]. IEEE Transactions on Information Theory, vol. 36, no. 3, pp. 553-558, 1990 [DOI: 10.1109/18.54902]
- [11] Coppersmith, D., Franklin, M., Patarin, J., Reiter, M. Low-Exponent RSA with Related Messages[J]. Advances in Cryptology — EUROCRYPT '96. Lecture Notes in Computer Science, vol 1070. Springer, Berlin, Heidelberg, 1996 [DOI: 10.1007/3-540-68339-9_1]
- [12] L. Adleman, K. Manders, G. Miller. On taking roots in finite fields[J]. 8th Annual Symposium on Foundations of Computer Science (sfcs 1977), pp. 175-178. Providence, RI, USA, 1977 [DOI: 10.1109/SFCS.1977.18]
- [13] Lenstra, A.K., Lenstra, H.W., Lovász, L. Factoring polynomials with rational coefficients[J]. Math. Ann. 261, 515-534, 1982 [DOI: 10.1007/BF01457454]
- [14] Babai, L. On Lovász' lattice reduction and the nearest lattice point problem[J]. Combinatorica 6, 1-13, 1986 [DOI: 10.1007/BF02579403]
- [15] Ravi Kannan, Minkowski's Convex Body Theorem and Integer Programming[J]. Mathematics of Operations Research 12(3):415-440, 1987 [DOI: 10.1287/moor.12.3.415]

- [16] Pan, Y., Zhang, F. Solving low-density multiple subset sum problems with SVP oracle[J]. *J Syst Sci Complex* 29, 228–242, 2016 [DOI: 10.1007/s11424-015-3324-9]
- [17] Coster, M.J., Joux, A., LaMacchia, B.A. et al. Improved low-density subset sum algorithms[J]. *Comput Complexity* 2, 111–128, 1992 [DOI: 10.1007/BF01201999]
- [18] Schnorr, C.P., Euchner, M. Lattice basis reduction: Improved practical algorithms and solving subset sum problems[J]. *Mathematical Programming* 66, 181–199, 1994 [DOI: 10.1007/BF01581144]
- [19] Hlaváč, M., Rosa, T. Extended Hidden Number Problem and Its Cryptanalytic Applications[J]. *Selected Areas in Cryptography(SAC 2006)*, 114–133, 2006 [DOI: 10.1007/978-3-540-74462-7_9]
- [20] Steven D. Galbraith, Shishay W. Gebregiyorgis, and Sean Murphy. Algorithms for the Approximate Common Divisor Problem[J]. *Cryptology ePrint Archive*, Paper 2016/215, 2016. [URL: <https://eprint.iacr.org/2016/215>]
- [21] Wang, Y., Aono, Y., Takagi, T. An Experimental Study of Kannan’ s Embedding Technique for the Search LWE Problem[J]. *Information and Communications Security*, pp. 541–553. 2018. [DOI: 10.1007/978-3-319-89500-0_47]
- [22] Jeffrey Hoffstein, Jill Pipher, Joseph H. Silverman. *An Introduction to Mathematical Cryptography*[M]. New York: Springer, 2008. [DOI: 10.1007/978-1-4939-1711-2]
- [23] Jean-Sébastien Coron, Agnese Gini. A Polynomial-Time Algorithm for Solving the Hidden Subset Sum Problem[J]. *Cryptology ePrint Archive*, Paper 2020/461, 2020. [URL: <https://eprint.iacr.org/2020/461>]
- [24] Luca De Feo. *Mathematics of Isogeny Based Cryptography*[J]. *CoRR*, 2017. [DOI: 10.48550/arXiv.1711.04062]
- [25] Craig Costello. Supersingular isogeny key exchange for beginners[J]. *Cryptology ePrint Archive*, Paper 2019/1321, 2019. [URL: <https://eprint.iacr.org/2019/1321>]
- [26] Dale Husemöller. *Elliptic Curves*[M]. New York: Springer, 2004.
- [27] Daniel J. Bernstein, Luca De Feo, Antonin Leroux, Benjamin Smith. Faster computation of isogenies of large prime degree[J]. *ANTS XIV, Open Book Series Vol. 4, No. 1*, 2020. [DOI: 10.48550/arXiv.2003.10118]
- [28] Wouter Castryck, Thomas Decru. An efficient key recovery attack on SIDH[J]. *Cryptology ePrint Archive*, Paper 2022/975, 2022. [URL: <https://eprint.iacr.org/2022/975>]
- [29] József Vass. A Generalization of Euler’s Criterion to Composite Moduli[J]. *arXiv*, 2015. [DOI: 10.48550/arXiv.1507.00098]